Assessment Report for Semantic Data Technologies MOD004979

Assessment Title:
Semantic Modelling for Supermarket Warehouse Dataset

Team:



Table of Contents

1. Introduction ***3***

2. Concept and Requirement Analysis ***3***

2.1 Aim of the Project 3

2.2 Gap in the Knowledge (Rationale) 3

2.3 List of Questions 4

3. Design ***6***

3.1 Ontology Summary 6

3.2 Classes 6

3.3 Properties 18

Object Properties: 18

Data Properties: 27

3.4 Individuals 36

3.5 Axioms 61

4. Requirements Mapping ***63***

4.1 Mapping Table 63

5. Evaluation and Use ***65***

5.1 Use case 1 65

5.2 Use case 2 66

5.3 Use case 3 66

5.4 Use case 4 67

5.5 Use case 5 67

6. Implementation ***69***

6.1 Connection to the Dataset: 69

6.2 NLP Techniques: 69

6.3 Overall Code: 70

6.4 Code Validation: 77

7. Conclusion and Future Work ***83***

8. References ***83***

1. Introduction

Supermarkets are an important part of modern society. They offer customers a wide range of products and services to meet their daily needs. Managing such a large-scale operation requires careful coordination of inventory, employees, vendors, brands and customer interactions. Warehouse systems in supermarkets play an important role in ensuring product availability, efficient stock management and smooth supply chain processes.

Increasing demand for efficient and effective techniques to implement such models makes semantic web technologies a potential solution to create comprehensive supermarket warehousing systems. Particularly, ontologies provide a structured way of knowledge representation by defining entities, attributes and its relationships, also it offers methods for advanced querying, analysis and getting insights from accessed data.

In the context of a supermarket warehouse system, this project's semantic ontology will provide with comprehensive framework to replicate various parts of the supermarket warehousing system, for instance product categories (Food, Drink, Electronics and etc), staff roles (Team members, Managers, Team Leaders), Brands and others. Furthermore, the ontology developed in this project will provide these elements as well as with their interconnections and help to identify the different insights across the available data. For example, it will help to answer various questions such as identifying out-of-stock items or analyzing vendor and brand contributions to the inventory.

2. Concept and Requirement Analysis

2.1 Aim of the Project

The primary objective of this project is to design and develop a semantic ontology that models and enhances the operations of a supermarket warehouse system, with ability to answer defined questions by using SPARQL queries and Natural Language Processing (NLP) techniques. The specific goals are as follows:

- To implement comprehensive semantic ontology using Protégé software for supermarket warehousing system with specific entities, classes, subclasses and their relationships to replicate real-world models of supermarket warehousing systems.
- To create predefined questions that will be answered by implemented warehousing ontology and map them with corresponding classes, data and object properties.
- To define data and object properties which will show interactions between classes like products, people or vendors. Furthermore, to populate created ontology with individuals to represent real-world data.
- To implement SPARQL queries in Fuseki endpoint to answer predefined questions such as cheapest products in bakery products or expensive products in certain categories of products.
- To develop functional code using Python and NLP techniques to map natural language questions to the SPARQL queries accurately and provide users with the opportunity to interact with the system effectively.
- To validate the written Python code and to ensure that it is working in a correct way with corresponding screenshots of the code workflow.

2.2 Gap in the Knowledge (Rationale)

Gap in the Knowledge means what is not known, what is left out, or what big questions have not been answered in a study area. When a plan for a supermarket warehouse is analysed and be compared to the ontology for this project, some of the missing parts of what is known cover these points:

- **Product Pricing and discounting:**

Gap: The system may not fully cover complex price plans, such as price cuts, deals, or changes in prices for different seasons or for both members and those who are not members.

Solution: Build more clear data features and links for various price forms (like past prices, seasonal prices, deals) and add them to the system.

- **Customer behaviour and preferences:**

Gap: How all types of the customers act might not be seen (like what they buy, what they enjoy, and their loyalty) from what is implemented now. This lack makes it hard to market to them in a personal way or give good suggestions.

Solution: Add ways to check what customers like, track their buying history, and see who is in the loyalty program to the things known. This helps make their shopping better and the designed marketing smarter.

- **Real-Time Order Tracking:**

Gap: The ontology might not show live order work and status shifts, like when things are sent or get to you, or if there are changes in when they will be delivered.

Solution: Make the setup better to add time details and dynamic statuses for getting orders done, such as "order done," "shipped," or "delivered."

- **Location and Store Layout:**

Gap: The ontology might not account for hard shop designs, many rows, or ways goods are sorted in varied shop spots.

Solution: Make the set-up better by adding data traits for row control, where goods are put inside the shop, and the levels of different shop places.

- **Supply Chain and Vendor Relations:**

Gap: The links between vendors, suppliers, and supermarkets might not be shown well. This includes how items are found, brought in, and kept track of in the supermarket's chain.

Solution: Put in more info on relationships and classes linked to managing the chain, like how well suppliers do, deals with vendors, or times for deliveries.

2.3 List of Questions

No.	Questions
1	Which are the 2 cheapest Bakery products?
2	How many products are sold in supermarket_1?
3	Which product has the fourth most difference between the prices of non-member and member?
4	What is the total amount of the most expensive shopping that has a value smaller than 50 that a non-member customer did?
5	In which aisles does Milk is placed?
6	List how many products that have stocks(not zero) according to their categories.
7	Member_1 bought which products?
8	List all the Products of those Vendors who have supplied at least 3 Products to the Supermarket
9	What Members of Staff work during Shift_2 ?
10	What are the top 3 most expensive products that belongs to Samsung brand ?
11	What is the location of warehouse_1?
12	List all the Products of those Aisles which displays more than 3 unique items.
13	List the supermarkets that opened before 2000?
14	Which staff member works in supermarket_4?
15	What is the highest revenue of a supermarket?
16	What products are sold by vendor_2?
17	What is the most expensive item in a bakery for non-members?
18	List orders that have tobacco products?
19	What is the average number of staff in the shift?
20	Which warehouse is responsible for providing drinks of Coca-Cola?
21	What is the average difference in member and non-member prices of packed foods?
22	How many products in the “Packed food” category are out of stock now?

23	Which supermarket has the most number of customers?
24	Which “Electronics” products need more than 1000 Watts?

Table 1: *List of Questions*

3. Design

3.1 Ontology Summary

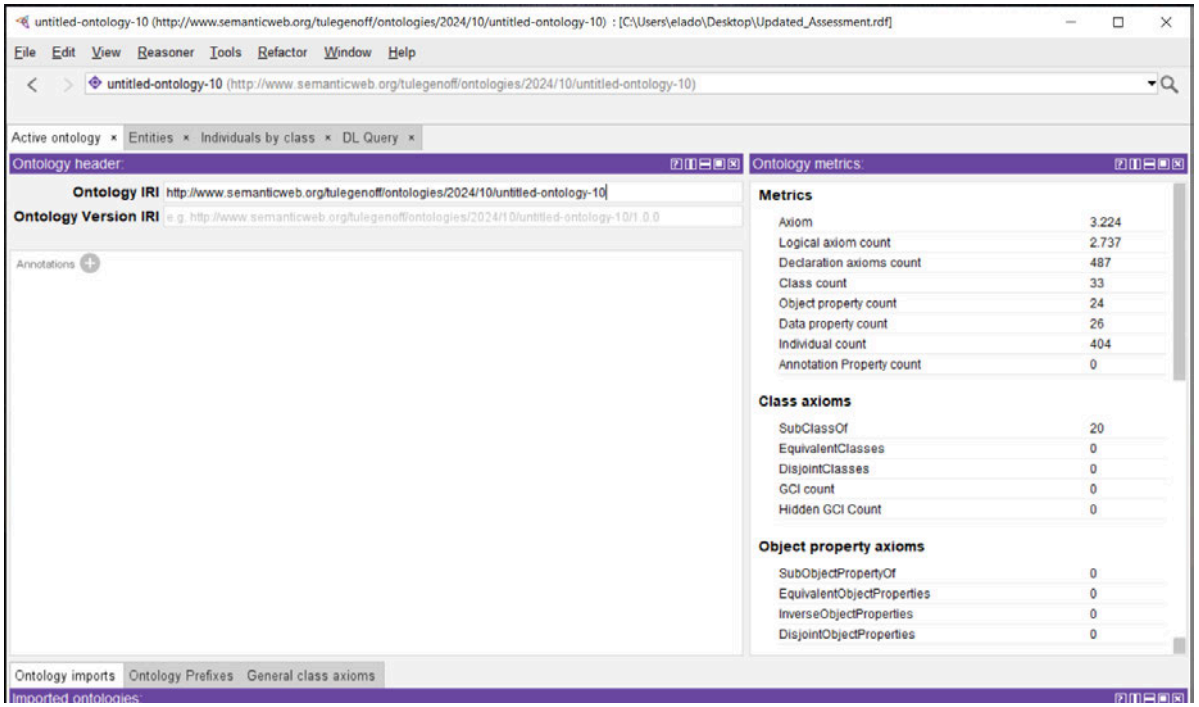


Figure 1.1: Supermarket Warehouse Ontology Overview

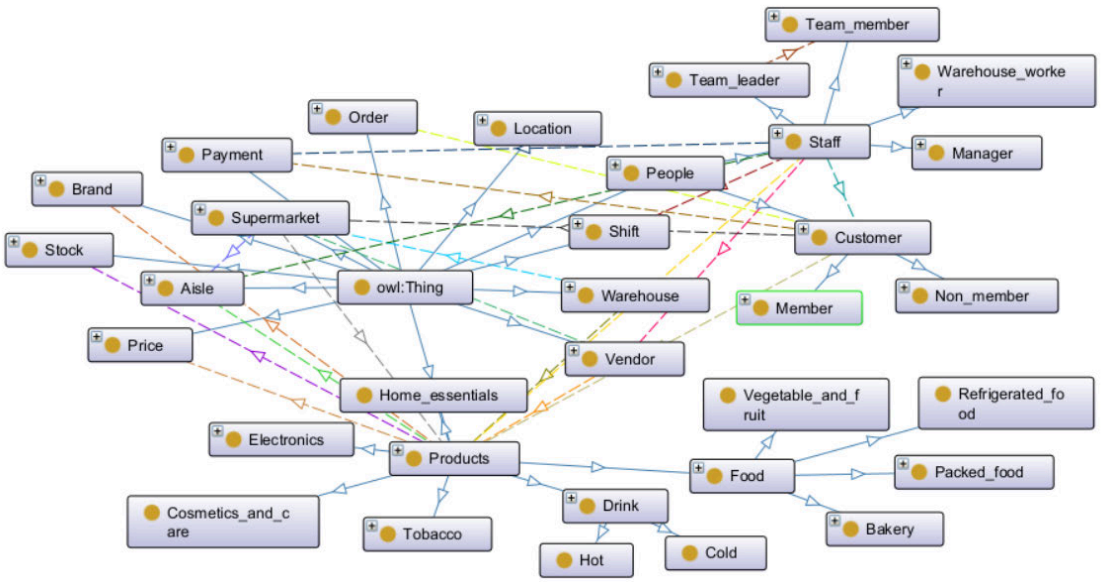


Figure 1.2: Graph of All Classes

3.2 Classes

Classes are groups of individuals that share common characteristics. They serve as a way to group data in an ontology.

Classes Role in Ontology
The classes in the model, along with their relationships, form the semantic model for the Supermarket Warehouse. They:

- Enable structured data representation.
- Support reasoning for data queries and insights.
- Simplify complex data relationships for better analysis and management.

Below is the screenshot of all classes used in this ontology and their explanation.

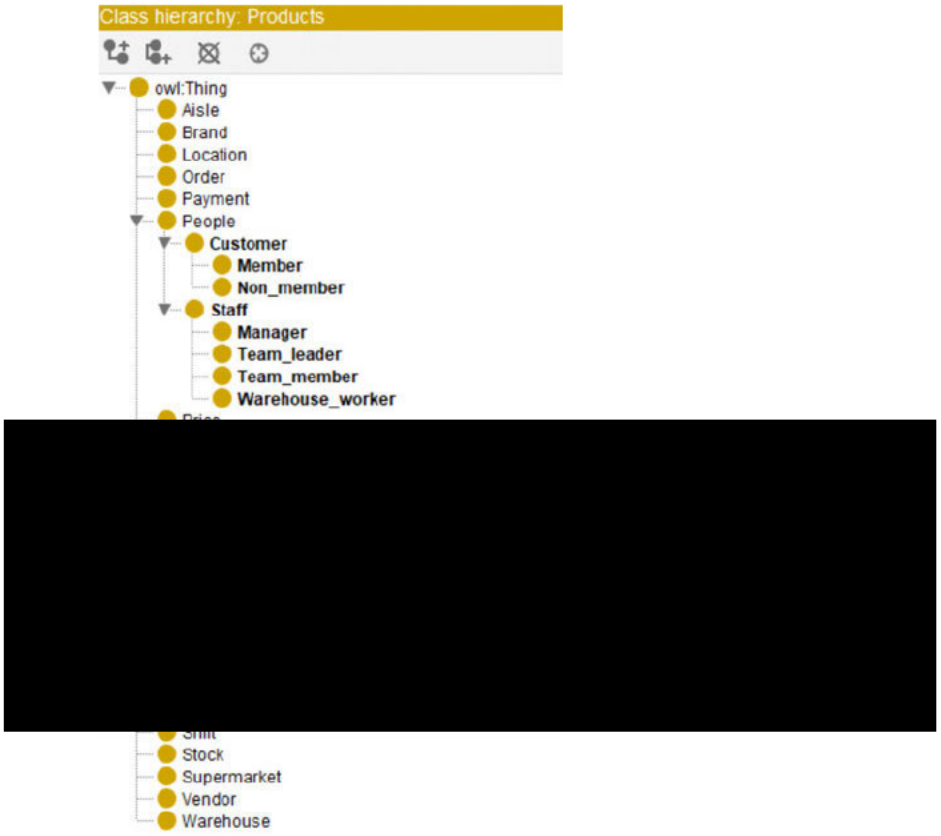


Figure 2.1: Classes

- **Aisle**

Description: Sections in the supermarket.

Role: Keeps track of location where same kind of products can be displayed.

Attributes:

- ➢ Name: Aisle name
- ➢ Number: Aisle number.

Usage in Ontology: Helps in organizing products.



Figure 2.2: Aisle Class in Protégé

- **Brand**

Description: Brands that are sold in supermarkets.

Role: Categorizes the products of the same brand for better inventory.

Attributes:

➢ Name: Name of the brand.

Usage in Ontology: Product distribution becomes easy.

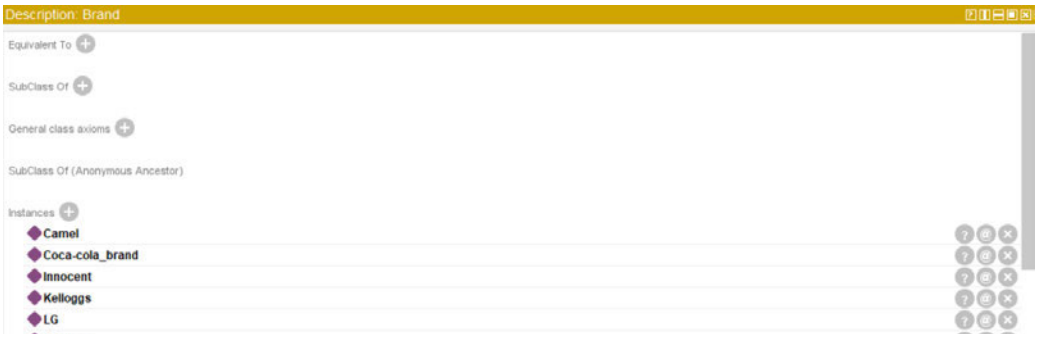


Figure 2.3: Brand Class in Protégé

- **Location**

Description: Place where supermarket is located.

Role: Defines where supermarkets and warehouses are situated.

Attributes:

➢ City: City of the location.
➢ Country: Country of the location.

Usage in Ontology: Links supermarkets and warehouses to physical places.

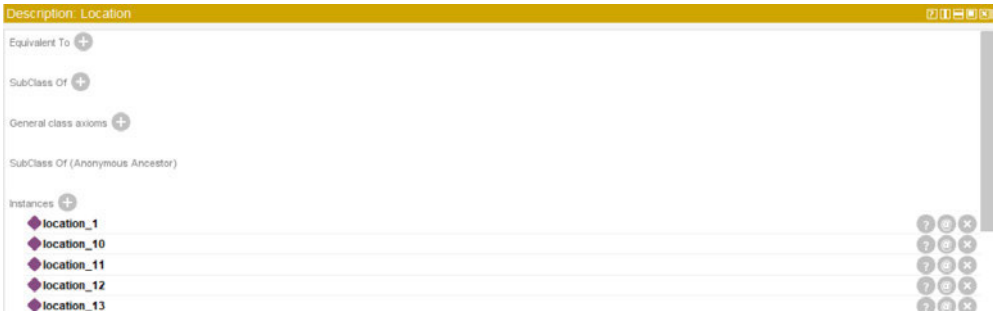


Figure 2.4: Location Class in Protégé

- **Order**

Description: Online orders that customers make.

Attributes:

➢ Order_no: To identify order by its unique order number.
➢ Order_date: Date of the order.

Usage in Ontology: Keeps track of items sold online.

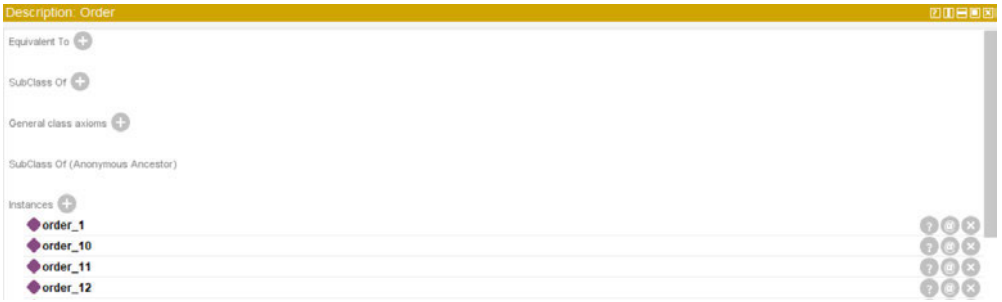


Figure 2.5: Order Class in Protégé

- **Payment**

Description: Payments made by customers when shopping in supermarkets.

Attributes:

➢ Amount: Payment amount.

Usage in Ontology: Tracks revenue and customer transactions.

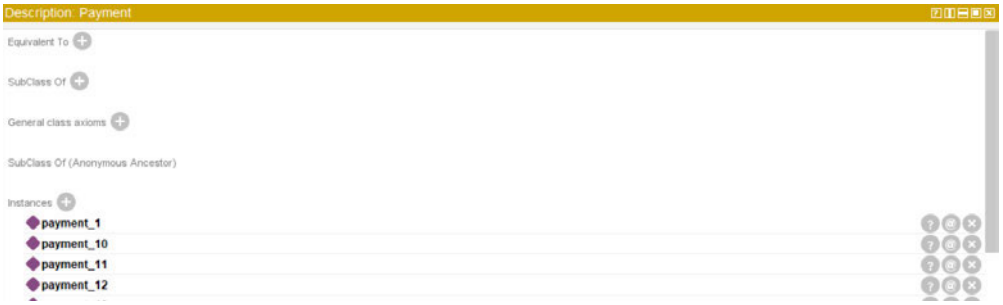


Figure 2.6: Payment Class in Protégé

- **People**

Description: People that can be associated with supermarkets.

Attributes:

- ➢ First_name
- ➢ Last_name
- ➢ Date_of_birth
- ➢ Age
- ➢ Gender

Role: Categorises staff and customer for better management.

Subclasses:

1. **Staff**: Employees working in supermarkets or warehouses.

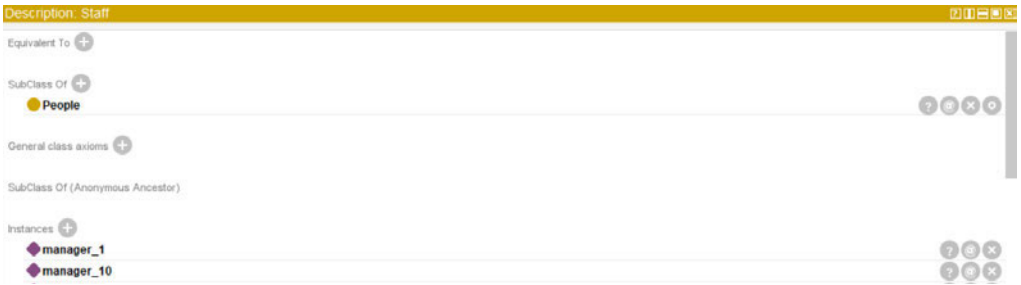


Figure 2.7: Staff Class in Protégé

a) Team Member: Members that deal with customers.



Figure 2.8: Team_member Class in Protégé

b) Manager: Person who manages the team members.

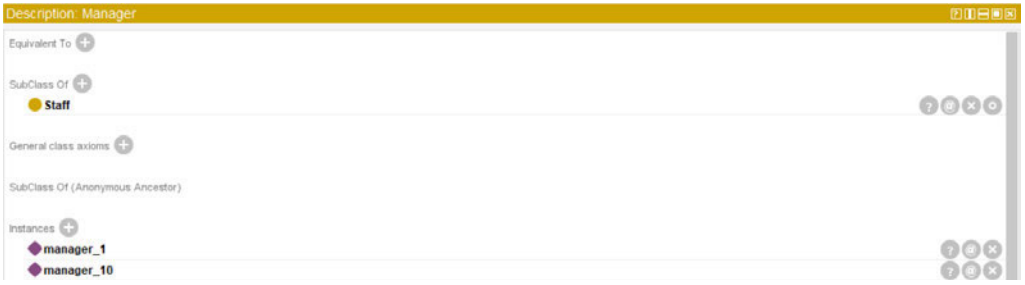


Figure 2.9: ManagerClass in Protégé

c) Team Leader: Leads team members.

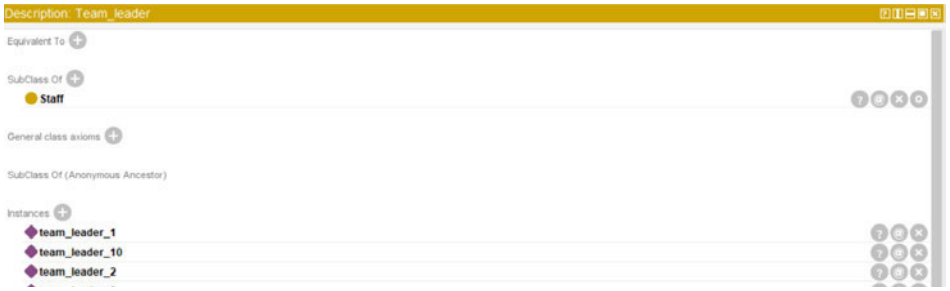


Figure 2.10: Manager Class in Protégé

d) Warehouse Worker: Staff working in warehouses.

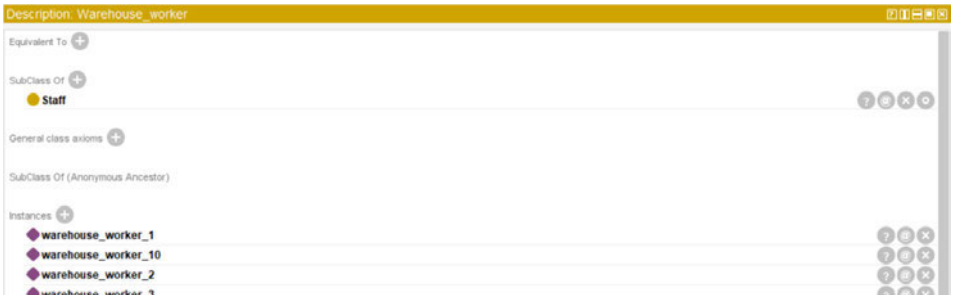


Figure 2.11: Warehouse_worker Class in Protégé

2. **Customer:** Shoppers in the supermarket.

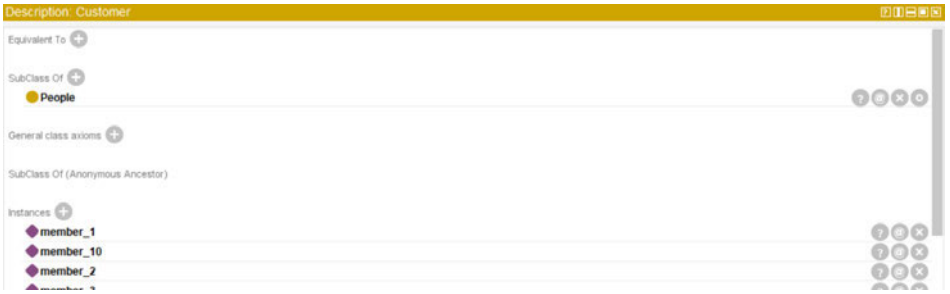


Figure 2.12: Customer Class in Protégé

a) Member: Loyalty program customers.

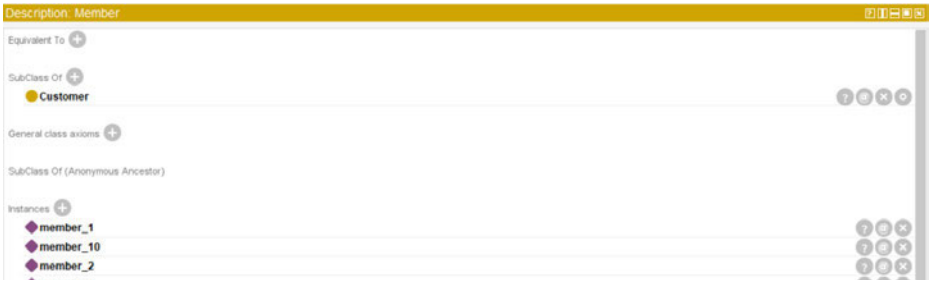


Figure 2.13: Member Class in Protégé

b) Non_member: Regular customers.

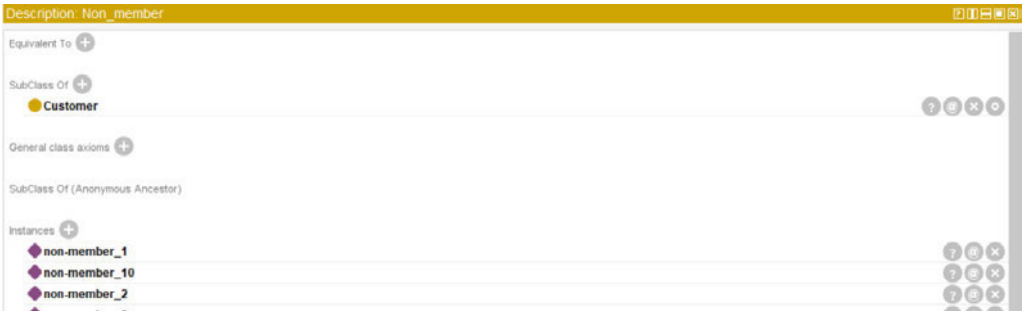


Figure 2.14: Non_member Class in Protégé

- **Price**

Description: Pricing details for products.

Role: Differentiates between member and non-member pricing.

Attributes:

- ➢ Member_price
- ➢ Non-member_price

Usage in Ontology: Enables price-related queries.

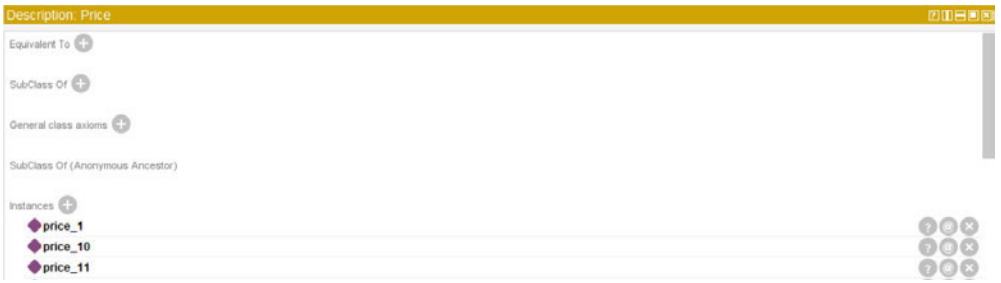


Figure 2.15: Price Class in Protégé

- **Products**

Description: Items that are sold in the supermarket.

Role: General class for all products.

Attributes:

- ➢ Product_no: Unique code to recognise each product.
- ➢ Name: Name of the product.

Usage in Ontology: Parent class for product-specific subclasses like food, electronics, and cosmetics.

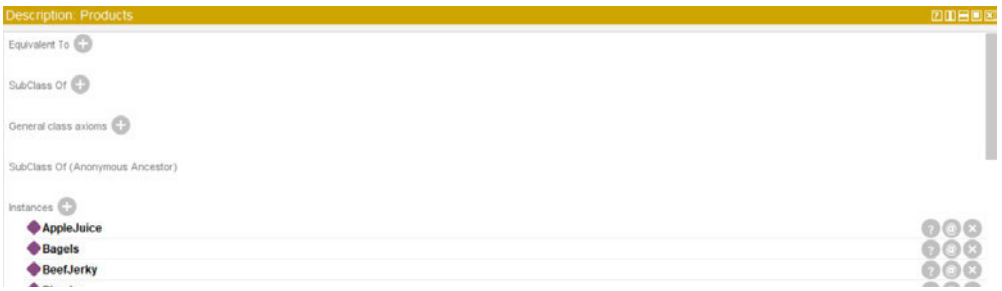


Figure 2.16: Price Class in Protégé

Subclasses:

a) Cosmetics and Care

Description: Personal care items.

Role: Tracks items toiletries and skincare products.

Usage in Ontology: To keep track of customer preferences.

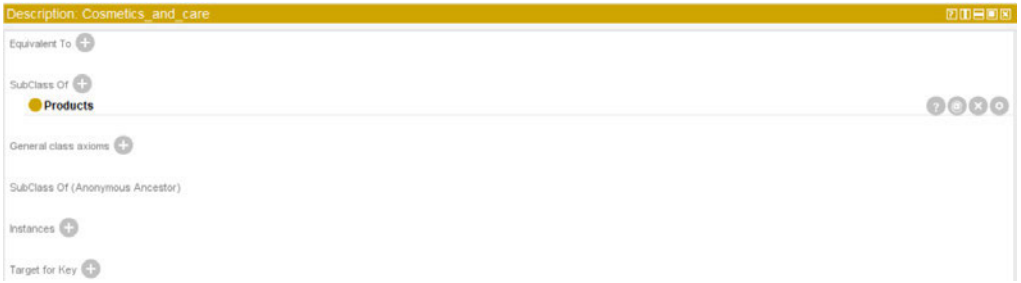


Figure 2.17: Cosmetics and Care Class in Protégé

b) Drink

Description: Subclass of products for beverages.

Role: Differentiates between hot and cold drinks.

Attributes:

- Calorie: Nutritional information.
- Date_of_expiry: Expiry date.

Usage in Ontology: Helps track beverage types and their storage needs.

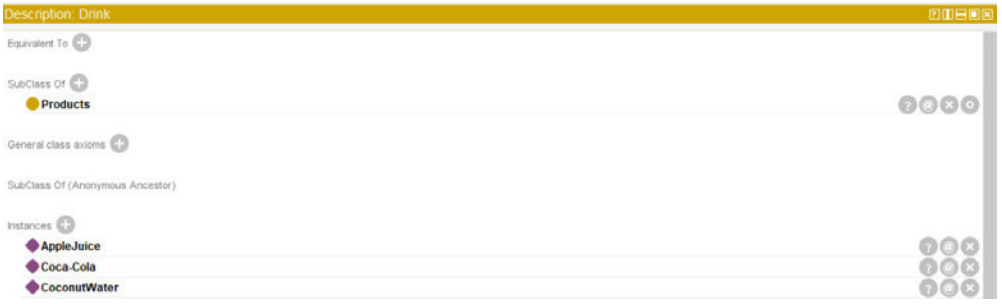


Figure 2.18: Drink Class in Protégé

Subcategories:

i. Hot: E.g., tea, coffee.



Figure 2.19: Hot Drink Class in Protégé

ii. Cold: E.g., juice, soda.



Figure 2.20: Cold Drink Class in Protégé

c) Electronics

Description: Represents electronic items sold.

Role: Tracks technical specifications.

Attributes:

- Watts: Power consumption.
- Voltage: Voltage requirements.

Usage in Ontology: keeps track of energy related appliances.

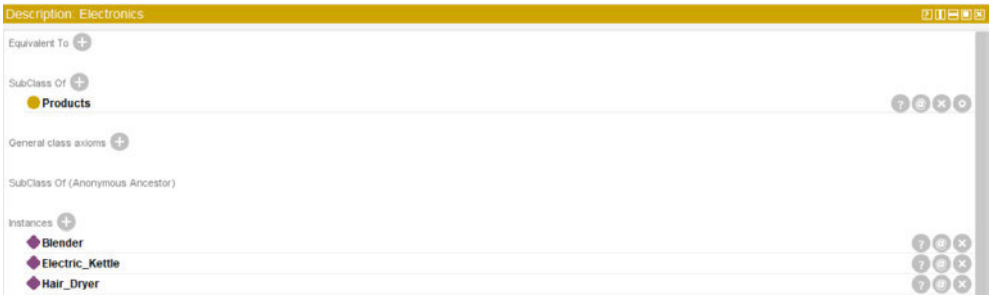


Figure 2.21: Electronics Class in Protégé

d) Food

Description: Subclass of products for food items.

Role: Keep track of items that are only sold fresh.

Attributes:

- Calorie: Nutritional information.
- Date_of_expiry: Expiry date.

Usage in Ontology: Enables tracking of perishable items.

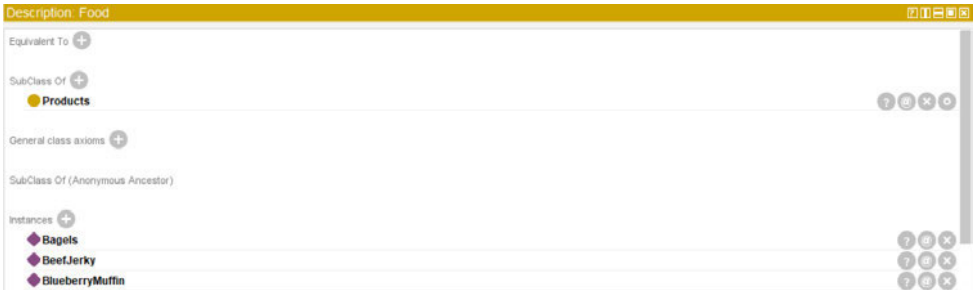


Figure 2.22: Food Class in Protégé

Subcategories:

i. Vegetable and Fruit: Fresh produce.

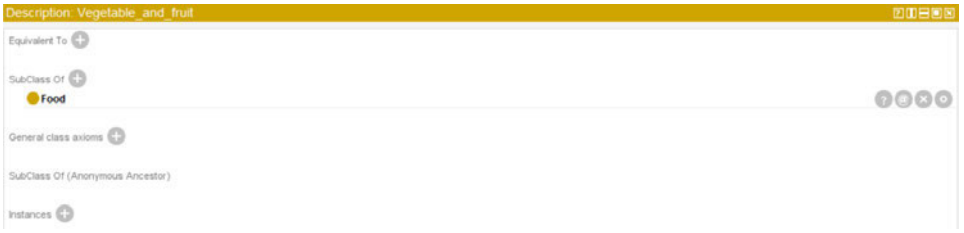


Figure 2.23: Vegetable and Fruit Class in Protégé

ii. Bakery: Baked good.



Figure 2.24: Bakery Class in Protégé

iii. Packed food: Pre-packaged items.

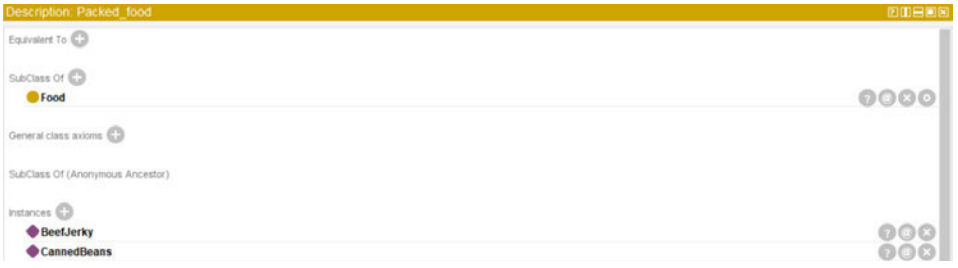


Figure 2.25: Packed_food Class in Protégé

iv. Refrigerated food: Items requiring cold storage.

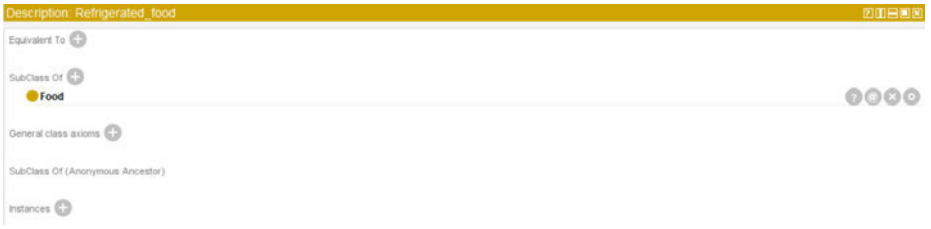


Figure 2.26: Refrigerated_food Class in Protégé

e) Home Essentials

Description: Household items like cleaning supplies and kitchen tools.

Role: Tracks non-perishable items essential for daily life.

Usage in Ontology: Helps analyse customer needs for home goods.

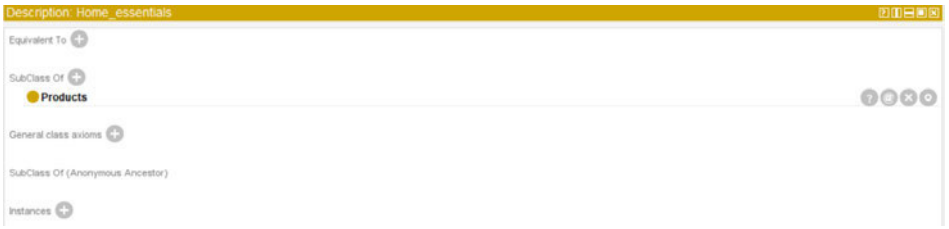


Figure 2.27: Home Essentials Class in Protégé

f) Tobacco

Description: Tobacco products.

Role: Tracks tobacco product details for regulations and sales.

Usage in Ontology: Addresses legal restrictions.

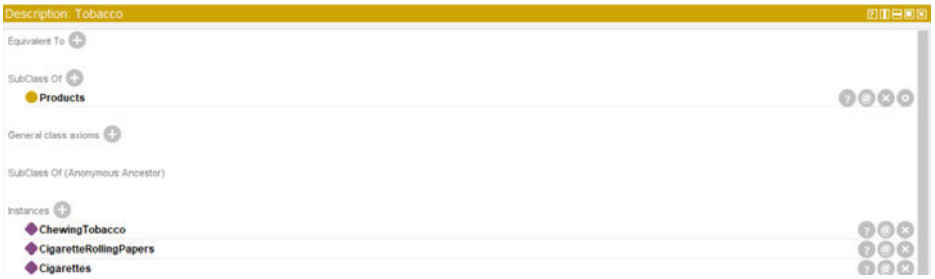


Figure 2.28: Tobacco Class in Protégé

- **Shift**

Description: Represents work shifts.

Attributes:

- ➢ Shift_number: Unique shift identifier.
- ➢ Working_hours: Hours in a shift.

Usage in Ontology: Manages staff schedules.

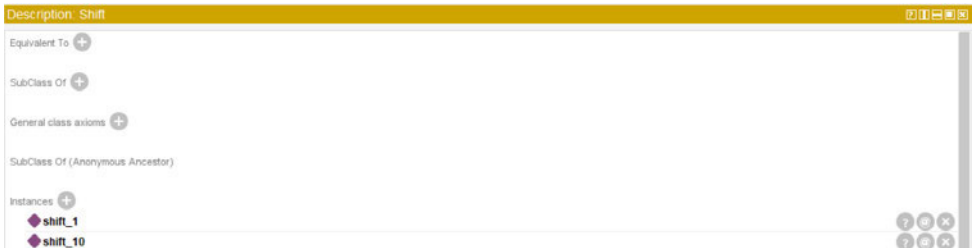


Figure 2.29: Shift Class in Protégé

- **Stock**

Description: Represents inventory levels.

Role: Tracks the quantity of products available in the supermarket or warehouse.

Attributes:

➢ Quantity: Number of items in stock.

Usage in Ontology: Enables stock management and restocking processes.

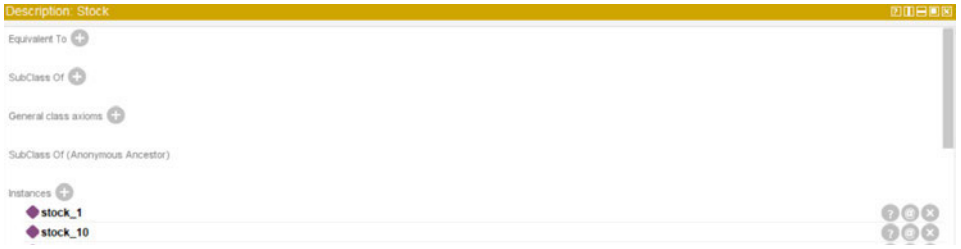


Figure 2.30: Stock Class in Protégé

- **Supermarket**

Description: Represents the supermarket entity.

Role: Keeps details about supermarkets, such as their name, opening year, operating hours, and revenue.

Attributes:

➢ Name: The name of the supermarket.
➢ Opening_year: The year the supermarket opened.
➢ Operating_hours: Hours the supermarket is open.
➢ Revenue: Revenue generated by the supermarket.

Usage in Ontology: Helps identify and categorize individual supermarkets and analyse their operations and financial performance.

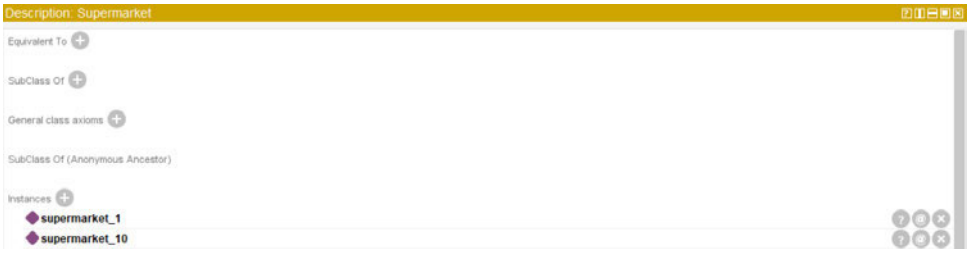


Figure 2.31: Supermarket Class in Protégé

- **Vendor**

Description: Represents suppliers.

Attributes:

➢ Name: Vendor's name.

Usage in Ontology: Tracks product supply chains.

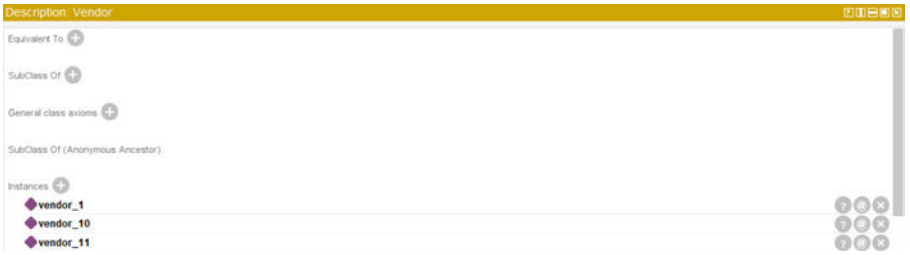


Figure 2.32: Vendor Class in Protégé

- **Warehouse**

Description: Represents storage facilities.

Attributes:

➢ Name: Warehouse name.
➢ Opening_year: Year the warehouse began operations.
➢ Operating_hours: Warehouse operating hours.

Usage in Ontology: Manages inventory storage and distribution.

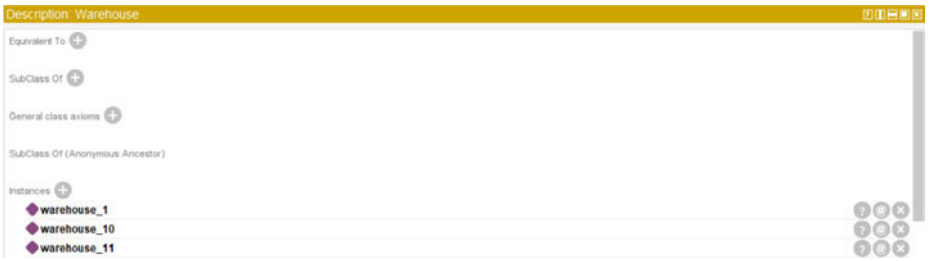


Figure 2.33: Warehouse Class in Protégé

3.3 Properties

Properties play a vital role in ontology development as they are used to define relationships and attributes of entities. They are divided into Object Properties and Data Properties. Object Properties tell the relations between entities; Data Properties show the attributes of entities.

Object Properties:

Object Properties are used for connecting different entities in an ontology. They specify how things interact and relate to each other in the domain. Below is a screenshot and detailed explanation of the object properties that are used in this ontology.



Figure 3.1: Object Properties

- **Assign_work:**

This object property is used to connect subclasses of Staff i.e., Team_leader and Team_member which are domain and range respectively. This property is used in the triple “Team_leader assigns work to Team_member”.

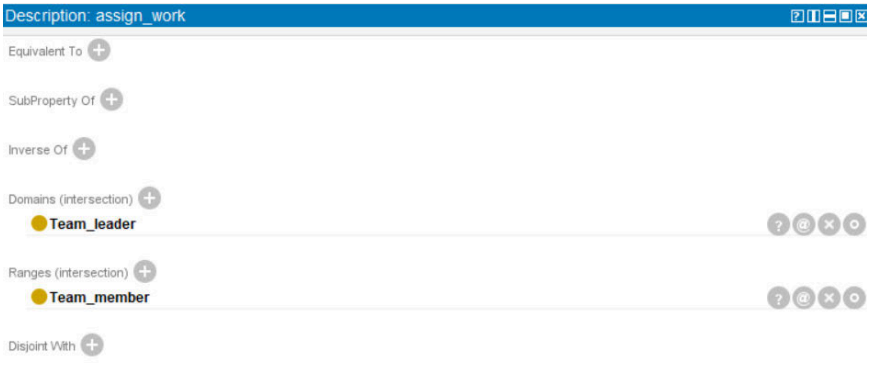


Figure 3.2: Assign_work Object Property in Protégé

- **Belong_to:**

Belong_to object property is used to map the relationship between Product entity and Brand entity as domain and range respectively. It satisfies the triple “Products belong to brand”.

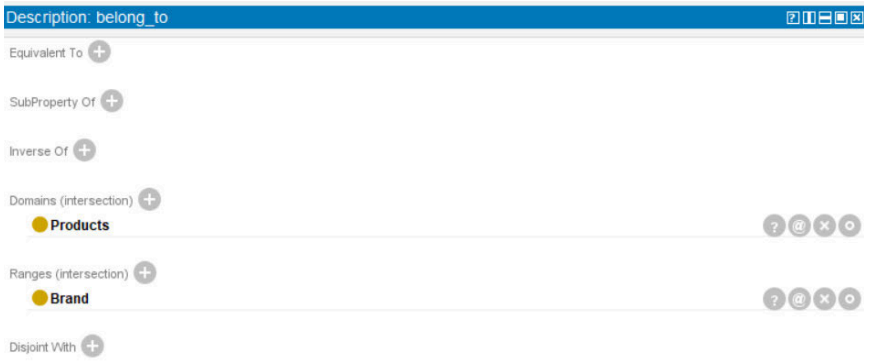


Figure 3.2: Belong_to Object Property in Protégé

- **Buy:**

The object property buy is used as a connection builder between Customer and Products entities where Customer being the domain and Products being range. It is used to describe the triple “Customer buy Products”.

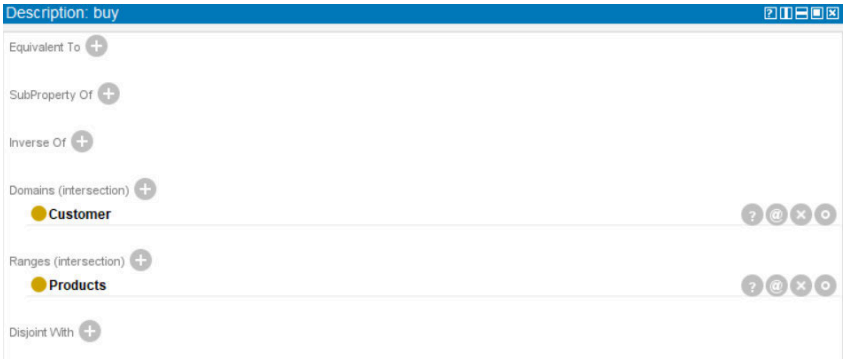


Figure 3.3: Buy Object Property in Protégé

- **Come_to:**

Come_to is an object property in which the domain is Vendor and range is Supermarket. It establishes the connection for the triple “Vendor come_to Supermarket”.

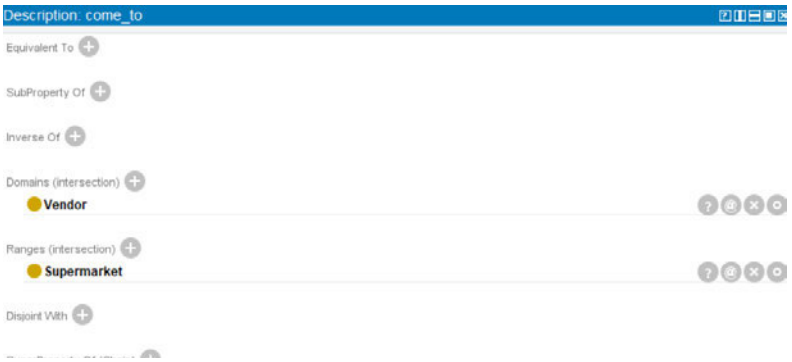


Figure 3.4: Come_to Object Property in Protégé

- **Coordinate_with:**

This object property creates a relationship between Staff and Vendor entity with the first one being domain and later as range. It defines the connection for the triple “Staff Coordinates with vendor”.

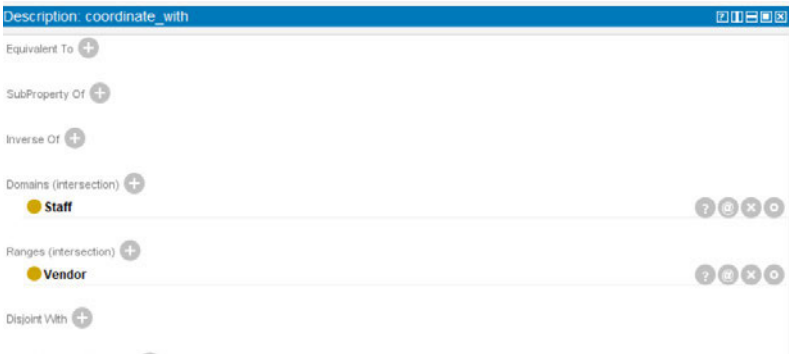


Figure 3.5: Coordinate_with Object Property in Protégé

- **Deal_with:**

The Staff and Customer are linked with each other through this object property. This object Property is used in the triple “Staff deal with customers”.

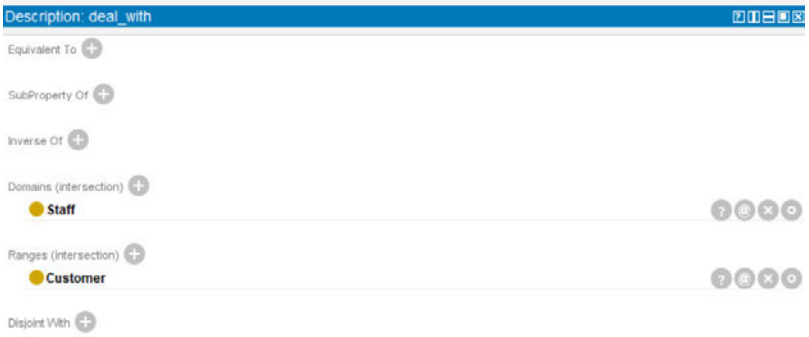


Figure 3.6: Deal_with Object Property in Protégé

- **Displayed_in:**

The triple “Products are displayed in aisle” uses this object property to create a linkage between Products and Aisle classes with Products being domain and Aisle being range.

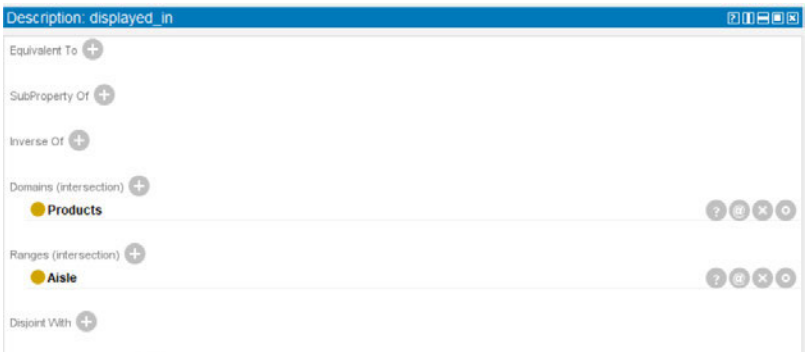


Figure 3.7: Displayed_in Object Property in Protégé

- **Do_shopping_at:**

The do_shopping_at object property is used to create a relationship between Customers and Supermarket classes with the first being domain and second being range. This property is used for the triple “Customers do shopping at Supermarket”.

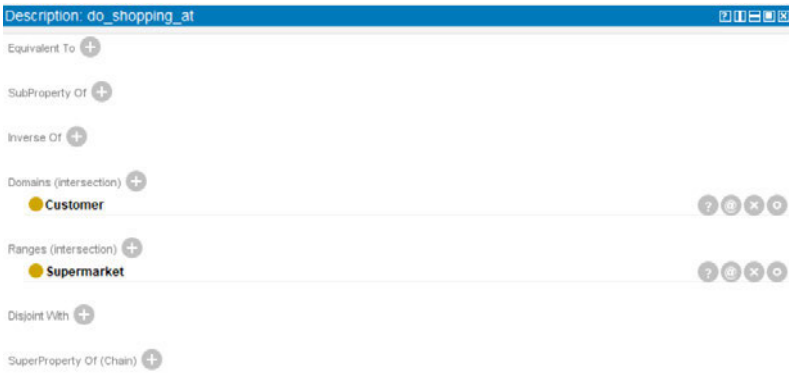


Figure 3.8: Do_shopping_at Object Property in Protégé

- **Give:**

The give object property uses Customer class as domain and Order class as range to satisfy the triple “Customers give Orders”.

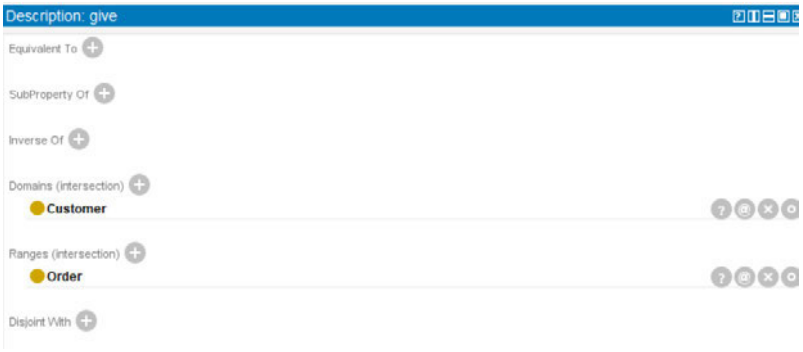


Figure 3.9: Give Object Property in Protégé

- **Handle:**

Handle object property satisfies the triple “Staff handles Payment” and is used as an interlinking property between Staff and Payment classes.

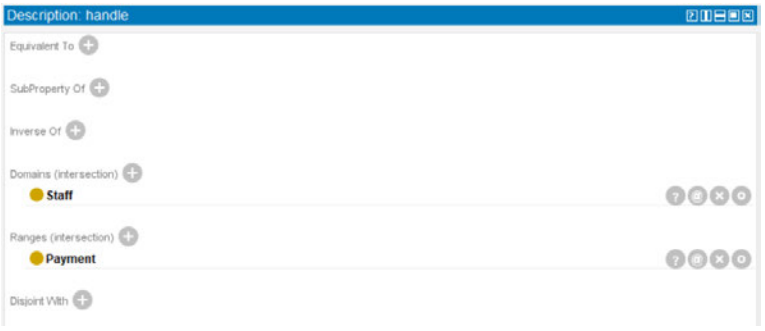


Figure 3.10: Handle Object Property in Protégé

- **Has_aisle:**

Has_aisle is used for “Supermarket has aisle” triple, as it builds the connection between Supermarket and Aisle classes as domain and range respectively.

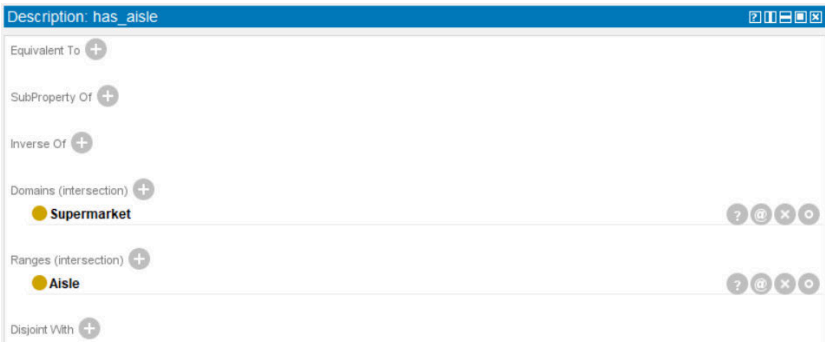


Figure 3.11: Has_aisle Object Property in Protégé

- **Has_location:**

This object property satisfies 2 triples “Supermarket has location” and “Warehouse has location” where Supermarket or Warehouse acts as domain and locations acts as a range.

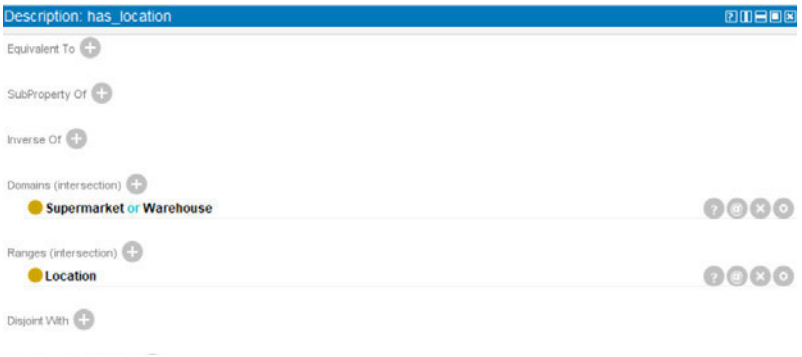


Figure 3.12: Has_location Object Property in Protégé

- **Has_price:**

Has_price is an object property in which the domain is Products and range is Price. It establishes the connection for the triple “Product has Price”.

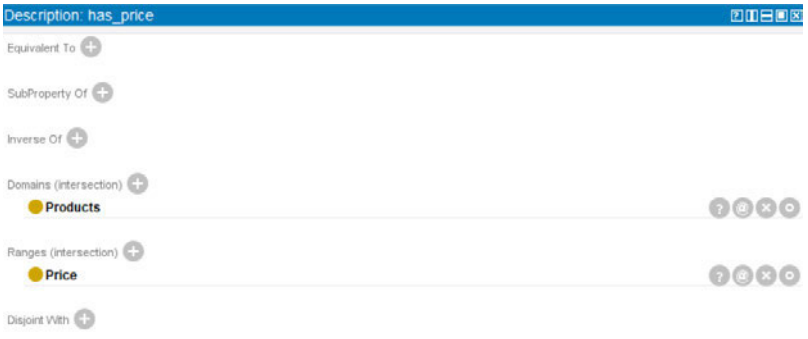


Figure 3.13: Has_price Object Property in Protégé

- **Has_product:**

Has_product object property is used to create a relationship between Warehouse class and Product class as domain and range respectively. It satisfies the triple “Warehouse has Products”.

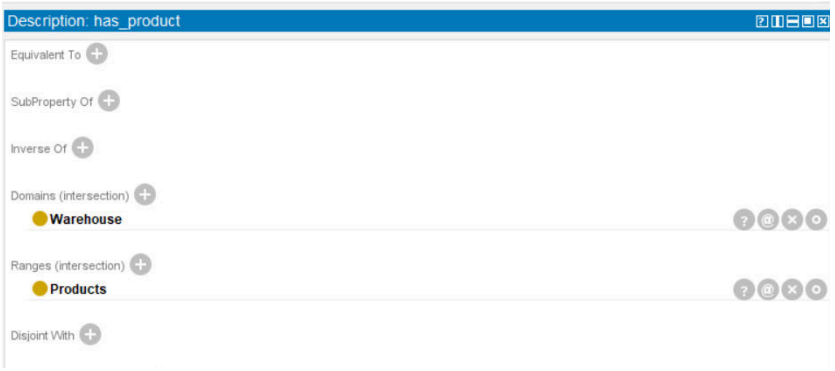


Figure 3.14: Has_product Object Property in Protégé

- **Has_shift:**

The has_shift object property is used to create a relationship between Staff and Shift classes with the Staff being domain and Shift being range. This property is used for the triple “Staff has Shift”.

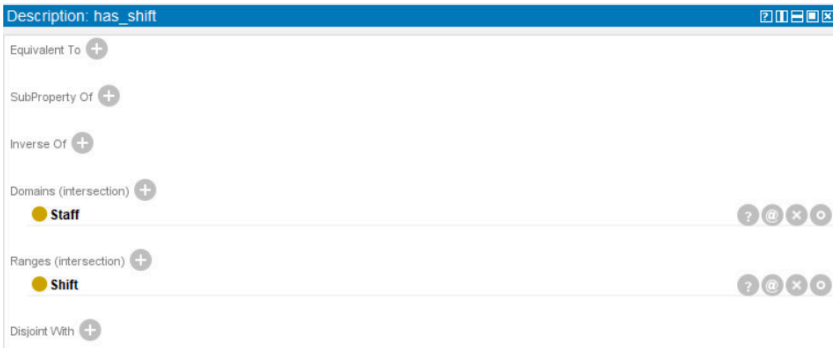


Figure 3.15: Has_shift Object Property in Protégé

- **In_stock:**

This object property creates a relationship between Product and Stock classes with the first one being domain and later as range. It defines the connection for the triple “Product is in stock”.

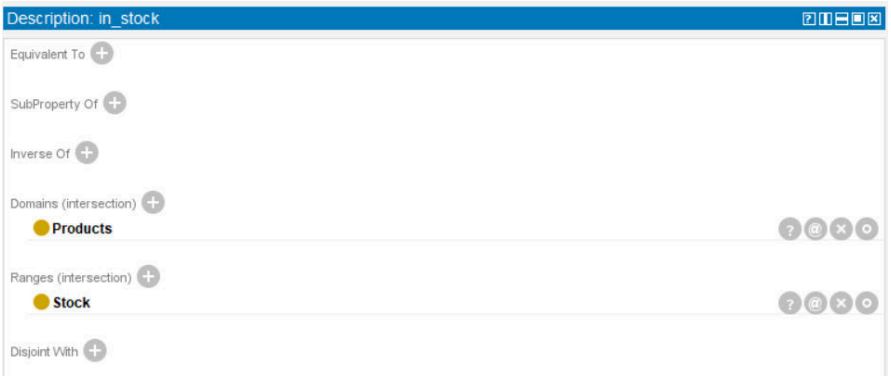


Figure 3.16: In_stock Object Property in Protégé

- **Include:**

This object property satisfies 2 triples “Order include Products” and “Payment include Products” where Order and Payment acts as domain and Products acts as a range.

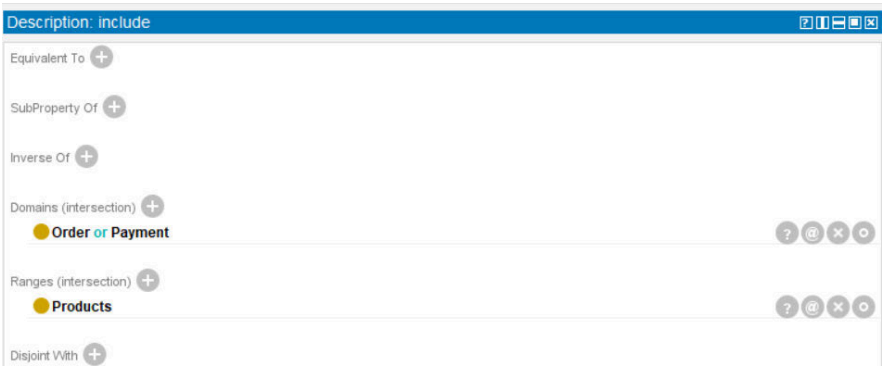


Figure 3.17: Include Object Property in Protégé

- **Make_payment:**

The object property make_payment is used for the triple “Customer makes payment” where Customer is domain and Payment is range.

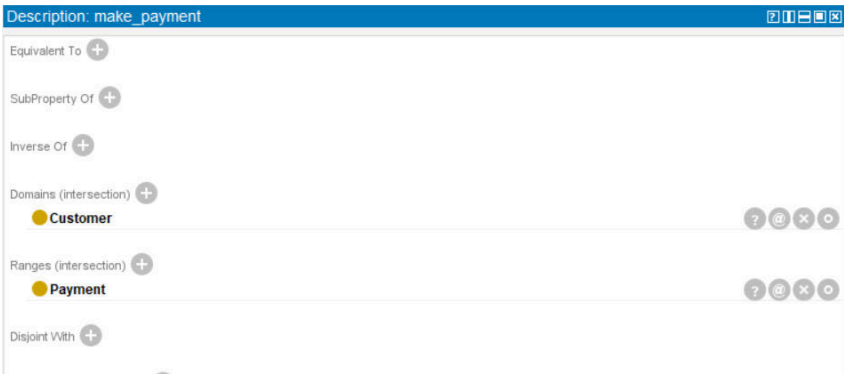


Figure 3.18: Make_payment Object Property in Protégé

- **Manage:**

“Staff manage Products” is the statement which is satisfied by Object property “manage” and Staff is the domain while Products is the range.

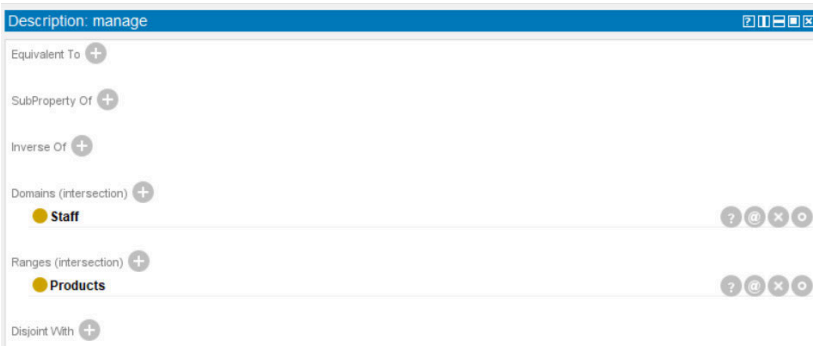


Figure 3.19: Manage Object Property in Protégé

- **Provide_for:**

The triple “Warehouse provide for Supermarket” uses this object property to create a linkage between Warehouse and Supermarket classes with Products being domain and Aisle being range.

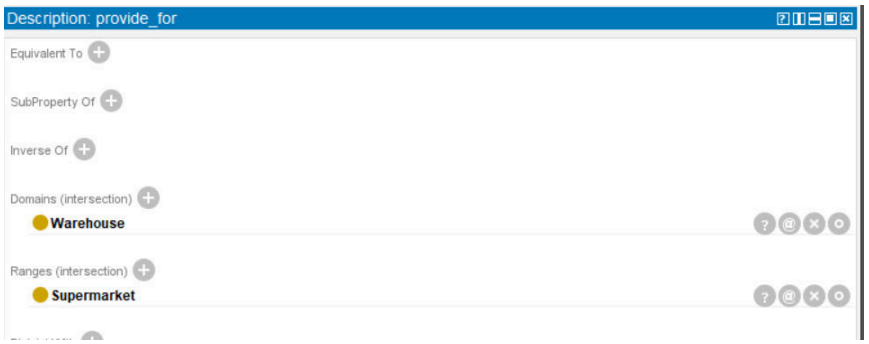


Figure 3.20: Provide_for Object Property in Protégé

- **Sell:**

The triple “Supermarket sells Products” has “sell” as object property while Supermarket and Products as domain and range respectively.

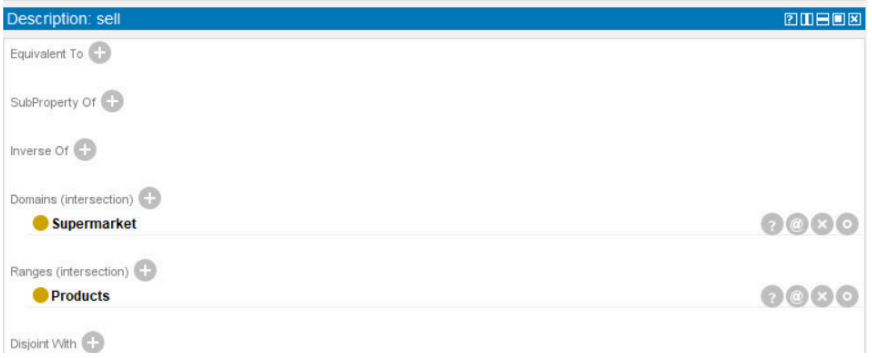


Figure 3.21: Sell Object Property in Protégé

- **Stock:**

Stock object property satisfies the triple “Staff stock Aisles” and is used as an interlinking property between Staff and Stock classes.

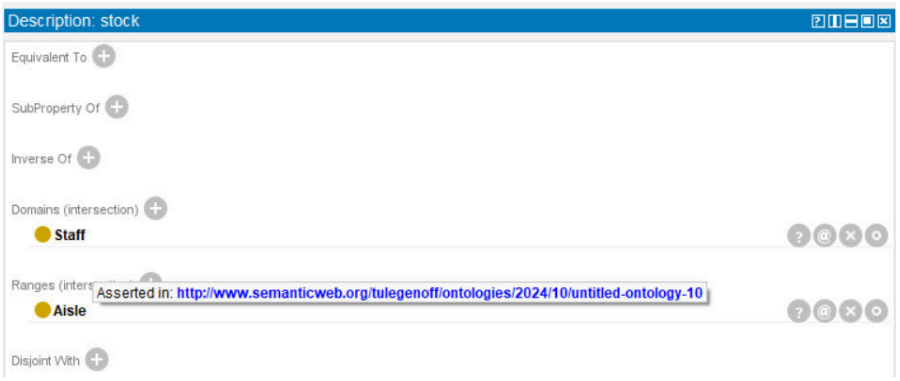


Figure 3.22: Stock Object Property in Protégé

- **Supply:**

Supply is an object property in which the domain is Vendor and range is Products. It establishes the connection for the triple “Vendor supplies Products”.

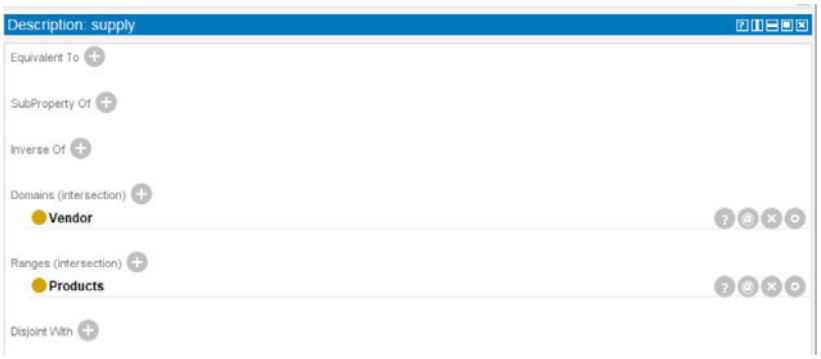


Figure 3.23: Supply Object Property in Protégé

- **Work_at:**

The Object property work_at is used to create a relationship between Staff as domain and Supermarket or Warehouse as Range to satisfy the triples “Staff works at Warehouse” and “Staff works at Supermarket”.

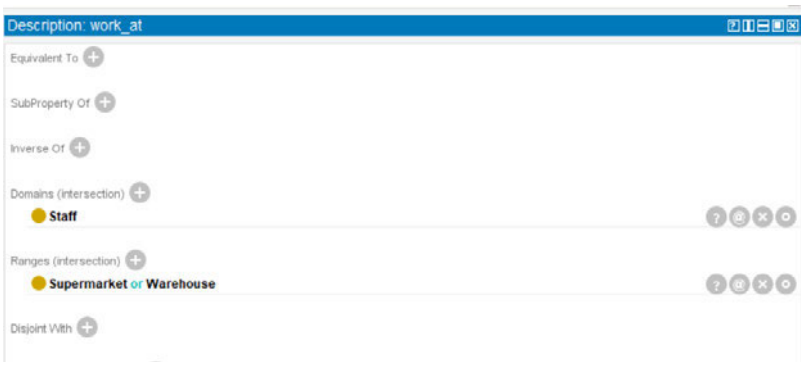


Figure 3.24: Work_at Object Property in Protégé

Data Properties:

Data Properties are used to define the characteristics and attributes of individuals of entities. It relates individuals to literal data (e.g., integers, floats, string, datetime etc.). Below is the screenshot of the data properties used in this ontology.

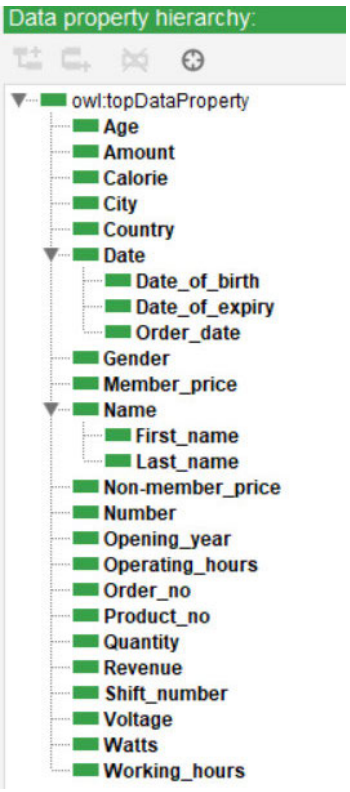


Figure 4.1: Data Properties

Below is the explanation of all the data properties including their domain and range where domain means entities and range mean the data type of Data properties:

- **Age:**
  Domain: People
  Range: Integer
  Purpose: to define age of people (including Customer and Staff)

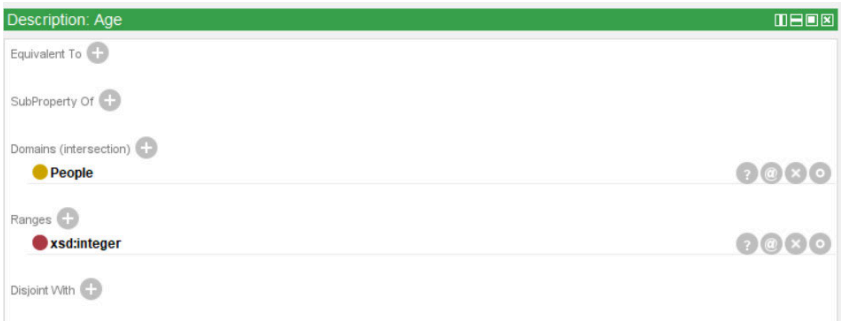


Figure 4.2: Age Data Property in Protégé

- **Amount:**
  Domain: Payment
  Range: Float
  Purpose: to define amount of Payment

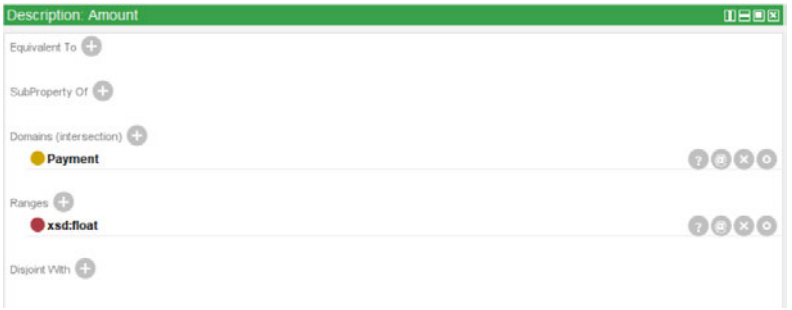


Figure 4.3: Amount Data Property in Protégé

- **Calorie:**
  Domain: Drink or Food
  Range: Integer
  Purpose: to define calories in food and drinks

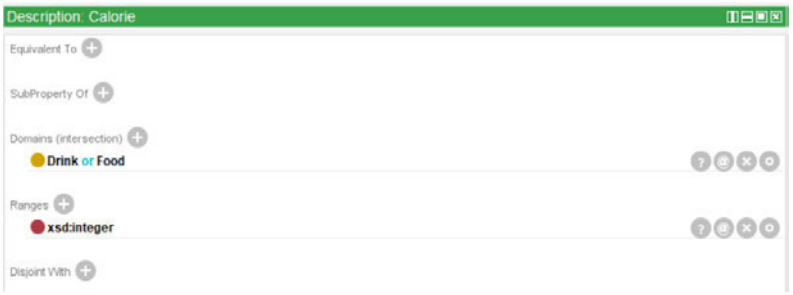


Figure 4.4: Calorie Data Property in Protégé

- **City:**
  Domain: Location
  Range: String
  Purpose: to define City of the location of Supermarket or Warehouse.

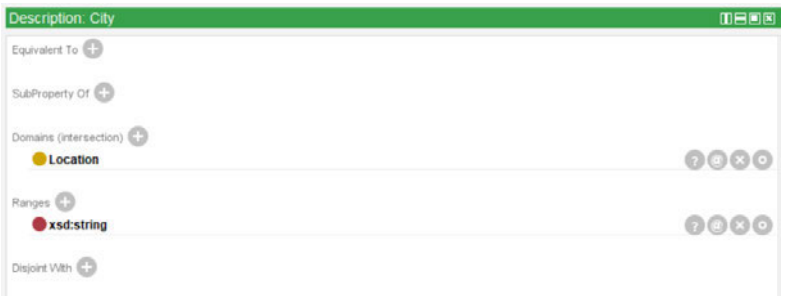


Figure 4.5: City Data Property in Protégé

- **Country:**
  Domain: Location
  Range: String
  Purpose: to define Country of the location of Supermarket or Warehouse.

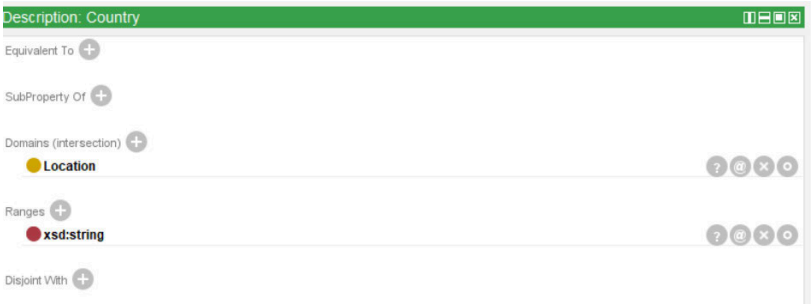


Figure 4.6: Country Data Property in Protégé

- **Date:**

This data property has 3 sub data properties.

1. Date_of_birth:

➢ Domain: People
➢ Range: String
➢ Purpose: to define date of birth of people (including Customer and Staff).



Figure 4.6: Date_of_birth Data Property in Protégé

2. Date_of_Expiry:
   ➢ Domain: Food or Drinks
   ➢ Range: String
   ➢ Purpose: to define date of expiry for foods and drinks.



Figure 4.7: Date_of_expiry Data Property in Protégé

3. Order_date:

➢ Domain: Order
➢ Range: String
➢ Purpose: to define date on which order was placed.



Figure 4.7: Order_date Data Property in Protégé

- **Gender:**
  Domain: People
  Range: String
  Purpose: to define gender of people (including Customer and Staff).

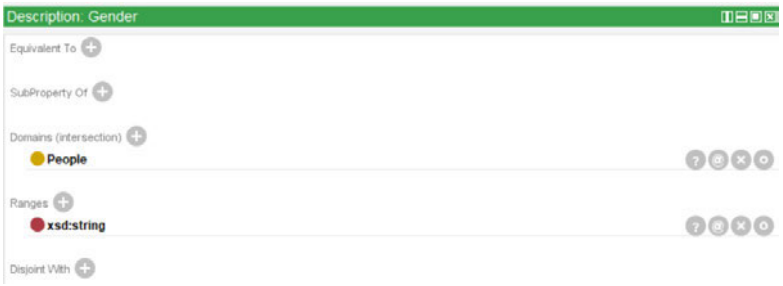


Figure 4.8: Gender Data Property in Protégé

- **Member_price:**
  Domain: Price
  Range: Float
  Purpose: to define price of Product for member customers.

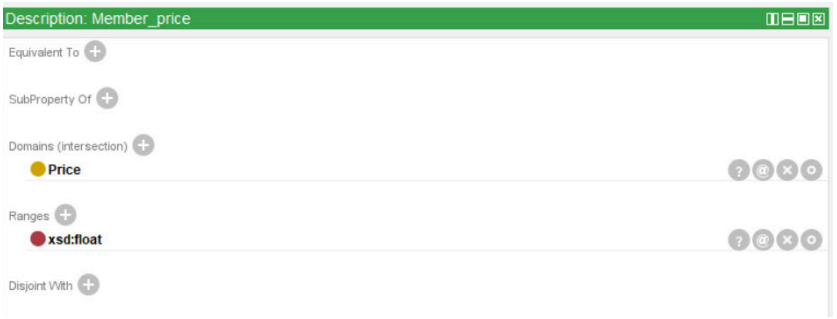


Figure 4.9: Member_price Data Property in Protégé

- **Name:**
  Domain: Aisle, Brand, People, Products, Supermarket, Vendor, Warehouse
  Range: String
  Purpose: To define the name of individuals in the ontology for Aisle, Brand, People, Products, Supermarket, Vendor, Warehouse classes

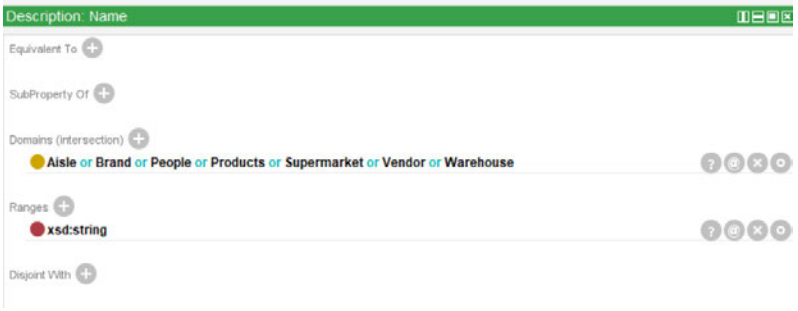


Figure 4.10: Name Data Property in Protégé

Also, Name has 2 sub data properties

1. First_name:
    - ➢ Domain: People
    - ➢ Range: String
    - ➢ Purpose: to define first name of people (including Customer and Staff).



Figure 4.11: First_name Data Property in Protégé

2. Last_name:
    - ➢ Domain: People
    - ➢ Range: String
    - ➢ Purpose: to define first name of people (including Customer and Staff).

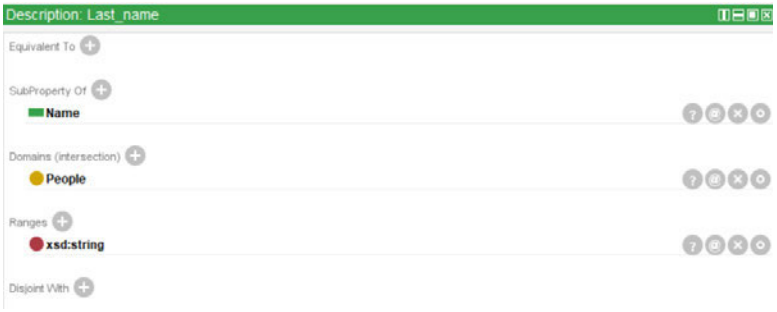


Figure 4.11: Last_name Data Property in Protégé

- **Non-Member_price:**
  Domain: Price
  Range: Float
  Purpose: to define price of Product for non member customers.

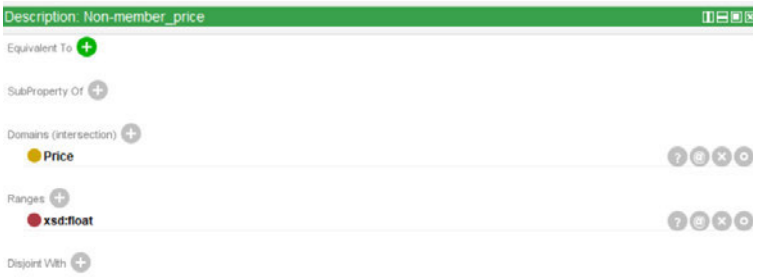


Figure 4.11: Non-member_price Data Property in Protégé

- **Number:**
  Domain: Aisle
  Range: Integer
  Purpose: to define number of Aisle.

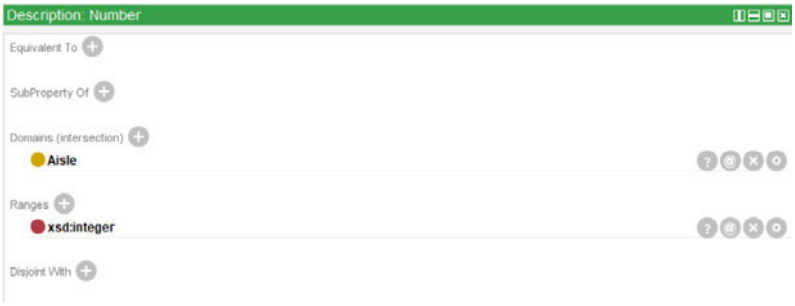


Figure 4.12: Number Data Property in Protégé

- **Opening_year:**
  Domain: Supermarket or Warehouse
  Range: Integer
  Purpose: to define the year in which Supermarket or Warehouse was opened.

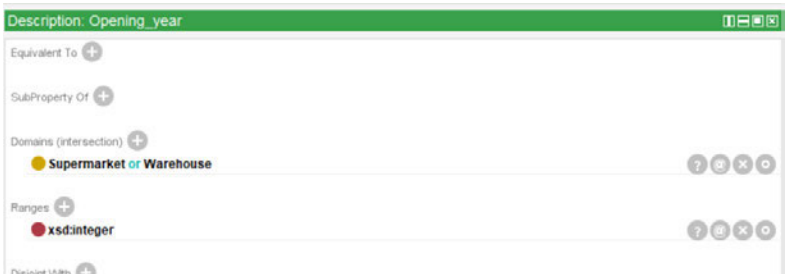


Figure 4.13: Opening_year Data Property in Protégé

- **Operating_hours:**
  Domain: Supermarket or Warehouse
  Range: String
  Purpose: to define Operating hours of Supermarket or Warehouse.

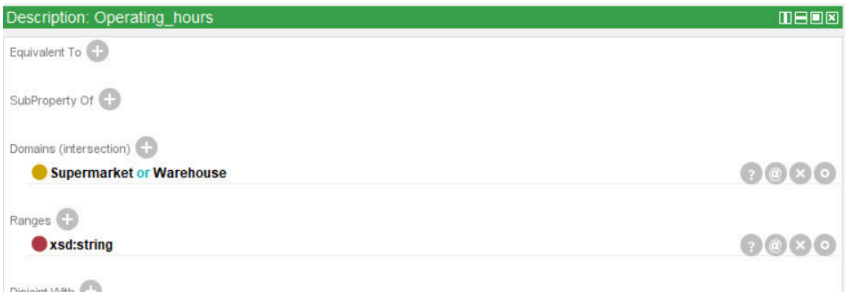


Figure 4.14: Operating_hour Data Property in Protégé

- **Order_no:**
  Domain: Order
  Range: Integer
  Purpose: to define Order number of Order. It is generally used to uniquely identify each order in supermarkets.

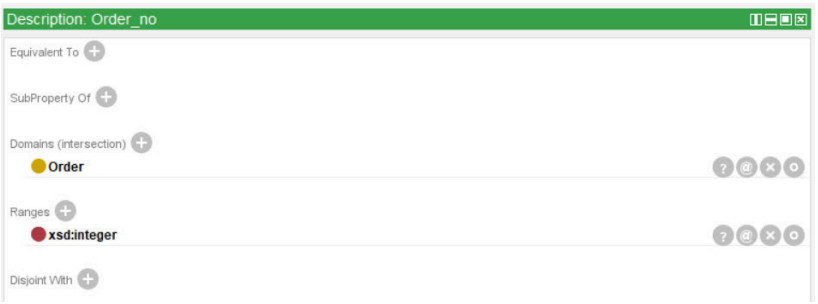


Figure 4.15: Order_no Data Property in Protégé

- **Product_no:**
  Domain: Products
  Range: Integer
  Purpose: to define Product number of Products. It is generally used to uniquely identify each Product in supermarkets.



Figure 4.16: Product_no Data Property in Protégé

- **Quantity:**
  Domain: Stock
  Range: Integer
  Purpose: to define the quantity of stock



Figure 4.17: Quantity Data Property in Protégé

- **Revenue:**
  Domain: Supermarket
  Range: float
  Purpose: to define the revenue earned by a particular supermarket.

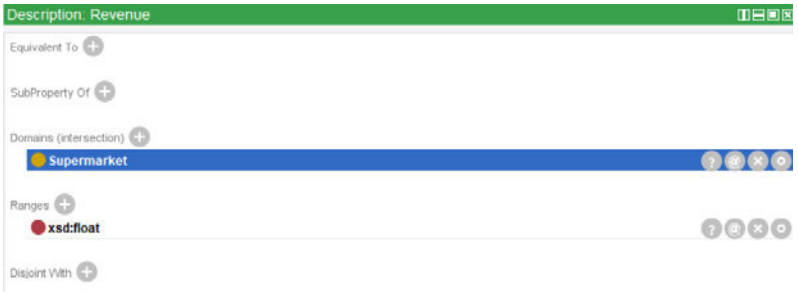


Figure 4.18: Revenue Data Property in Protégé

- **Shift_number:**
  Domain: Shift
  Range: Integer
  Purpose: to define the Shift number of Shift. Using shift number because each shift number will have specific hours which will be assigned to each Staff member.



Figure 4.19: Shift_number Data Property in Protégé

- **Voltage:**
  Domain: Electronics
  Range: String
  Purpose: to define voltage of Electronic Products.

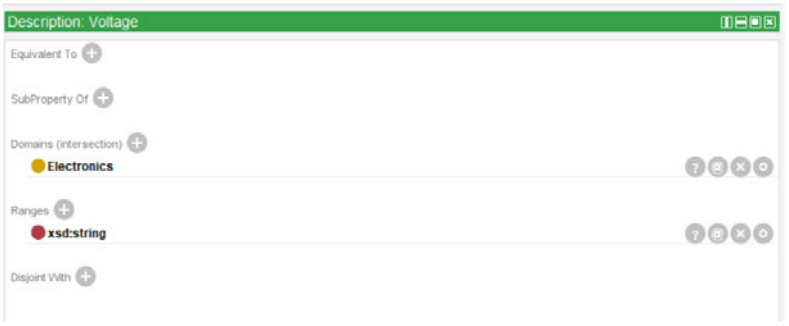


Figure 4.20: Voltage Data Property in Protégé

- Watts:
  Domain: Electronics
  Range: Integer
  Purpose: to define Watts of Electronic Products.

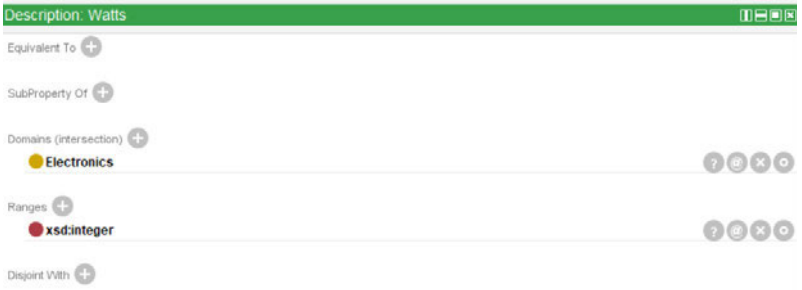


Figure 4.21: Watts Data Property in Protégé

- **Working_hours:**
  Domain: Shift
  Range: String
  Purpose: to define the working_hours of each shift.

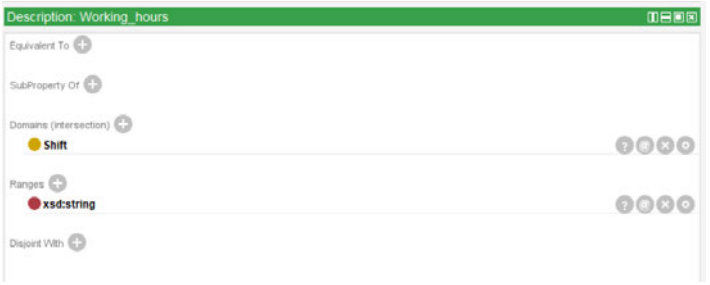


Figure 4.22: Working_hours Data Property in Protégé

3.4 Individuals

Individuals are one of the fundamental components in an ontology ('Ontology components', 2024). They can be seen as the data in an ontology. With individuals, representatives of the classes can be created as well as some data and object properties can be used to give details. An ontology can exist without any individuals, it is called a Knowledge Base that contains individuals.

In this project, there are 404 individuals constructed in Protégé from different classes. At the start of the individual production process, an Excel sheet is created and all the information about individuals is written inside it. ChatGPT is used at several points to create dummy data (https://openai.com/index/chatgpt/, 2022). All individuals in addition to data properties and object properties of them are added to the Supermarket Warehouse ontology with the rules written in Protégé taking data from Excel sheets.

While writing rules, data types of the data according to ranges that are indicated in Protégé for data properties are specified as "(xsd:*data_type*)" except string types. The reason behind it is about Protégé. The system takes every part of the data as string type unless it is specifically said to be another type.

Below is the information for individual examples for all classes and rules used.

- **Aisle:** There are 38 individuals added to the ontology. According to design, each Aisle individual is unique and can be present at one Supermarket only.

Aisle		
individual_name	name	number
aisle_bakery_1	Bakery	1
aisle_bakery_2	Bakery	2
aisle_electronics_1	Electronics	3
aisle_electronics_2	Electronics	4
aisle_packed_food_1	Packed Food	5
aisle_packed_food_2	Packed Food	6
aisle_packed_food_3	Packed Food	7
aisle_drink_hot	Hot Drinks	8
aisle_drink_2	Drinks	9
aisle_drink_3	Drinks	10
aisle_tobacco_1	Tobacco Products	11
aisle_tobacco_2	Tobacco Products	12
aisle_drink_4	Drinks	13
aisle_drink_5	Drinks	14

Figure 5.1: Aisle Individuals Information on Excel

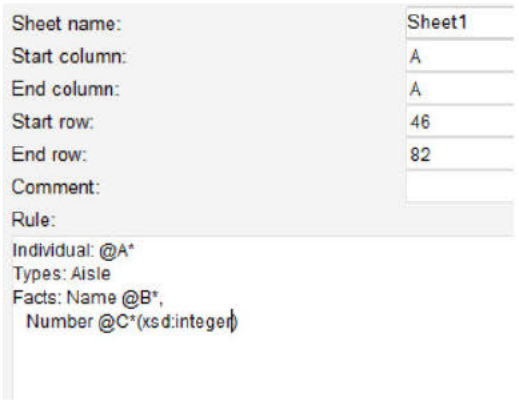


Figure 5.2: Rule to Add Aisle Individuals

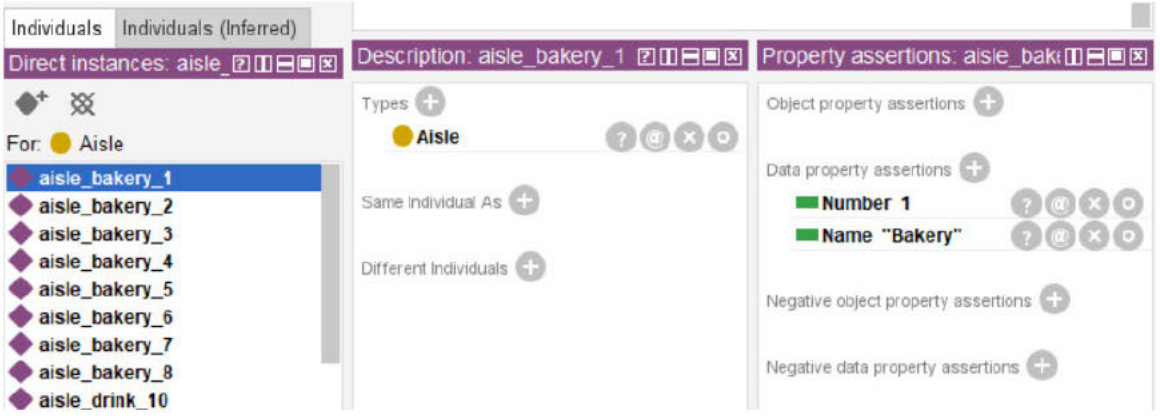


Figure 5.3: Aisle Individual Example from Protégé

- **Brand:** There are 19 individuals added to the ontology. Individuals build according to Products data.

Brand	
individual_name	name
Nestle	Nestle
Waitrose	Waitrose
Warburton	Warburton
Samsung	Samsung
Sony	Sony
LG	LG
Marlboro	Marlboro
Camel	Camel
Coca-Cola_brand	Coca-Cola
Pepsi	Pepsi
Kelloggs	Kellogg's
Philips	Philips
Innocent	Innocent
Tesco	Tesco
Vita	Vita
Monster	Monster
Nong-shim	Nong-shim
Sainsburys	Sainsbury's
WildWest	Wild West

Figure 5.4: Brand Individuals Information on Excel

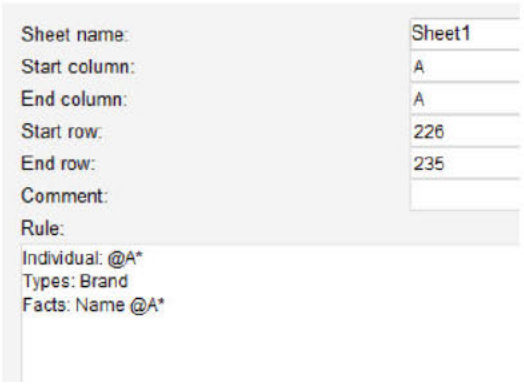


Figure 5.5: Rule to Add Brand Individuals

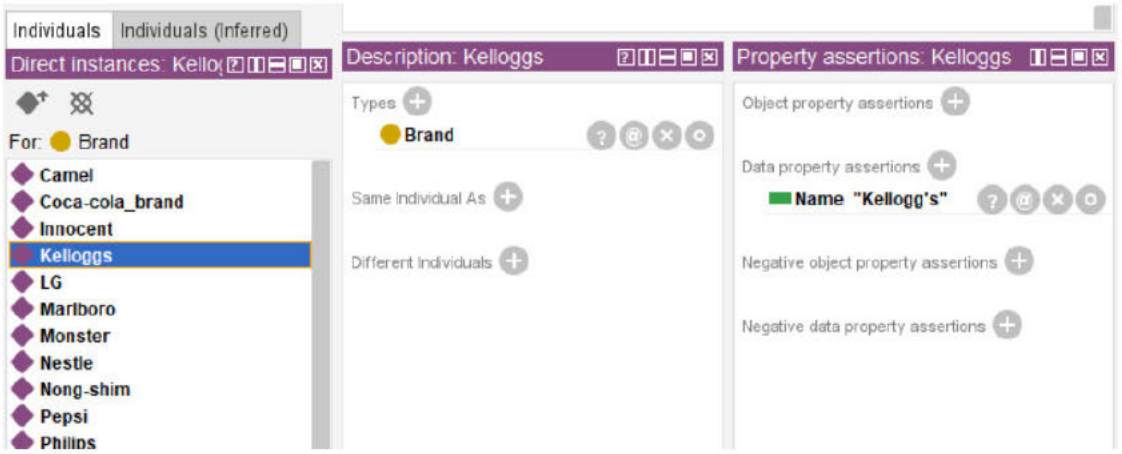


Figure 5.6: Aisle Individual Example from Protégé

- **Location:** There are 24 individuals added to the ontology.

Location		
individual_name	City	Country
location_1	New York	United States
location_2	Tokyo	Japan
location_3	London	United Kingdom
location_4	Paris	France
location_5	Sydney	Australia
location_6	Berlin	Germany
location_7	Rome	Italy
location_8	Madrid	Spain
location_9	Mumbai	India
location_10	São Paulo	Brazil
location_11	Cape Town	South Africa

Figure 5.7: Location Individuals Information on Excel

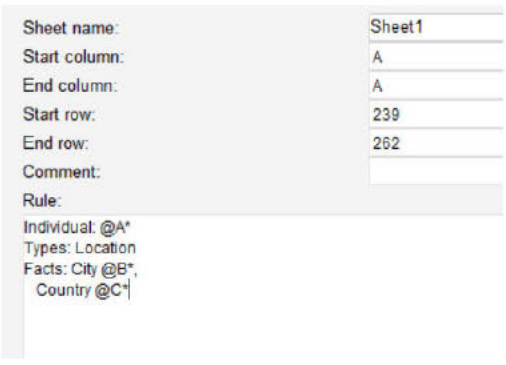


Figure 5.8: Rule to Add Location Individuals

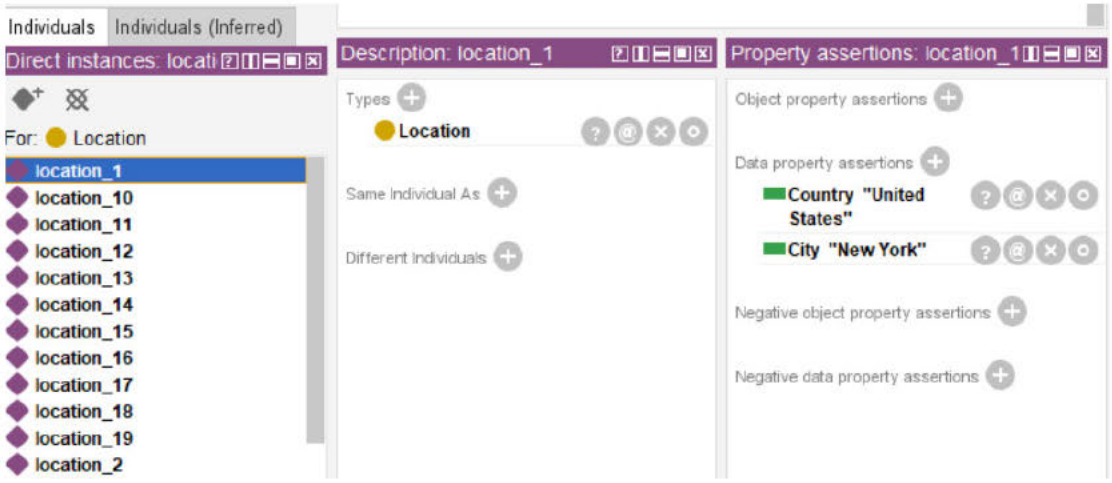


Figure 5.9: Location Individual Example from Protégé

- **Order:** There are 23 individuals added to the ontology.

Order		
individual_name	Order_no	Order_date
order_1	1001	1-11-24
order_2	1002	2-11-24
order_3	1003	3-11-24
order_4	1004	4-11-24
order_5	1005	5-11-24
order_6	1006	6-11-24
order_7	1007	7-11-24
order_8	1008	8-11-24
order_9	1009	9-11-24
order_10	1010	10-11-24
order_11	1011	1-11-24

Figure 5.10: Order Individuals Information on Excel

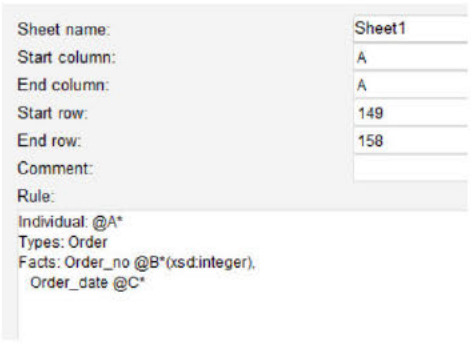


Figure 5.11: Rule to Add Order Individuals

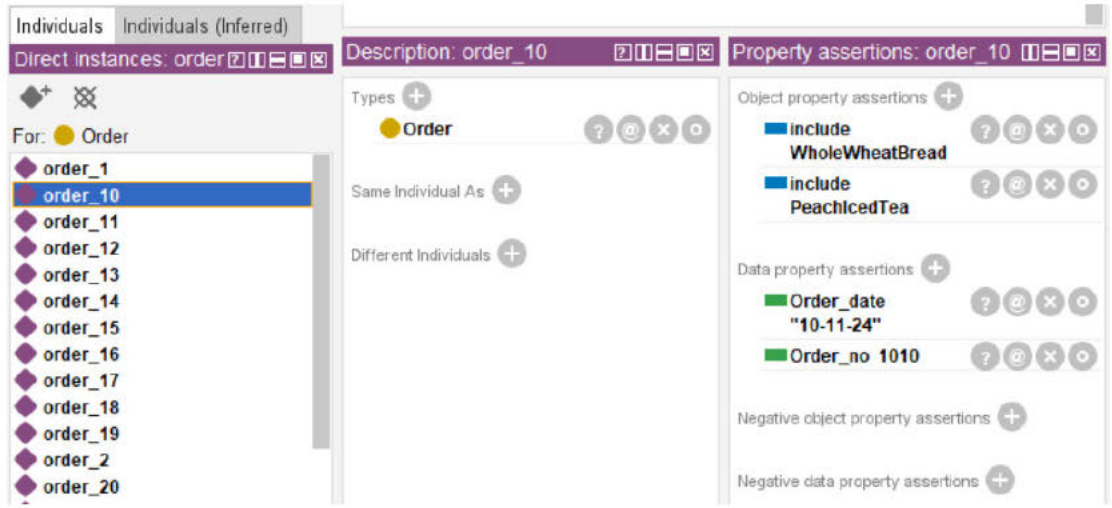


Figure 5.12: Order Individual Example from Protégé

- **Payment:** There are 20 individuals added to the ontology.

Payment	
individual_name	amount
payment_1	1.1
payment_2	45.8
payment_3	160.39
payment_4	35.4
payment_5	10
payment_6	6.9
payment_7	17.1
payment_8	1899.99
payment_9	650.9
payment_10	45.68

Figure 5.13: Payment Individuals Information on Excel

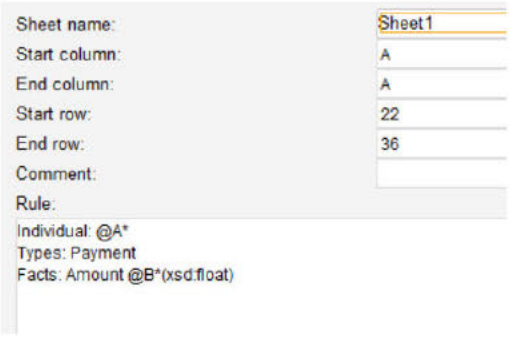


Figure 5.14: Rule to Add Payment Individuals

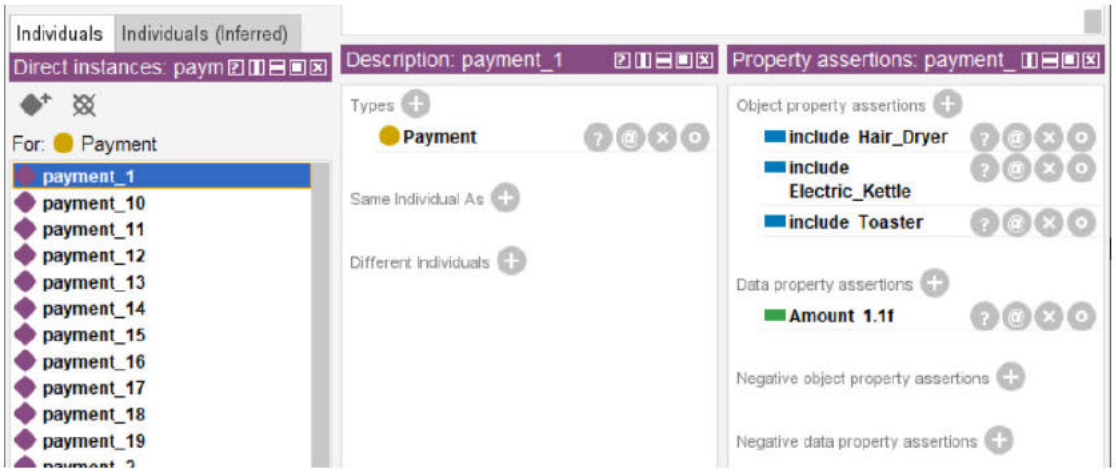


Figure 5.15: Order Individual Example from Protégé

- **People:** There are 10 individuals added to the ontology for each of the subclasses that is in total 60 People individuals. All subclasses of People class have the same data properties.

Member Customer					
individual_name	first_name	last_name	date_of_birth	age	gender
member_1	John	Smith	15-5-90	34	Male
member_2	Emma	Johnson	22-11-95	29	Female
member_3	Liam	Williams	5-3-88	36	Male
member_4	Sophia	Brown	10-7-00	24	Female
member_5	Noah	Jones	18-9-93	31	Male
member_6	Olivia	Garcia	30-4-97	27	Female
member_7	James	Martinez	12-8-92	32	Male
member_8	Ava	Davis	25-2-99	25	Female
member_9	Lucas	Hernandez	18-6-94	30	Male
member_10	Mia	Lopez	20-1-01	23	Female

Figure 5.16: Member Individuals Information on Excel

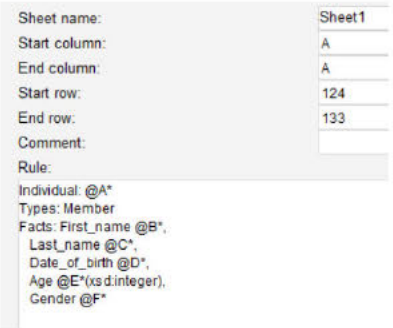


Figure 5.17: Rule to Add Member Individuals

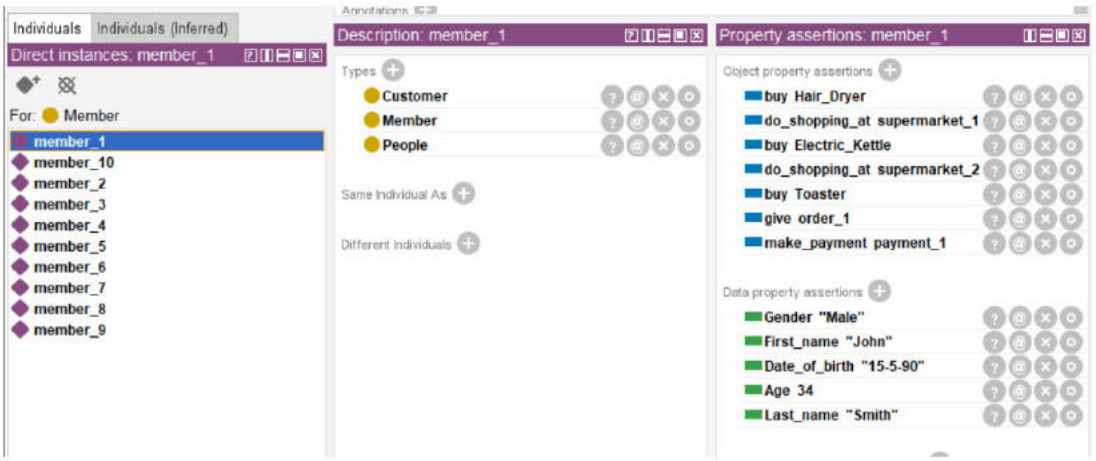


Figure 5.18: Member Individual Example from Protégé

Non-member Customer					
non-member_1	Ethan	Gonzalez	5-12-91	33	Male
non-member_2	Isabella	Wilson	15-7-96	28	Female
non-member_3	Mason	Anderson	28-9-89	35	Male
non-member_4	Charlotte	Thomas	11-3-03	21	Female
non-member_5	Logan	Taylor	22-10-98	26	Male
non-member_6	Amelia	Moore	14-2-90	34	Female
non-member_7	Elijah	Jackson	3-11-93	31	Male
non-member_8	Harper	Martin	21-8-97	27	Female
non-member_9	Alexander	Lee	17-12-87	37	Male
non-member_10	Evelyn	Perez	19-5-95	29	Female

Figure 5.19: Non-member Individuals Information on Excel

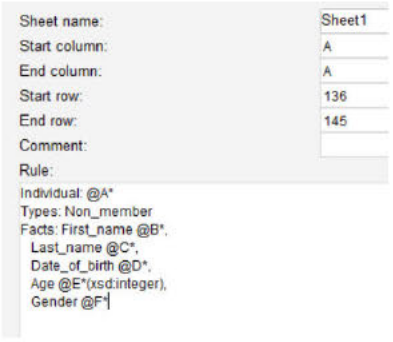


Figure 5.20: Rule to Add Non-member Individuals

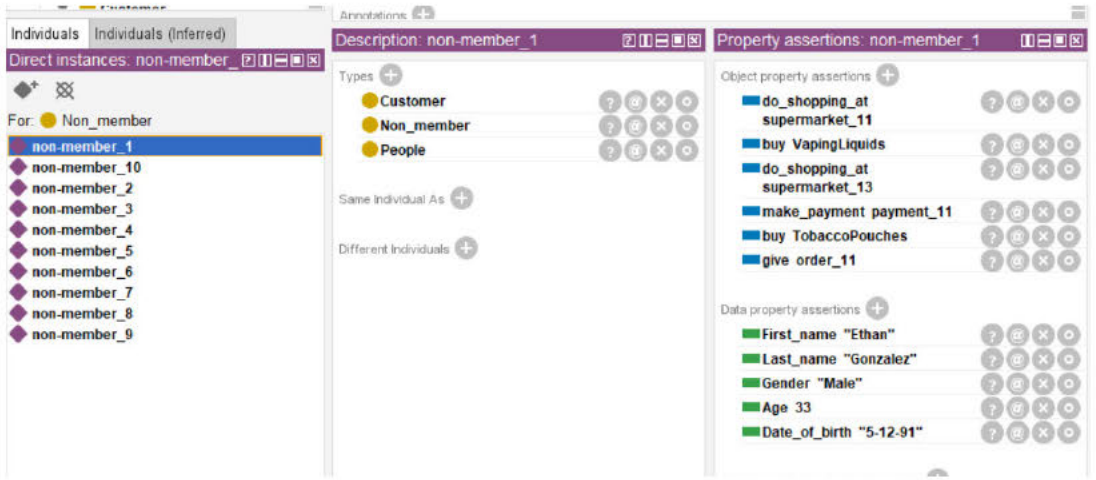


Figure 5.21: Non-member Individual Example from Protégé

Manager					
individual_name	first_name	last_name	date_of_birth	age	gender
manager_1	Ethan	Gonzalez	5-12-91	33	Male
manager_2	Isabella	Wilson	15-7-96	28	Female
manager_3	Mason	Anderson	28-9-89	35	Male
manager_4	Charlotte	Thomas	11-3-03	21	Female
manager_5	Logan	Taylor	22-10-98	26	Male
manager_6	Amelia	Moore	14-2-90	34	Female
manager_7	Elijah	Jackson	3-11-93	31	Male
manager_8	Harper	Martin	21-8-97	27	Female
manager_9	Alexander	Lee	17-12-87	37	Male
manager_10	Evelyn	Perez	19-5-95	29	Female

Figure 5.22: Manager Individuals Information on Excel

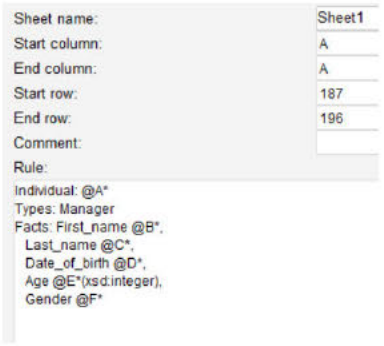


Figure 5.23: Rule to Add Manager Individuals

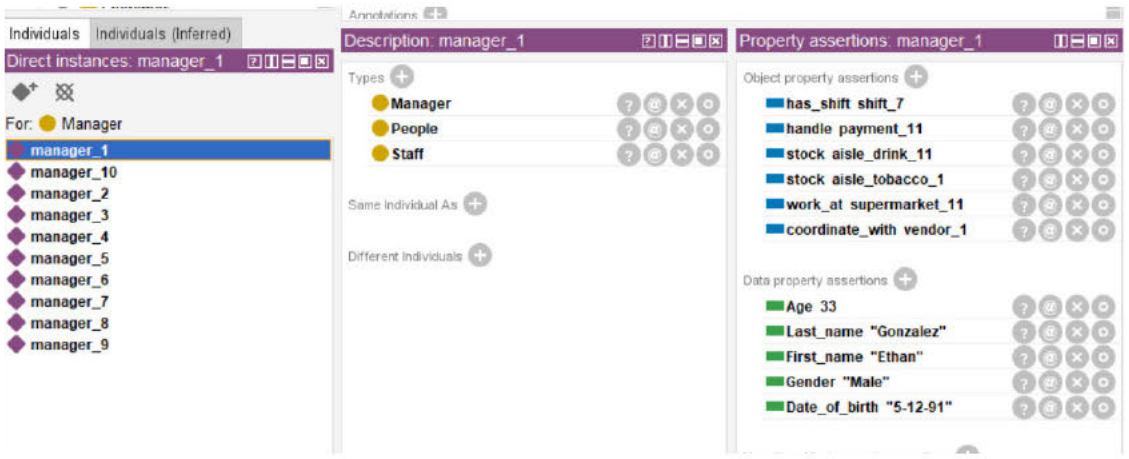


Figure 5.24: Manager Individual Example from Protégé

Team Leader					
individual_name	first_name	last_name	date_of_birth	age	gender
team_leader_1	Benjamin	White	11-11-86	38	Male
team_leader_2	Lily	Hall	25-3-94	30	Female
team_leader_3	Henry	Allen	14-7-92	32	Male
team_leader_4	Grace	Young	3-5-98	26	Female
team_leader_5	William	King	19-8-90	34	Male
team_leader_6	Hannah	Scott	8-6-02	22	Female
team_leader_7	Matthew	Adams	29-1-89	35	Male
team_leader_8	Zoe	Baker	12-12-97	27	Female
team_leader_9	Michael	Clark	10-4-93	31	Male
team_leader_10	Chloe	Nelson	7-11-01	23	Female

Figure 5.25: Team_leader Individuals Information on Excel

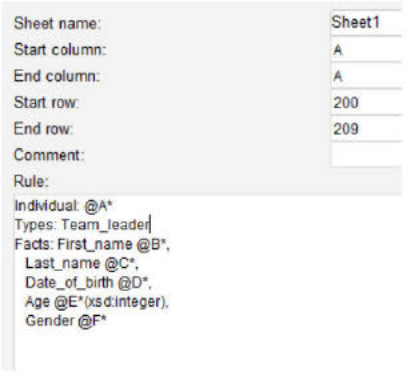


Figure 5.26: Rule to Add Team_leader Individuals

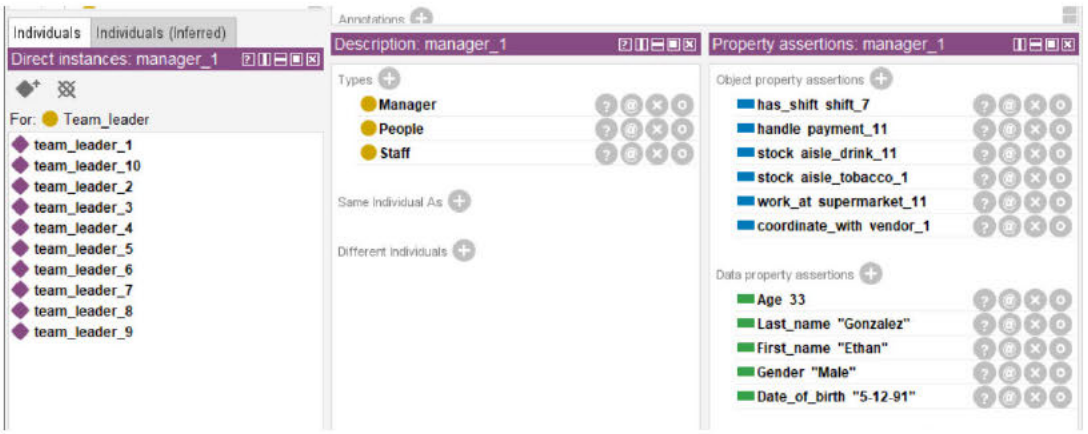


Figure 5.27: Team_leader Individual Example from Protégé

Team Member					
individual_name	first_name	last_name	date_of_birth	age	gender
team_member_1	John	Smith	15-5-90	34	Male
team_member_2	Emma	Johnson	22-11-95	29	Female
team_member_3	Liam	Williams	5-3-88	36	Male
team_member_4	Sophia	Brown	10-7-00	24	Female
team_member_5	Noah	Jones	18-9-93	31	Male
team_member_6	Olivia	Garcia	30-4-97	27	Female
team_member_7	James	Martinez	12-8-92	32	Male
team_member_8	Ava	Davis	25-2-99	25	Female
team_member_9	Lucas	Hernandez	18-6-94	30	Male
team_member_10	Mia	Lopez	20-1-01	23	Female

Figure 5.28: Team_member Individuals Information on Excel

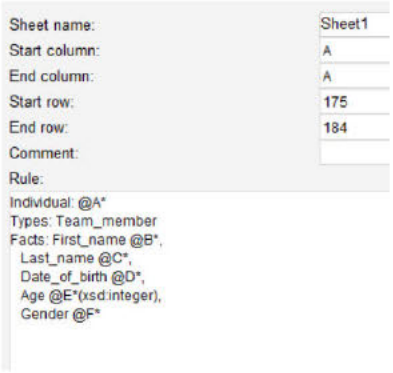


Figure 5.29: Rule to Add Team_member Individuals

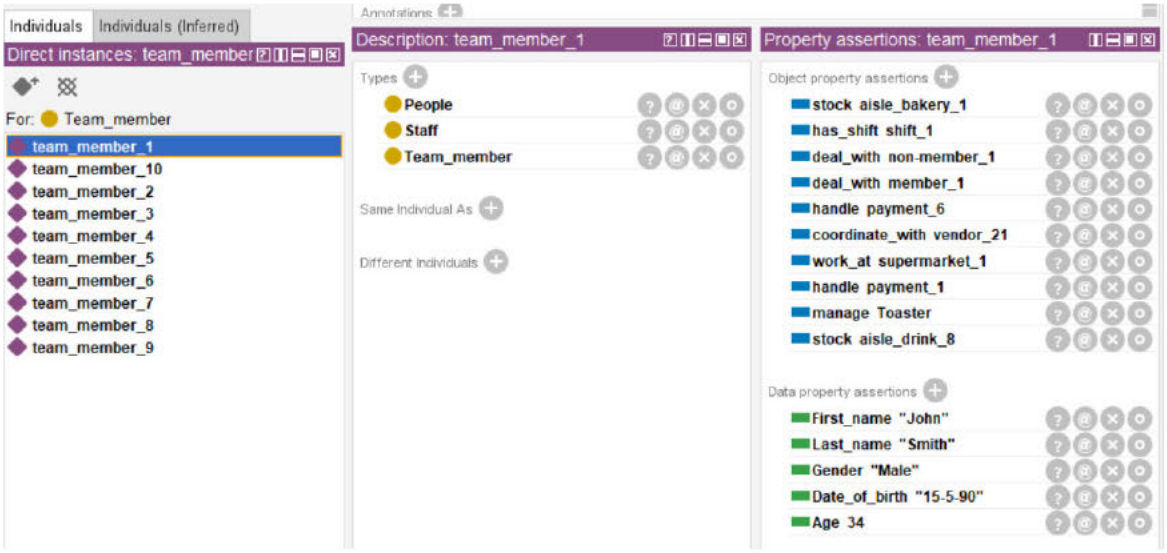


Figure 5.30: Team_member Individual Example from Protégé

Warehouse Worker					
individual_name	first_name	last_name	date_of_birth	age	gender
warehouse_worker_1	Daniel	Carter	5-10-88	36	Male
warehouse_worker_2	Ella	Mitchell	18-2-95	29	Female
warehouse_worker_3	David	Perez	9-3-91	33	Male
warehouse_worker_4	Emily	Roberts	30-9-99	25	Female
warehouse_worker_5	Samuel	Turner	1-12-90	34	Male
warehouse_worker_6	Scarlett	Collins	24-8-00	24	Female
warehouse_worker_7	Jackson	Edwards	15-5-96	28	Male
warehouse_worker_8	Abigail	Stewart	22-1-03	21	Female
warehouse_worker_9	Aiden	Morris	6-4-87	37	Male
warehouse_worker_10	Victoria	Rivera	26-7-94	30	Female

Figure 5.31: Warehouse_worker Individuals Information on Excel

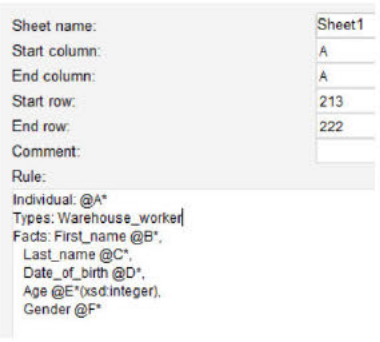


Figure 5.32: Rule to Add Warehouse_worker Individuals

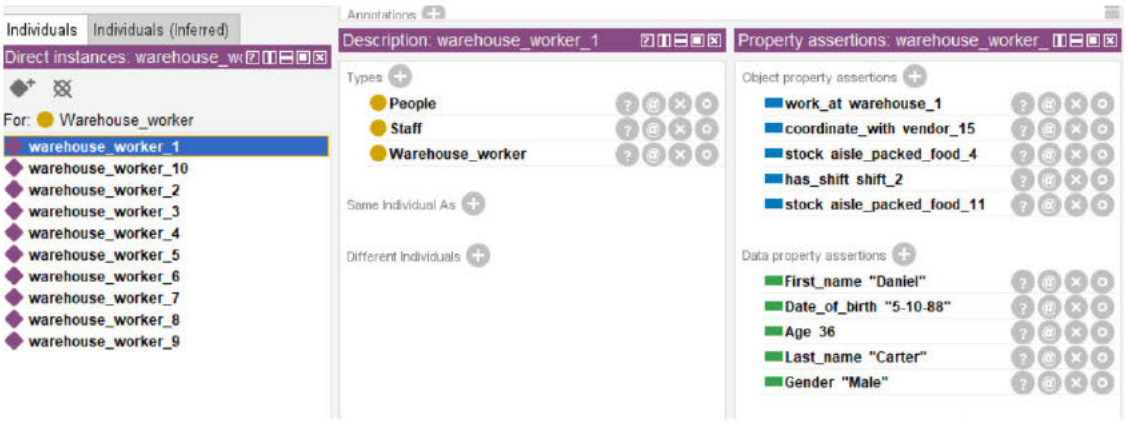


Figure 5.33: Warehouse_worker Individual Example from Protégé

- **Price:** There are 50 individuals added to the ontology, one for each Product individual. When values are generated, Product prices are considered.

Price		
individual_name	Member	Non-member
price_1	2.99	3.49
price_2	1.29	1.79
price_3	1.89	2.29
price_4	3.99	4.49
price_5	0.59	0.79
price_6	1.99	2.49
price_7	2.49	2.99
price_8	1.49	1.89
price_9	3.49	4.19
price_10	2.69	3.19
price_11	5.99	7.49
price_12	4.99	5.59

Figure 5.34: Price Individuals Information on Excel

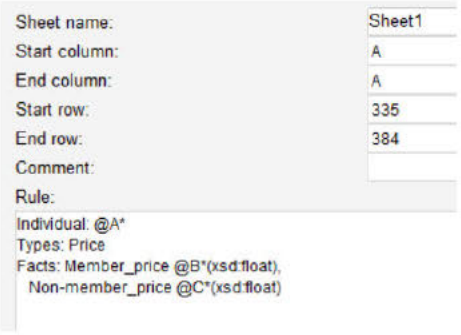


Figure 5.35: Rule to Add Price Individuals

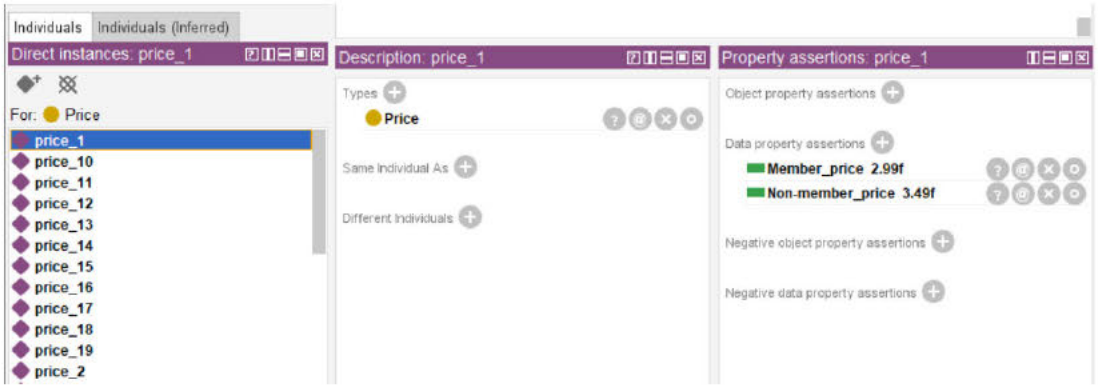


Figure 5.36: Price Individual Example from Protégé

- **Shift:** There are 10 individuals added to the ontology.

Shift		
individual_name	Shift_number	Working_hours
shift_1	1	08:00 - 16:00
shift_2	2	16:00 - 00:00
shift_3	3	00:00 - 08:00
shift_4	4	09:00 - 17:00
shift_5	5	10:00 - 18:00
shift_6	6	12:00 - 20:00
shift_7	7	07:00 - 15:00
shift_8	8	14:00 - 22:00
shift_9	9	22:00 - 06:00
shift_10	10	06:00 - 14:00

Figure 5.37: Shift Individuals Information on Excel

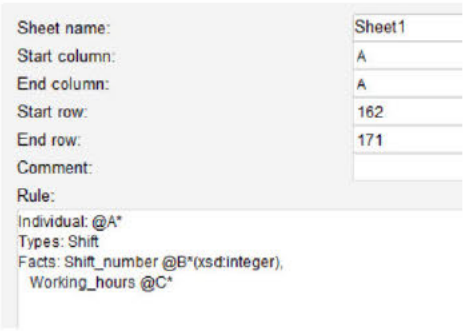


Figure 5.38: Rule to Add Shift Individuals

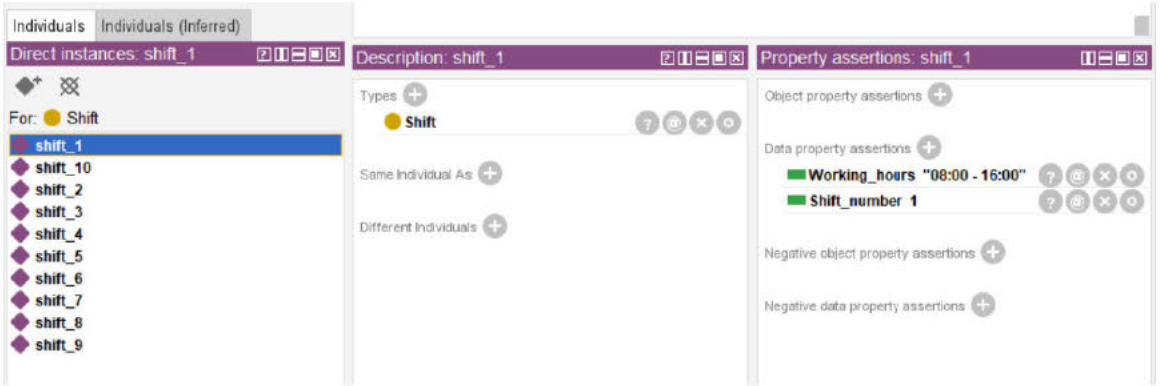


Figure 5.39: Shift Individual Example from Protégé

- **Stock:** There are 50 individuals added to the ontology, one for each Product individual.

Stock	
individual_name	quantity
stock_1	0
stock_2	0
stock_3	0
stock_4	0
stock_5	20
stock_6	50
stock_7	100
stock_8	35

Figure 5.40: Stock Individuals Information on Excel



Figure 5.41: Rule to Add Stock Individuals

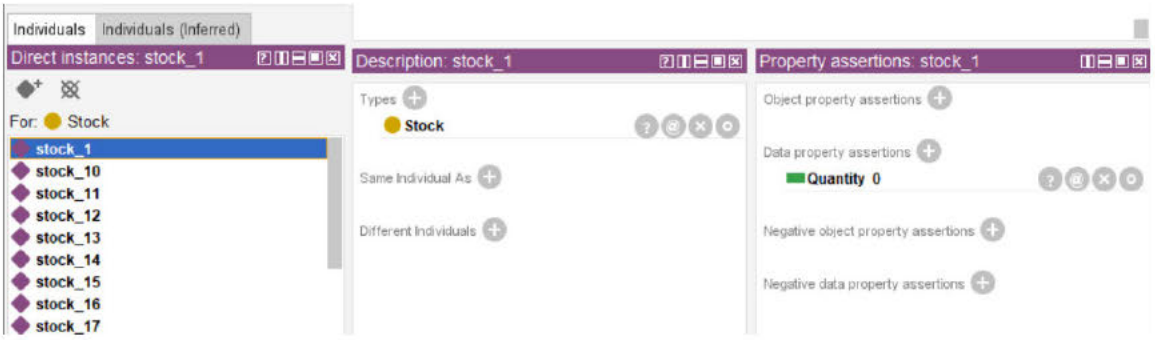


Figure 5.42: Stock Individual Example from Protégé

- **Product:** There are 10 individuals added to the ontology for each of the subclasses that are used when answering the questions in the List of Questions table. Therefore, 50 Product individuals exist in the ontology. Every individual has a name and a unique product_no while they have diverse additional data properties according to their types.

Drink				
individual name	Product No	Name	Calorie	Expiry Date
OrangeJuice	2001	Orange Juice	150	30-11-24
AppleJuice	2002	Apple Juice	120	10-12-24
GrapeJuice	2003	Grape Juice	140	15-12-24
Lemonade	2004	Lemonade	110	25-11-24
Coca-Cola	2005	Coca-Cola	150	5-12-24
PeachIcedTea	2006	Peach Iced Tea	90	1-12-24
CoconutWater	2007	Coconut Water	60	15-1-25
Milk	2008	Milk	100	28-11-24
EnergyDrink	2009	Energy Drink	200	18-11-24
SparklingWater	2010	Sparkling Water	0	1-3-25

Figure 5.43: Drink Individuals Information on Excel

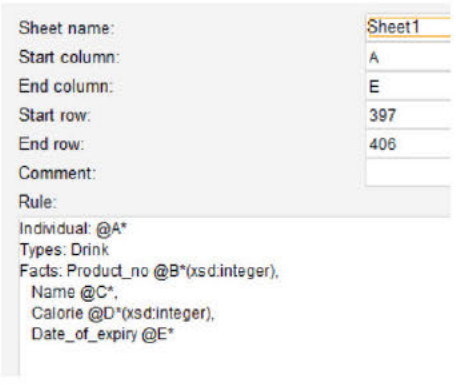


Figure 5.44: Rule to Add Drink Individuals

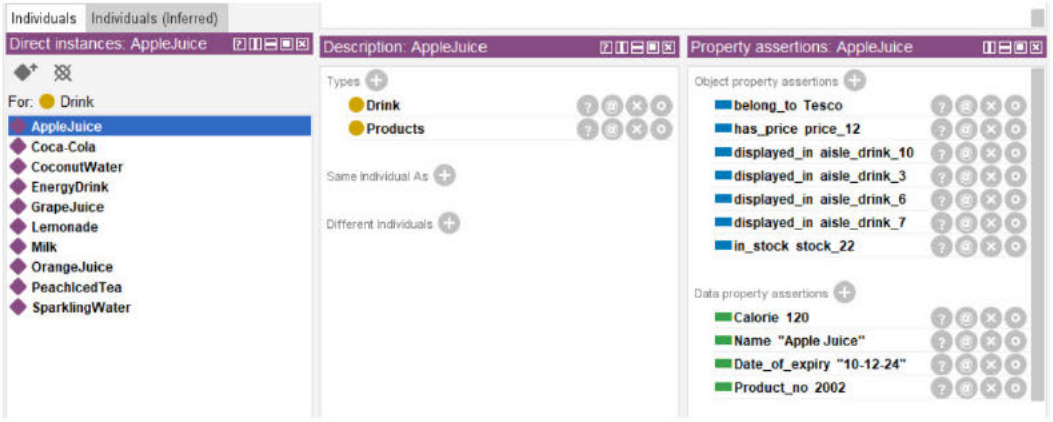


Figure 5.45: Drink Individual Example from Protégé

Electronics				
individual_name	name	watts	voltage	product_no
Refrigerator	Refrigerator	800	220-240	5001
Microwave	Microwave	1000	220-240	5002
Toaster	Toaster	750	220-240	5003
Electric_Kettle	Electric Kettle	1500	220-240	5004
Hair_Dryer	Hair Dryer	2000	220-240	5005
Smartphone	Smartphone	5	100-240	5006
Portable_Heater	Portable Heater	2000	220-240	5007
LED_TV	LED TV	80	100-240	5008
Vacuum_Cleaner	Vacuum Cleaner	1600	220-240	5009
Blender	Blender	600	220-240	5010

Figure 5.46: Electronics Individuals Information on Excel

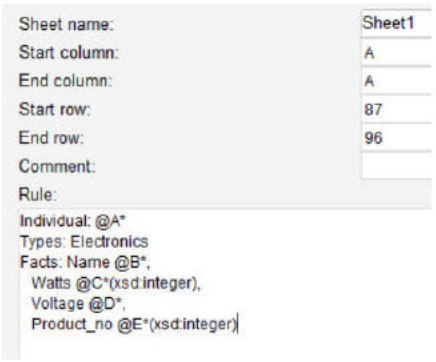


Figure 5.47: Rule to Add Electronics Individuals

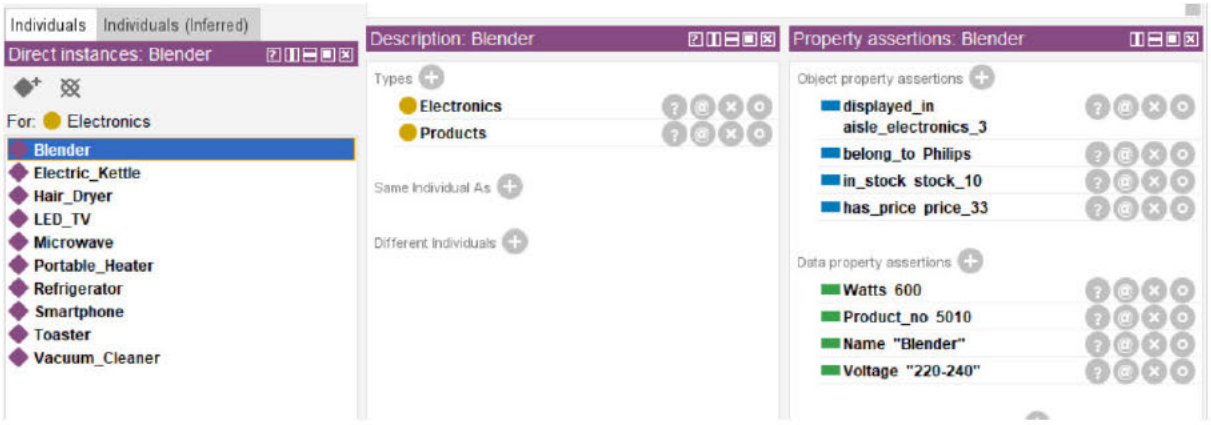


Figure 5.48: Electronics Individual Example from Protégé

Bakery				
individual name	Product No	Name	Calorie	Expiry Date
SourdoughBread	4001	Sourdough Bread	180	22-11-24
Croissant	4002	Croissant	230	18-11-24
WholeWheatBread	4003	Whole Wheat Bread	150	20-11-24
ChocolateChipMuffin	4004	Chocolate Chip Muffin	350	24-11-24
Bagels	4005	Bagels	220	21-11-24
CinnamonRolls	4006	Cinnamon Rolls	270	17-11-24
LemonPoundCake	4007	Lemon Pound Cake	300	23-11-24
RaisinBread	4008	Raisin Bread	210	19-11-24
GarlicBread	4009	Garlic Bread	250	16-11-24
BlueberryMuffin	4010	Blueberry Muffin	320	20-11-24

Figure 5.49: Bakery Individuals Information on Excel

Sheet name:	Sheet1
Start column:	A
End column:	E
Start row:	423
End row:	432
Comment:	
Rule:	

```
Individual: @A*
Types: Bakery
Facts: Product_no @B*(xsd:integer),
  Name @C*,
  Calorie @D*(xsd:integer),
  Date_of_expiry @E*
```

Figure 5.50: Rule to Add Bakery Individuals

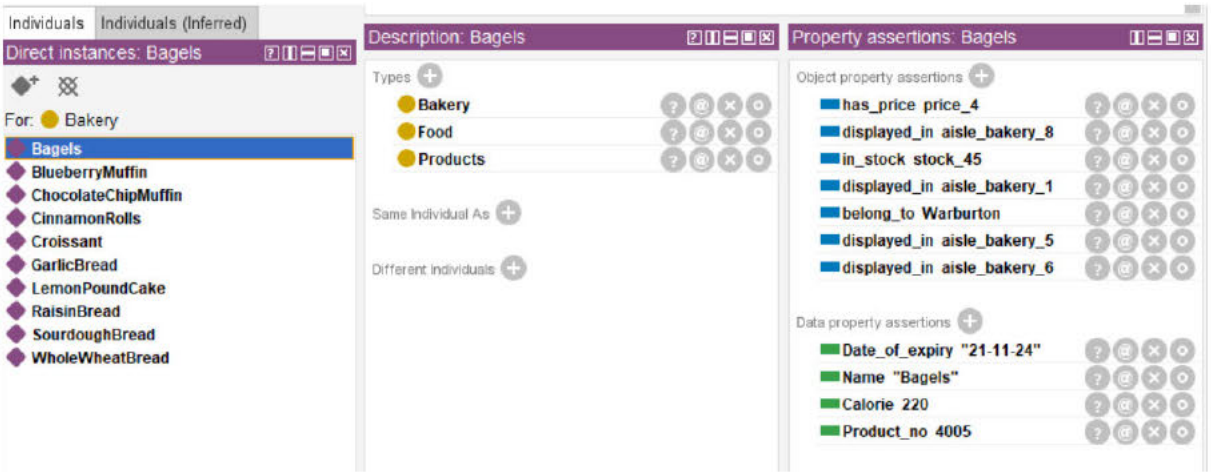


Figure 5.51: Bakery Individual Example from Protégé

Packed food				
individual name	Product No	Name	Calorie	Expiry Date
InstantNoodles	3001	Instant Noodles	350	15-2-25
CannedTuna	3002	Canned Tuna	200	20-6-26
CannedBeans	3003	Canned Beans	150	30-3-25
FrozenPizza	3004	Frozen Pizza	800	5-12-24
MicrowavePopcorn	3005	Microwave Popcorn	200	1-12-24
GranolaBars	3006	Granola Bars	120	10-11-24
Ready-to-eatRice	3007	Ready-to-eat Rice	400	28-2-25
CannedSoup	3008	Canned Soup	250	15-5-25
BeefJerky	3009	Beef Jerky	300	10-1-25
FruitSnacks	3010	Fruit Snacks	100	20-11-24

Figure 5.52: Packed_food Individuals Information on Excel

Sheet name:	Sheet1
Start column:	A
End column:	E
Start row:	410
End row:	419
Comment:	
Rule:	

```
Individual: @A*
Types: Packed_food
Facts: Product_no @B*(xsd:integer),
  Name @C*,
  Calorie @D*(xsd:integer),
  Date_of_expiry @E*
```

Figure 5.53: Rule to Add Packed_food Individuals

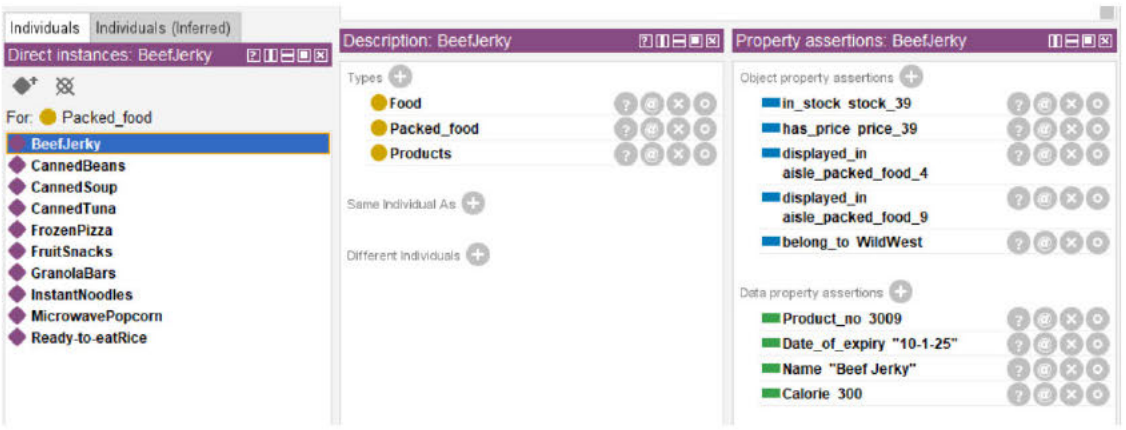


Figure 5.54: Packed_food Individual Example from Protégé

Tobacco Products		
individual_name	Name	Product_no
Cigarettes	Cigarettes	1001
Cigars	Cigars	1002
PipeTobacco	Pipe Tobacco	1003
ChewingTobacco	Chewing Tobacco	1004
Snuff-Dry_or_Moist	Snuff (Dry or Moist)	1005
LooseLeafTobacco	Loose Leaf Tobacco	1006
Tobacco Pouches	Tobacco Pouches	1007
CigaretteRollingPaper	Cigarette Rolling Pa	1008
TobaccoSticks	Tobacco Sticks	1009
VapingLiquids	Vaping Liquids	1010

Figure 5.55: Tobacco Individuals Information on Excel

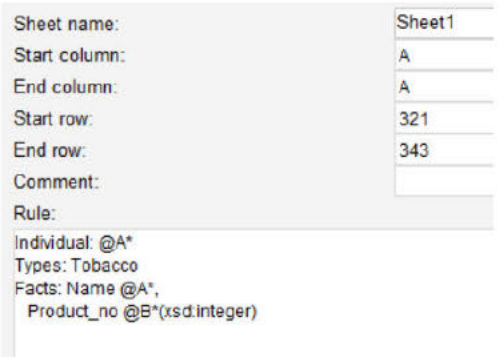


Figure 5.56: Rule to Add Tobacco Individuals

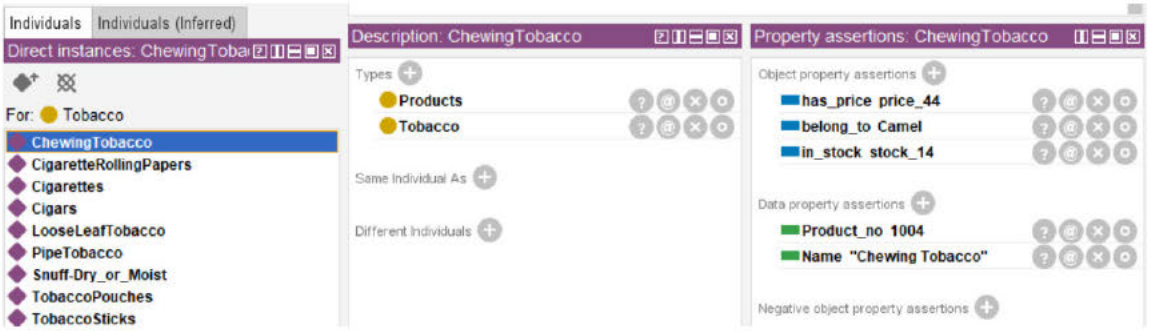


Figure 5.57: Tobacco Individual Example from Protégé

- **Supermarket:** There are 15 individuals added to the ontology.

Supermarket				(in billion)
individual_name	name	opening_year	operating_hours	revenue
supermarket_1	Tesco Extra	1905	6 AM - Midnight	65.70
supermarket_2	Sainsbury's Sup	1905	7 AM - 11 PM	33.80
supermarket_3	Asda Supercentı	1905	6 AM - Midnight	23.30
supermarket_4	Morrisons	1905	7 AM - 10 PM	21.00
supermarket_5	Aldi	1905	8 AM - 10 PM	17.80
supermarket_6	Lidl	1905	8 AM - 10 PM	10.60
supermarket_7	Co-op	1905	7 AM - 11 PM	13.10
supermarket_8	Waitrose	1905	8 AM - 9 PM	8.10
supermarket_9	Iceland	1905	8 AM - 8 PM	4.10
supermarket_10	Marks & Spence	1905	7 AM - 10 PM	6.60
supermarket_11	SPAR	1905	7 AM - 10 PM	3.70
supermarket_12	Farmfoods	1905	9 AM - 8 PM	1.00
supermarket_13	Lidl	1905	8 AM – 9 PM	10.60
supermarket_14	Co-op	1905	7 AM – 10 PM	13.11
supermarket_15	Waitrose	1905	8 AM - 9 PM	8.12

Figure 5.58: Supermarket Individuals Information on Excel

Sheet name:	Sheet1
Start column:	A
End column:	A
Start row:	3
End row:	17
Comment:	
Rule:	

```
Individual: @A*
Types: Supermarket
Facts: Name @B*,
  Opening_year @C*(xsd:integer),
  Operating_hours @D*,
  Revenue @E*(xsd:float)
```

Figure 5.59: Rule to Add Supermarket Individuals

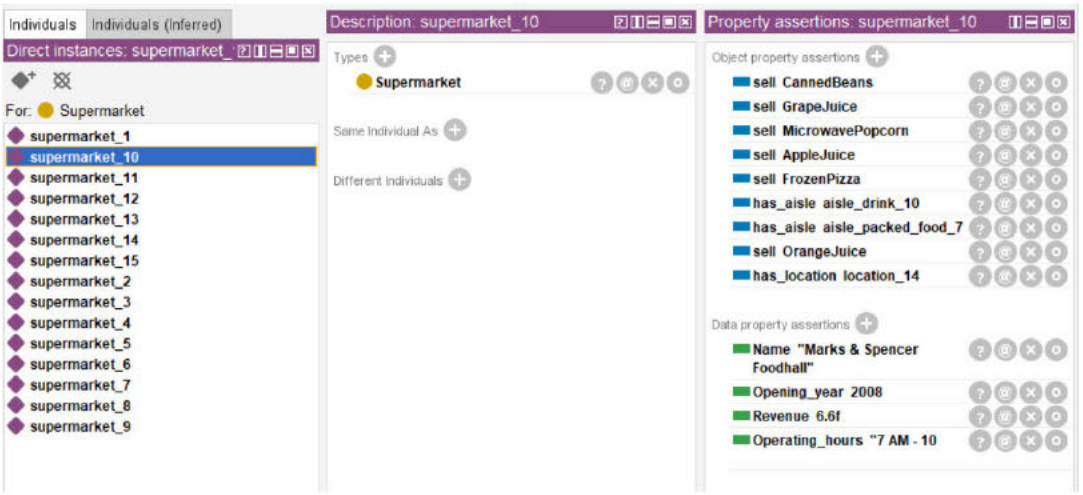


Figure 5.60: Supermarket Individual Example from Protégé

- **Vendor:** There are 23 individuals added to the ontology.

Vendor	
individual_name	name
vendor_1	VendorPro Solutions
vendor_2	PrimeSupply Partners
vendor_3	QuickTrade Vendors
vendor_4	MarketLink Distributors
vendor_5	SourceHub Supplies
vendor_6	EliteVendor Group
vendor_7	TradeWave Solutions
vendor_8	SupplyCraft Enterprises
vendor_9	SupplySync Vendors
vendor_10	FreshLink Trading

Figure 5.61: Vendor Individuals Information on Excel

Sheet name:	Sheet1
Start column:	A
End column:	A
Start row:	347
End row:	369
Comment:	
Rule:	

```
Individual: @A*
Types: Vendor
Facts: Name @B*
```

Figure 5.62: Rule to Add Vendor Individuals

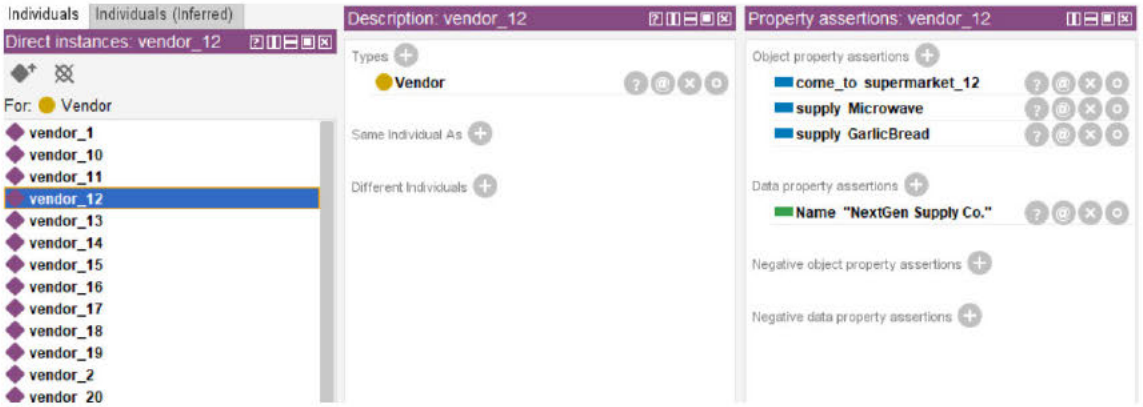


Figure 5.63: Vendor Individual Example from Protégé

- **Warehouse:** There are 23 individuals added to the ontology.

Warehouse			
indivdual_name	Name	Opening years	Operating Hours
warehouse_1	FreshStock Wareho	1987	8 am - 6 pm
warehouse_2	GreenMarket Supply	1670	7:30am - 9pm
warehouse_3	PrimePick Warehou	2000	9am - 4pm
warehouse_4	SuperSave Depot	1784	12pm - 10pm
warehouse_5	Harvest Hub	1729	6am - 3pm
warehouse_6	QuickServe Storage	1876	6:30am - 7pm
warehouse_7	Grocery Vault	1964	4am - 4pm
warehouse_8	FoodFlow Warehous	1954	5am - 3pm
warehouse_9	MarketMax Wareho	1980	6am - 6pm

Figure 5.64: Warehouse Individuals Information on Excel

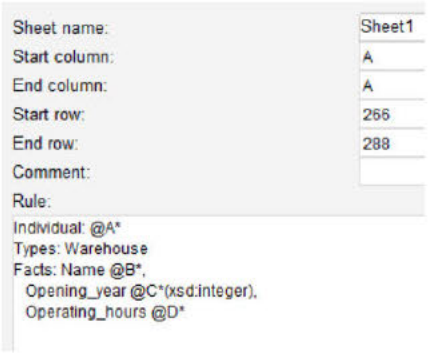


Figure 6.65: Rule to Add Warehouse Individuals

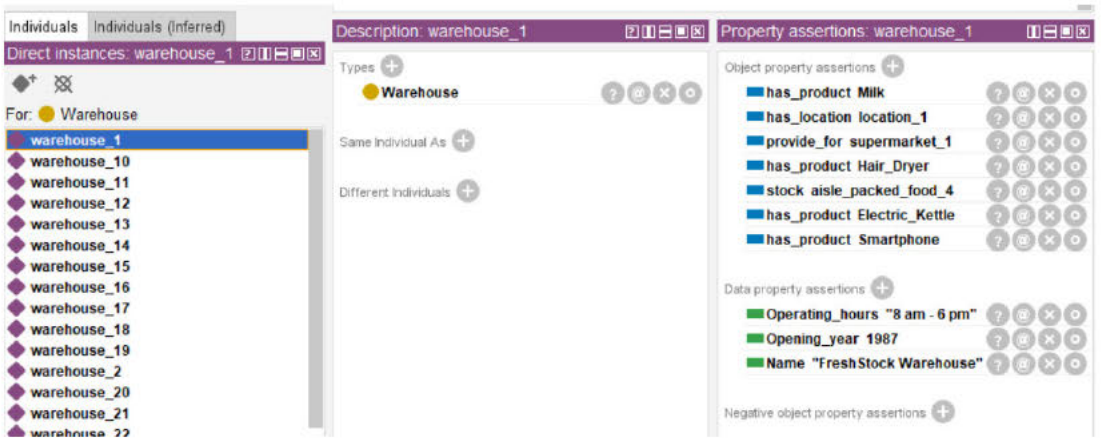


Figure 5.66: Warehouse Individual Example from Protégé

Connecting Individuals using Object Properties:

While individuals are created in Protégé, data properties are also specified for each of them. After this process, building a connection between these individuals is a need. For this purpose, object properties are used and links are constructed according to domains and ranges of the object properties that is remarked before in the Supermarket Ontology.

This operation is done in a similar way of creating individuals and adding data properties to them and it is writing rules for each object property. Connections are constructed in the Excel sheet first. The most important point of that process is to use individual names (not any value of individuals) when matching individuals. All rules are drafted in a same template: in the first column, there is domain and in the second column, there is range. The only thing changing is the type of the object property.

Below are the examples of the Excel sheet tables and rules written.

- Object property: has_location

has_location	
Supermarket IN	Location IN
supermarket_1	location_1
supermarket_2	location_2
supermarket_3	location_3
supermarket_4	location_14
supermarket_5	location_6
supermarket_6	location_6

Figure 6.1: " Supermarket has a Location" *connection on Excel sheet*

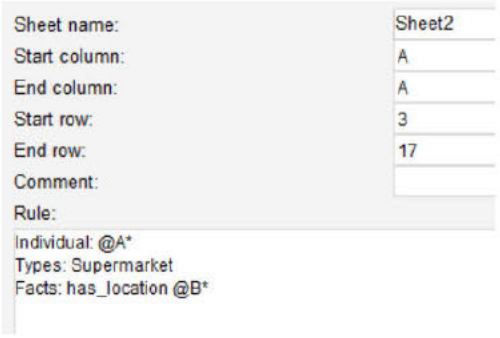


*Figure 6.2: Rule to add "*Supermarket has a Location" *connection*

warehouse_1	location_1
warehouse_2	location_3
warehouse_3	location_3
warehouse_4	location_1
warehouse_5	location_1
warehouse_6	location_1

Figure 6.3: " Warehouse has a Location" *connection on Excel sheet*



*Figure 6.4: Rule to add "*Warehouse has a Location" *connection*

- Object property: belong_to

belong_to	
Refrigerator	Samsung
Microwave	Samsung
Toaster	Philips
Electric_Kettle	Philips
Hair_Dryer	Philips
Smartphone	Samsung
Portable_Heater	Sony

*Figure 6.5: "*Products belong to Brand." *connection on Excel sheet*

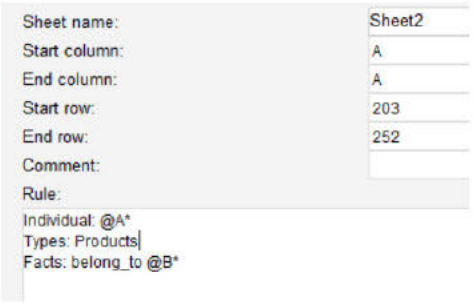


*Figure 6.6: Rule to add "*Products belong to Brand." *connection*

- Object property: sell

sell	
supermarket_1	Refrigerator
supermarket_1	Electric_Kettle
supermarket_3	Refrigerator
supermarket_3	Microwave
supermarket_1	Hair_Dryer
supermarket_3	Toaster

*Figure 6.7: "*Supermarket sell Products." *connection on Excel sheet*



*Figure 6.8: Rule to add "*Supermarket sell Products." *connection*

- Object property: work_at

work_at	
Staff	Supermarket - Warehouse
team_member_1	supermarket_1
manager_1	supermarket_11
team_leader_1	supermarket_1
warehouse_worker_1	warehouse_1
team_member_2	supermarket_2
team_member_3	supermarket_3

*Figure 6.9: "*Staff works at Supermarket/Warehouse." *connection on Excel sheet*



*Figure 6.10: Rule to add "*Staff works at Supermarket/Warehouse." *connection*

- Object property: has_aisle

has_aisle	
supermarket_1	aisle_bakery_1
supermarket_1	aisle_electronics_1
supermarket_1	aisle_packed_food_1
supermarket_1	aisle_drink_2
supermarket_2	aisle_tobacco_1
supermarket_2	aisle_drink_3
supermarket_2	aisle_packed_food_2
supermarket_3	aisle_drink_4

*Figure 6.11: "*Supermarket has Aisle." *connection on Excel sheet*

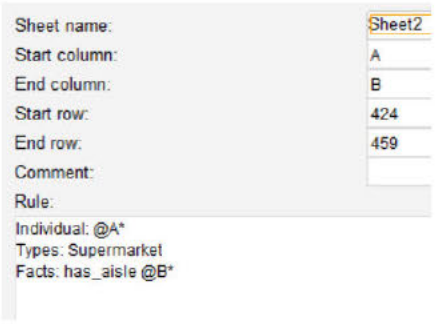


*Figure 6.12: Rule to add "*Supermarket has Aisle." *connection*

- Object property: displayed_in

displayed_in	
Electric_Kettle	aisle_electronics_1
Hair_Dryer	aisle_electronics_1
Milk	aisle_drink_2
EnergyDrink	aisle_drink_2
CannedTuna	aisle_packed_food_1
GranolaBars	aisle_packed_food_1
Croissant	aisle_bakery_1

*Figure 6.13: "*Products are displayed in Aisle." *connection on Excel sheet*

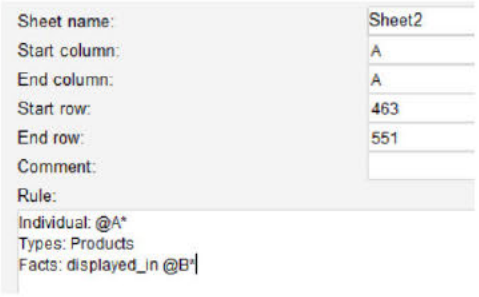


*Figure 6.14: Rule to add "*Products are displayed in Aisle." *connection*

- Object property: do_shopping_at

do_shopping_at	
individual_name	
member_1	supermarket_1
member_1	supermarket_2
member_2	supermarket_5
member_2	supermarket_2
non-member_1	supermarket_13
non-member_1	supermarket_11
non-member_2	supermarket_14
non-member_2	supermarket_12
non-member_3	supermarket_15
non-member_3	supermarket_13

*Figure 6.15: "*Customer do shopping at Supermarket." *connection on Excel sheet*

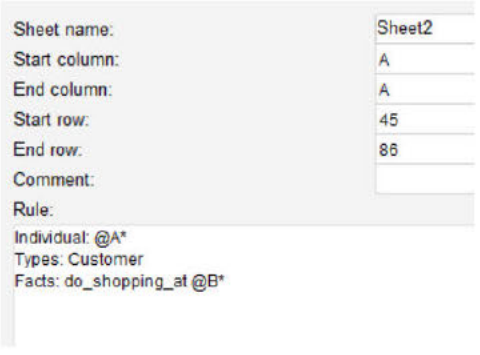


*Figure 6.16: Rule to add "*Customers do shopping at Supermarket." *connection*

- Object property: buy

buy	
member_1	Toaster
member_1	Electric_Kettle
member_1	Hair_Dryer
member_2	Smartphone
member_2	Portable_Heater
member_2	Electric_Kettle
non-member_2	Blender
non-member_1	VapingLiquids

*Figure 6.17: "*Customer buy Products.*" connection on Excel sheet*

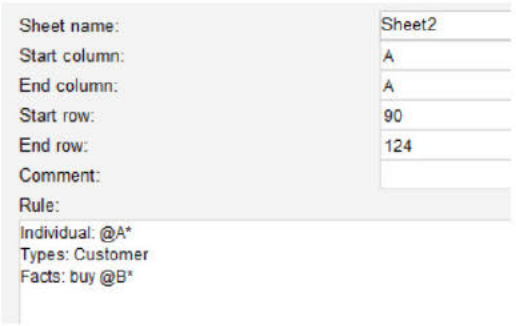


*Figure 6.18: Rule to add "*Customer buy Products.*" connection*

- Object property: deal_with

deal_with	
team_member_1	member_1
team_member_2	member_2
team_member_3	member_3
team_member_1	non-member_1
team_member_2	non-member_2
team_member_3	non-member_3

*Figure 6.19: "*Staff deals with Customers.*" connection on Excel sheet*

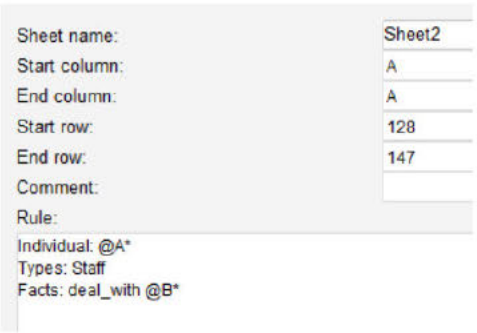


*Figure 6.20: Rule to add "*Staff deals with Customers.*" connection*

- Object property: manage

manage	
team_member_1	Toaster
team_member_2	Electric_Kettle
team_leader_1	PipeTobacco
team_leader_2	ChewingTobacco
team_member_3	Hair_Dryer

*Figure 6.21: "*Staff manages Products.*" connection on Excel sheet*



*Figure 6.22: Rule to add "*Staff manages Products.*" connection*

- Object property: make_payment

make_payment	
member_1	payment_1
member_2	payment_2
member_3	payment_3
non-member_1	payment_11
non-member_2	payment_12
non-member_3	payment_13
non-member_4	payment_14
non-member_5	payment_15

*Figure 6.23: "*Customer makes Payment.*" connection on Excel sheet*

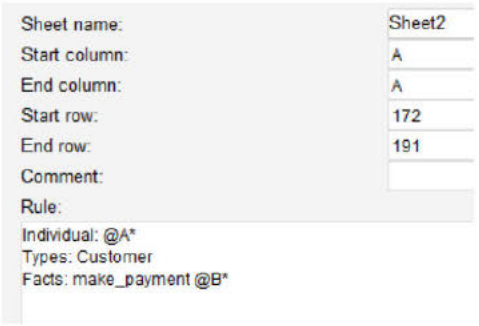


*Figure 6.24: Rule to add "*Customer makes Payment.*" connection*

- Object property: include

include	
payment_1	Toaster
payment_1	Electric_Kettle
payment_1	Hair_Dryer
payment_2	Smartphone
payment_2	Portable_Heater
payment_2	Electric_Kettle
payment_3	LED_TV
payment_3	Hair_Dryer
payment_4	Vacuum_Cleaner

*Figure 6.25: "*Payment include Products.*" connection on Excel sheet*

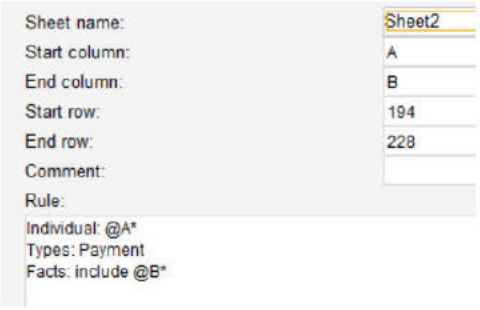


*Figure 6.26: Rule to add "*Payment include Products.*" connection*

include	
order	products
order_1	Cigarettes
order_1	PeachIcedTea
order_2	GrapeJuice
order_2	GranolaBars
order_2	OrangeJuice
order_3	Vacuum_Cleaner
order_3	LED_TV
order_4	InstantNoodles
order_4	CannedTuna
order_5	FruitSnacks
order_5	Ready-to-eatRice

*Figure 6.27: "*Order include Products.*" connection on Excel sheet*

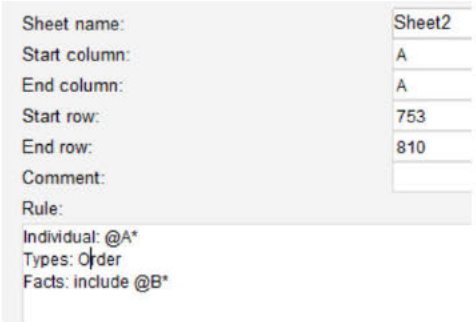


*Figure 6.28: Rule to add "*Order include Products.*" connection*

- Object property: handle

handle	
team_member_1	payment_1
team_member_2	payment_2
team_member_3	payment_3
manager_1	payment_11
manager_2	payment_12
manager_3	payment_13

Figure 6.29: "Staff handles Payment." connection on Excel sheet

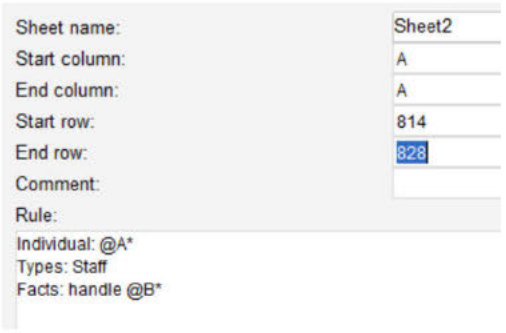


Figure 6.30: Rule to add "Staff handles Payment." connection

- Object property: supply

supply	
vendor_1	LooseLeafTobacco
vendor_1	Snuff-Dry_or_Moist
vendor_2	Ready-to-eatRice
vendor_2	SourdoughBread
vendor_2	BeefJerky
vendor_3	EnergyDrink
vendor_3	CannedTuna
vendor_4	WholeWheatBread
vendor_4	FrozenPizza
vendor_4	Croissant

Figure 6.31: "Vendor supplies Products." connection on Excel sheet

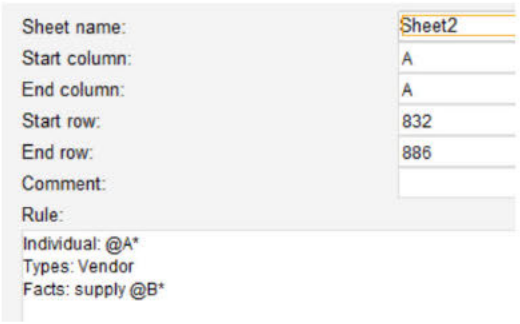


Figure 6.32: Rule to add "Vendor supplies Products." connection

- Object property: give

give	
member_1	order_1
member_2	order_2
member_3	order_3
non-member_1	order_11
non-member_2	order_12
non-member_3	order_13
non-member_4	order_14
non-member_5	order_15

Figure 6.33: "Customer give Order." connection on Excel sheet

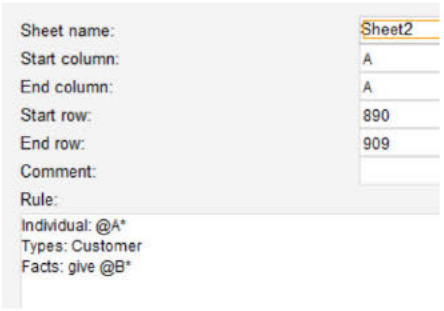


Figure 6.34: Rule to add "Customer give Order." connection

- Object property: has_shift

has_shift	
team_member_1	shift_1
team_member_2	shift_2
manager_1	shift_7
team_leader_1	shift_7
warehouse_worker_1	shift_2
team_member_3	shift_3
team_member_4	shift_4

Figure 6.35: "Staff has Shift." connection on Excel sheet

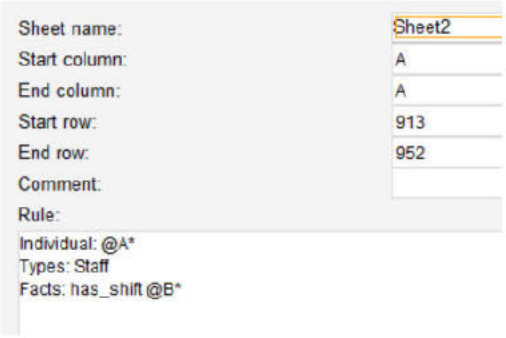


Figure 6.36: Rule to add "Staff has Shift." connection

- Object property: stock

stock	
team_member_1	aisle_bakery_1
team_member_2	aisle_bakery_2
team_leader_1	aisle_packed_food_2
manager_1	aisle_tobacco_1
warehouse_worker_1	aisle_packed_food_4
team_member_3	aisle_electronics_1

Figure 6.37: "Staff stocks Aisle." connection on Excel sheet

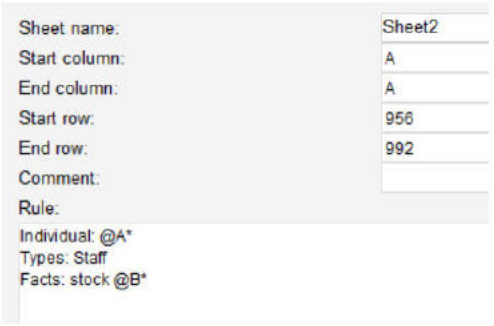


Figure 6.38: Rule to add "Staff stocks Aisle." connection

- Object property: has_product

has	
warehouse_1	Hair_Dryer
warehouse_1	Smartphone
warehouse_1	Milk
warehouse_1	Electric_Kettle
warehouse_2	OrangeJuice
warehouse_2	AppleJuice
warehouse_2	VapingLiquids
warehouse_2	FrozenPizza

Figure 6.39: "Warehouse has Products." connection on Excel sheet



Figure 6.40: Rule to add "Warehouse has Products." connection

- Object property: provide_for

provide_for	
warehouse_1	supermarket_1
warehouse_2	supermarket_2
warehouse_3	supermarket_3
warehouse_4	supermarket_4
warehouse_5	supermarket_5
warehouse_6	supermarket_6
warehouse_7	supermarket_7
warehouse_8	supermarket_8

*Figure 6.41: "*Warehouse provides for Supermarket." *connection on Excel sheet*

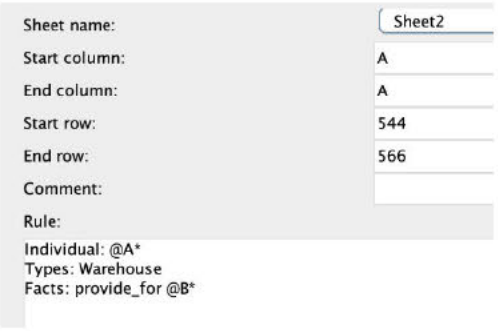


*Figure 6.42: Rule to add "*Warehouse provides for Supermarket." *connection*

- Object property: come_to

come_to	
vendor_1	supermarket_1
vendor_2	supermarket_2
vendor_3	supermarket_3
vendor_4	supermarket_4
vendor_5	supermarket_5
vendor_6	supermarket_6
vendor_7	supermarket_7
vendor_8	supermarket_8

*Figure 6.43: "*Vendor comes to Supermarket." *connection on Excel sheet*

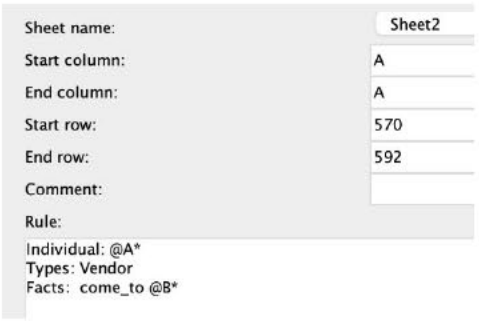


*Figure 6.44: Rule to add "*Vendor comes to Supermarket." *connection*

- Object property: coordinate_with

coordinate_with	
manager_1	vendor_1
team_leader_1	vendor_11
team_member_1	vendor_21
manager_2	vendor_2
manager_3	vendor_3
manager_4	vendor_4
manager_5	vendor_5

*Figure 6.45: "*Staff coordinates with Vendor." *connection on Excel sheet*

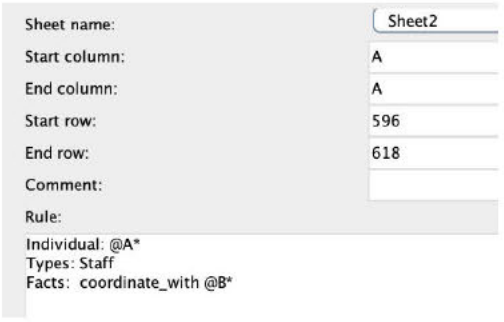


*Figure 6.46: Rule to add "*Staff coordinates with Vendor." *connection*

- Object property: assign_work

assign_work	
team_leader_1	team_member_1
team_leader_2	team_member_2
team_leader_3	team_member_3
team_leader_4	team_member_4
team_leader_5	team_member_5
team_leader_6	team_member_6

*Figure 6.47: "*Team_leader assigns work to Team_member.*" connection on Excel sheet*

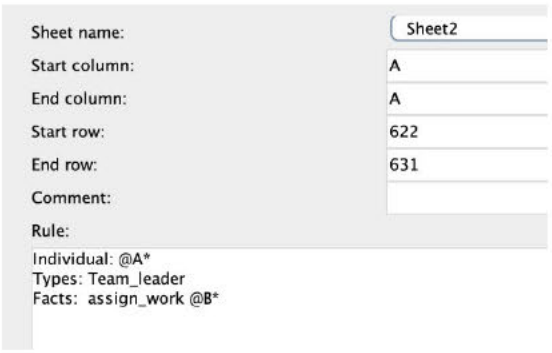


*Figure 6.48: Rule to add "*Team_leader assigns work to Team_member.*" connection*

- Object property: in_stock

in_stock	
Refrigerator	stock_1
Microwave	stock_2
Toaster	stock_3
Electric_Kettle	stock_4
Hair_Dryer	stock_5
Smartphone	stock_6
Portable_Heater	stock_7
LED_TV	stock_8

*Figure 6.49: "*Product is in Stock.*" connection on Excel sheet*

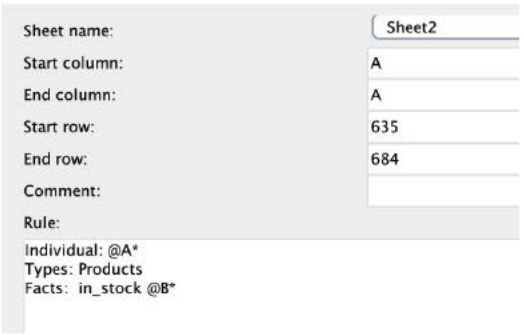


*Figure 6.50: Rule to add "*Product is in Stock.*" connection*

- Object property: has_price

has_price	
SourdoughBread	price_1
Croissant	price_2
ChocolateChipMuffin	price_3
Bagels	price_4
CinnamonRolls	price_5
LemonPoundCake	price_6
RaisinBread	price_7
GarlicBread	price_8

*Figure 6.51: "*Product has Price.*" connection on Excel sheet*

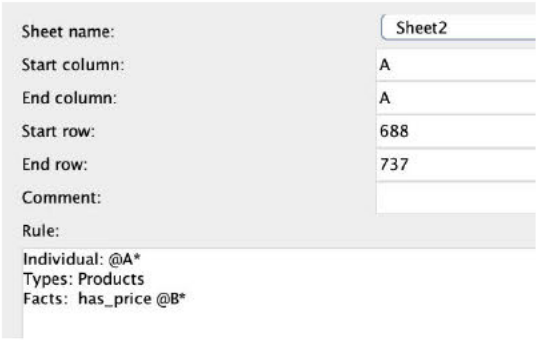


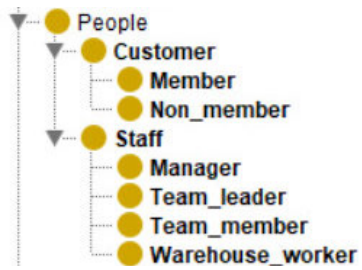
*Figure 6.52: Rule to add "*Product has Price.*" connection*

3.5 Axioms

Reasoning in an ontology can be provided with different techniques and axioms. In Supermarket Warehouse ontology, subclasses and subproperties are used to give meaning and complete the reasoning.

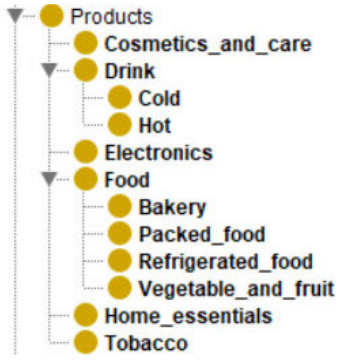
When subclasses are constructed, “is-a relationship” is observed and all organisation is done accordingly. “Is-a relationship” is used when a class A is a subclass of another class B (so, B is a superclass of A) ('is-a', 2024). In that situation it can be said that any object of A “is a” B, also, and that individuals of A carry all specialities of B in addition to A’s specialities.

Below are some examples of subclasses in the Supermarket Warehouse Ontology.



Example 1: Manager is a Staff and also People.

Figure 7.1: People Class and its Subclasses



Example 2: Packed_food is a Food and also a Product.

Figure 7.2: Products Class and its Subclasses

- Complex Descriptions for Classes:

There are various kinds of rules to create limits between classes in Protégé (OWL). In this project, “or” relation is used to build a union of classes when defining domains and ranges of data and object properties. The reason to implement them in this way is because all classes are different from each other in this ontology, and they should not be connected with “and” particle. If “or” is not used specifically, classes added to domains and ranges are connected with “and” as default.

Below are some examples of this rule in the Supermarket Warehouse Ontology.

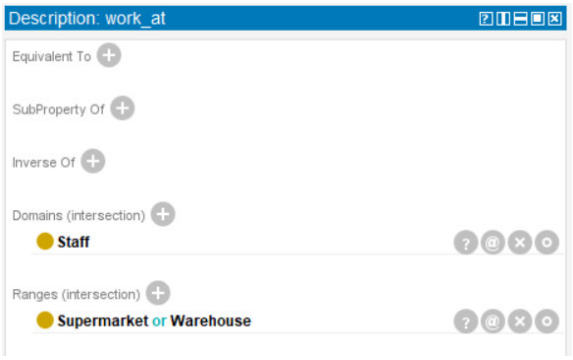


Figure 7.3: work_at Object Property Domains and Ranges

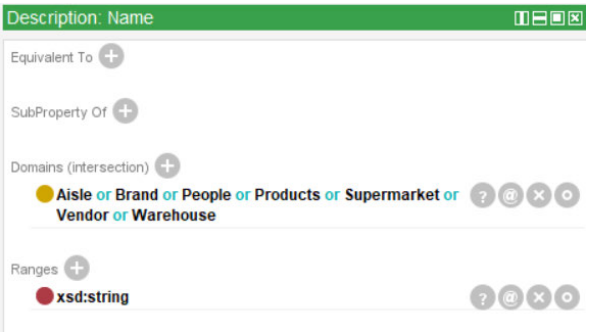


Figure 7.4: Name Data Property Domains and Ranges

- Complex Descriptions for Properties:

In addition to Complex Descriptions for Classes, different regulations can be attached in Protégé (OWL) for properties. In the project, there are three kinds of date occurs as well as First_name and Last_name values that can be used as Name for People individuals. To coordinate this situation, “SubProperty Of” feature is used for Date and Name data properties and screenshots of them are presented below.

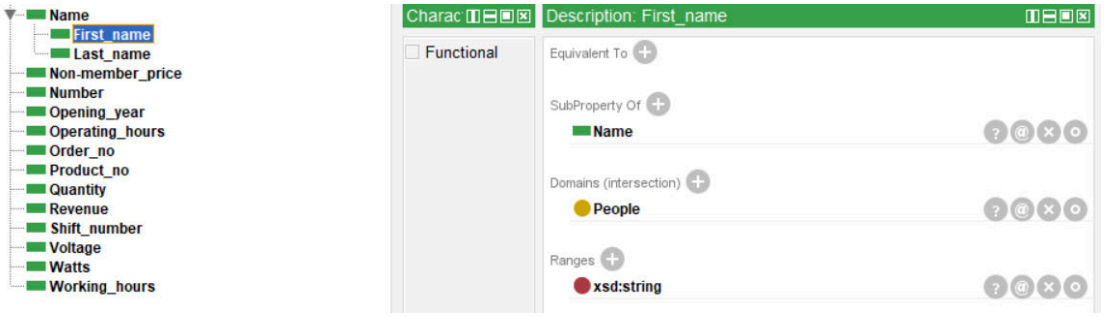


Figure 7.5: First_name Data Property Specialities

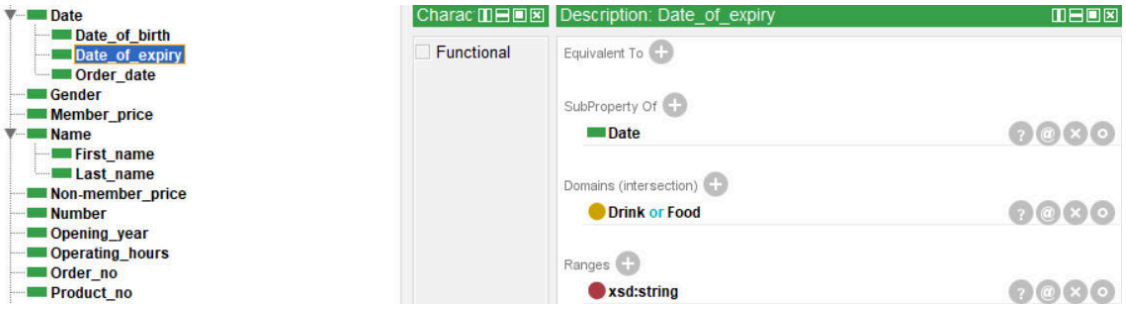


Figure 7.6: Date_of_expiry Data Property Specialities

In Protégé, there is a tool called “Reasoner” to control the structure and validate all regulations if they are appropriate or not according to axioms and rules that are added to the ontology. For the Supermarket Warehouse ontology, this tool is used and there is no error according to it.

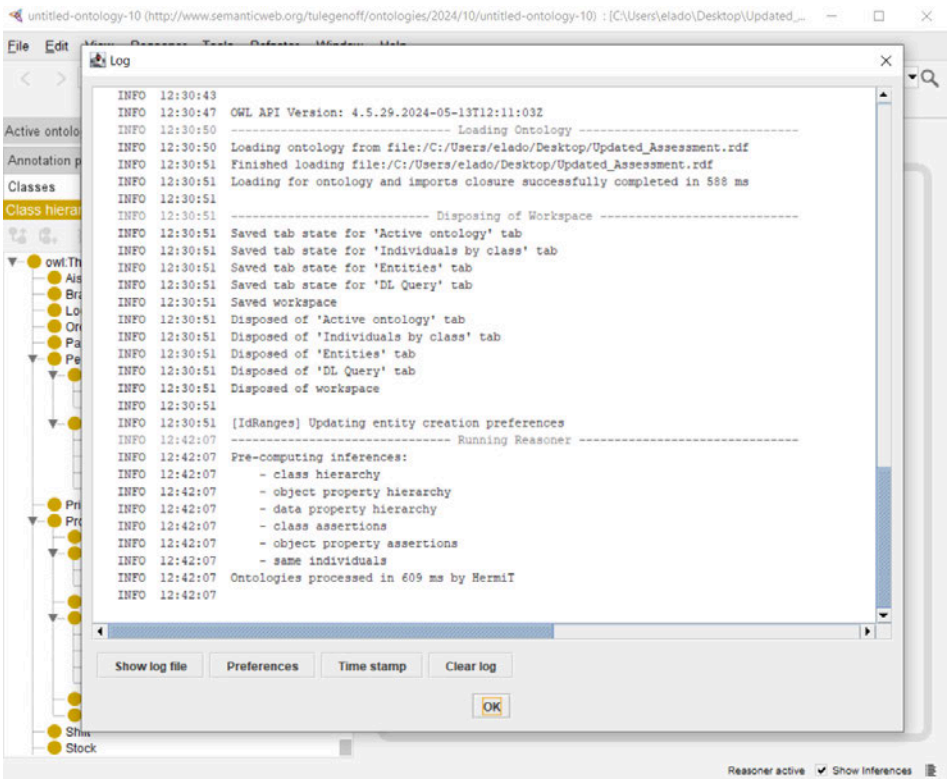


Figure 7.7: Reasoner Output

4. Requirements Mapping

4.1 Mapping Table

Question	Mapping
Which are the 2 cheapest Bakery products?	This question can be answered using Products and Price classes, data properties (non-member price), object properties(has_price).
How many products are sold in supermarket_1?	This question can be answered using Product and Supermarket classes, object properties(sell).
Which product has the fourth most difference between the prices of non-member and member?	This question can be answered using Price and Product classes, data properties (member_price, non-member_price).
What is the total amount of the most expensive shopping that has a value smaller than 50 that a non-member customer did?	This question can be answered using Customer (Non-member) and Payment classes, data properties (amount), object properties(make_payment).
In which aisles does Milk is placed?	This question can be answered using Aisle and Product classes, data properties (name) object properties(displayed_in).

List how many products that have stocks(not zero) according to their categories.	This question can be answered using Stock and Product classes, data properties (quantity), object properties(in_stock).
Member_1 bought which products?	This Question can be answered using Member(Subclass of Customer) and Product classes, Data properties(first_name,last_name, product_no, name) and Object properties will be(buy)
List all the Products of those Vendors who have supplied at least 3 Products to the Supermarket	This Question can be answered using Product and Vendor class, Data Properties(Product name, vendor name) Object Properties(supply)
What Members of Staff work Shift_2 ?	This Question can be answered using Shift and Staff class, Data properties(shift_no, first_name, last_name) Object properties(has_shift)
What are the top 3 most expensive products that belongs to Samsung brand ?	This Question can be answered using Product and Brand class, Data Properties(Product_no, name) Object properties(belong_to)
What is the location of warehouse_1?	This Question can be answered using Warehouse and Location class, Data Properties(name, city, Country) Object Properties(has_location)
List all the Products of those Aisles which displays more than 3 unique items.	This Question can be answered using Vendor and Aisle class, Data Properties(product_name, aisle name) object Properties(displayed_in)
List the supermarkets that opened before 2000?	This question can be answers Supermarket class, Data properties(supermarket_name, opening_year)
Which staff member works in supermarket_4?	This question can be answers using staff class, Data properties(first_name, last_name,supermarket_name) object properties(works_at)
What is the highest revenue of a supermarket?	This question can be answers using Supermarket class, data properties(supermarket_name, supermaket_revenue)

What products are sold by vendor_2?	This question can be answers using products and vendor class, data_properties(product_name, employee_name) object_properties(supply)
What is the most expensive item in a bakery for non-members?	This question can be answered using products, and price class, data_properties(product_name, price) object_properties(displayed_in, has_price)
List orders that have tobacco products?	This question can be answers using tobacco products and orders class, data_properties(product_name, order_no) object_properties(give, include)
What is the average number of staff in the shift?	This question can be answered using Staff and Shift class, data properties (shift_number), object properties(has_shift).
Which warehouse is responsible for providing drinks of Coca-Cola?	This question can be answered using Warehouse, Drinks class and data properties (warehouse name, product name), object properties(has_product, belong_to).
What is the average difference in member and non-member prices of packed foods?	This question can be answered using Products, Price class and data properties (product_no, name, non-member price, member price), object properties(has_price).
How many products in the “Packed food” category are out of stock now?	This question can be answered using Products and Stock class and data properties (product_no, name, quantity), object properties(in_stock).
Which supermarket has the most number of customers?	This question can be answered using Supermarket and Customer class, data properties (name) and object properties (do_shopping_at)
Which “Electronics” products need more than 1000 Watts?	This question can be answered using Electronics class and data properties (name, Watts).

Table 2: Requirement Mapping

5. Evaluation and Use

5.1 Use case 1

Question: In which aisles does Milk is placed?

SPARQL Code and Output:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX market: <http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#>

SELECT ?aisle_name ?no
WHERE {
  ?product a market:Products.
  ?product market:Name ?name.
  ?product market:displayed_in ?aisle.
  ?aisle market:Name ?aisle_name.
  ?aisle market:Number ?no
  FILTER(str(?name)="Milk")
}
```

Table | Response | 3 results in 0.111 seconds | Simple view | Ellipse | Filter query results | Page size: 50

aisle_name	no
1Drinks	"9"^^<http://www.w3.org/2001/XMLSchema#integer>
2Drinks	"21"^^<http://www.w3.org/2001/XMLSchema#integer>
3Drinks	"14"^^<http://www.w3.org/2001/XMLSchema#integer>

Showing 1 to 3 of 3 entries

Figure 8.1: SPARQL query and result for use case 1

Explanation: The query lists aisle names and aisle numbers of all the aisles that contain Milk in it in the dataset.

5.2 Use case 2

Question: What is the total amount of the most expensive shopping that has a value smaller than 50 that a non-member customer did?

SPARQL Code and Output:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX market: <http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#>

SELECT ?a
WHERE {
  ?x a market:Non_member.
  ?x market:make_payment ?payment.
  ?payment market:Amount ?a.
  FILTER(?a < 50)
}
ORDER BY DESC(?a)
LIMIT 1
```

Table | Response | 1 result in 0.056 seconds | Simple view | Ellipse | Filter query results | Page size: 50

a
1"23.5"^^<http://www.w3.org/2001/XMLSchema#float>

Showing 1 to 1 of 1 entry

Figure 8.2: SPARQL query and result for use case 2

Explanation: The query lists the amount(value) of the most expensive shopping that a non-member customer did and has a value smaller than 50 in the dataset.

5.3 Use case 3

Question: List how many products that have stocks(not zero) according to their categories.

SPARQL Code and Output:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX market: <http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#>

SELECT ?sub_classes (COUNT(?stock) AS ?product_count)
WHERE {
  ?sub_classes rdfs:subClassOf+ market:Products.
  ?product a ?sub_classes.
  ?product market:in_stock ?stock.
  ?stock market:Quantity ?q.
  FILTER(?q > 0)
}
GROUP BY ?sub_classes
```

Table | Response | 6 results in 0.181 seconds | Simple view | Ellipse | Filter query results | Page size: 50

	sub_classes	product_count
1	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Drink>	"10"^^<http://www.w3.org/2001/XMLSchema#integer>
2	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Food>	"16"^^<http://www.w3.org/2001/XMLSchema#integer>
3	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Electronics>	"6"^^<http://www.w3.org/2001/XMLSchema#integer>
4	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Tobacco>	"7"^^<http://www.w3.org/2001/XMLSchema#integer>
5	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Bakery>	"10"^^<http://www.w3.org/2001/XMLSchema#integer>
6	<http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Packed_food>	"6"^^<http://www.w3.org/2001/XMLSchema#integer>

Showing 1 to 6 of 6 entries << < 1 > >>

Figure 8.3: SPARQL query and result for use case 3

Explanation: The query lists all level subclasses of “Products” class and product counts inside these subclasses that have a stock bigger than zero in the dataset.

5.4 Use case 4

Question: Which product has the fourth most difference between the prices of non-member and member?

SPARQL Code and Output:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX market: <http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#>

SELECT ?name ?no ?diff
WHERE {
  ?product a market:Products.
  ?product market:Name ?name.
  ?product market:Product_no ?no.
  ?product market:has_price ?p.
  ?p market:Non-member_price ?nmp.
  ?p market:Member_price ?mp
  BIND (?nmp - ?mp AS ?diff)
}
ORDER BY DESC(?diff)
OFFSET 3
LIMIT 1
```

Table | Response | 1 result in 0.351 seconds | Simple view | Ellipse | Filter query results | Page size: 50

	name	no	diff
1	Refrigerator	"5001"^^<http://www.w3.org/2001/XMLSchema#integer>	"50.0"^^<http://www.w3.org/2001/XMLSchema#float>

Showing 1 to 1 of 1 entry << < 1 > >>

Figure 8.4: SPARQL query and result for use case 4

Explanation: The query list the product name and product number of the product that has the fourth most difference between non-member and member price for it along with the amount of the difference.

5.5 Use case 5

Question: List all the Products of those Vendors who have supplied at least 3 Products to the Supermarket.

SPARQL Code and Output:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX market: <http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#>

SELECT ?product_name ?product_no ?vendor_name
WHERE {
  ?vendor a market:Vendor.
    {{
        SELECT ?vendor (COUNT(?product) AS ?product_count)
        WHERE {
          ?vendor a market:Vendor.
          ?vendor market:supply ?product
        }
        GROUP BY ?vendor
        HAVING(COUNT(?product) >= 3)
    }}
  ?vendor market:Name ?vendor_name.
  ?vendor market:supply ?product.
  ?product market:Name ?product_name.
  ?product market:Product_no ?product_no
}
```

Table | Response | 34 results in 0.045 seconds | Simple view ☐ Ellipse ☑ | Filter query results | Page size: 50

product_name	product_no	vendor_name
1Blender	"5010"^^<http://www.w3.org/2001/XMLSchema#integer>	FreshLink Trading
2LED TV	"5008"^^<http://www.w3.org/2001/XMLSchema#integer>	FreshLink Trading
3Toaster	"5003"^^<http://www.w3.org/2001/XMLSchema#integer>	FreshLink Trading
4Beef Jerky	"3009"^^<http://www.w3.org/2001/XMLSchema#integer>	PrimeSupply Partners
5Ready-to-eat Rice	"3007"^^<http://www.w3.org/2001/XMLSchema#integer>	PrimeSupply Partners
6Sourdough Bread	"4001"^^<http://www.w3.org/2001/XMLSchema#integer>	PrimeSupply Partners
7Croissant	"4002"^^<http://www.w3.org/2001/XMLSchema#integer>	MarketLink Distributors
8Frozen Pizza	"3004"^^<http://www.w3.org/2001/XMLSchema#integer>	MarketLink Distributors
9Whole Wheat Bread	"4003"^^<http://www.w3.org/2001/XMLSchema#integer>	MarketLink Distributors
10Blender	"5010"^^<http://www.w3.org/2001/XMLSchema#integer>	SourceHub Supplies
11LED TV	"5008"^^<http://www.w3.org/2001/XMLSchema#integer>	SourceHub Supplies
12Toaster	"5003"^^<http://www.w3.org/2001/XMLSchema#integer>	SourceHub Supplies
13Canned Soup	"3008"^^<http://www.w3.org/2001/XMLSchema#integer>	EliteVendor Group
14Chocolate Chip Muffin	"4004"^^<http://www.w3.org/2001/XMLSchema#integer>	EliteVendor Group
15Fruit Snacks	"3010"^^<http://www.w3.org/2001/XMLSchema#integer>	EliteVendor Group
16Blueberry Muffin	"4010"^^<http://www.w3.org/2001/XMLSchema#integer>	TradeWave Solutions
17Lemon Pound Cake	"4007"^^<http://www.w3.org/2001/XMLSchema#integer>	TradeWave Solutions
18Pipe Tobacco	"1003"^^<http://www.w3.org/2001/XMLSchema#integer>	TradeWave Solutions
19Coca-Cola	"2005"^^<http://www.w3.org/2001/XMLSchema#integer>	Logistics Coordinator
20Lemonade	"2004"^^<http://www.w3.org/2001/XMLSchema#integer>	Logistics Coordinator
21Peach Iced Tea	"2006"^^<http://www.w3.org/2001/XMLSchema#integer>	Logistics Coordinator
22Granola Bars	"3006"^^<http://www.w3.org/2001/XMLSchema#integer>	Distribution Supervisor
23Grape Juice	"2003"^^<http://www.w3.org/2001/XMLSchema#integer>	Distribution Supervisor
24Orange Juice	"2001"^^<http://www.w3.org/2001/XMLSchema#integer>	Distribution Supervisor
25Energy Drink	"2009"^^<http://www.w3.org/2001/XMLSchema#integer>	Supply Chain Manager
26Milk	"2008"^^<http://www.w3.org/2001/XMLSchema#integer>	Supply Chain Manager
27Sparkling Water	"2010"^^<http://www.w3.org/2001/XMLSchema#integer>	Supply Chain Manager
28Cigars	"1002"^^<http://www.w3.org/2001/XMLSchema#integer>	Operations Support Specialist
29Cinnamon Rolls	"4006"^^<http://www.w3.org/2001/XMLSchema#integer>	Operations Support Specialist
30Microwave	"5002"^^<http://www.w3.org/2001/XMLSchema#integer>	Operations Support Specialist
31Apple Juice	"2002"^^<http://www.w3.org/2001/XMLSchema#integer>	Inventory Analyst
32Bagels	"4005"^^<http://www.w3.org/2001/XMLSchema#integer>	Inventory Analyst
33Frozen Pizza	"3004"^^<http://www.w3.org/2001/XMLSchema#integer>	Inventory Analyst
34Sourdough Bread	"4001"^^<http://www.w3.org/2001/XMLSchema#integer>	Inventory Analyst

Showing 1 to 34 of 34 entries << < 1 > >>

Figure 8.5: SPARQL query and result for use case 5

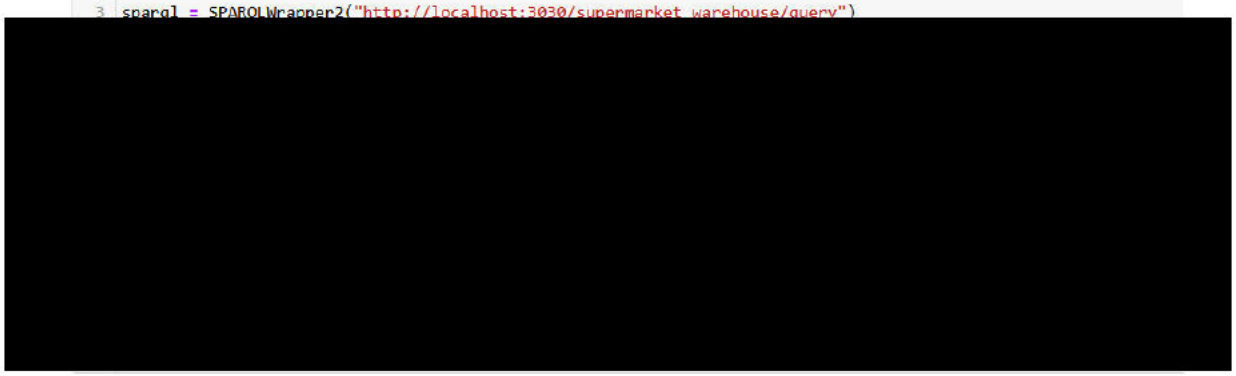
Explanation: The query lists product name and product number of all products along with the name of the vendor that supplies these products to the supermarkets, only if that vendor supplies at least 3 products in the dataset.

6. Implementation

6.1 Connection to the Dataset:

Connection to the SPARQL endpoint/dataset is successfully done. To prove this, one example query is used to get all Supermarket names in the dataset. Query is written on Jupyter notebook and the result is received.

```
In [24]:  1 from SPARQLWrapper import SPARQLWrapper2
          2
          3 sparql = SPARQLWrapper2("http://localhost:3030/supermarket_warehouse/query")
```



```
SPAR
Waitrose
Sainsbury's Superstore
Lidl
Marks & Spencer Foodhall
Aldi
Lidl
Co-op
Waitrose
Iceland
Tesco Extra
Farmfoods
Asda Supercentre
Morrisons
Co-op
```

Figure 9.1: Code to Connect to Dataset and the Received Result

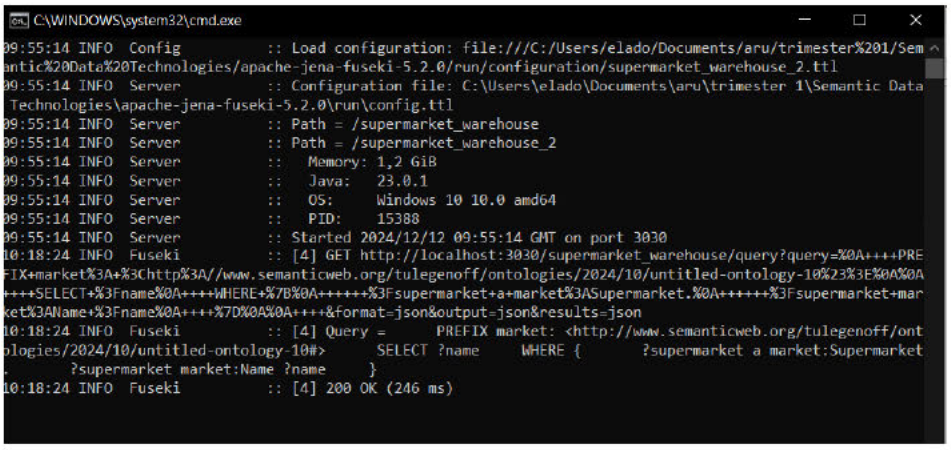


Figure 9.2: "200 OK" Message Received on CMD

6.2 NLP Techniques:

Natural Language Processing (NLP) is a working area that aims to give computers the ability to understand natural languages ('Natural language processing', 2024). This topic involves studies for computer science, artificial intelligence and machine learning as well as linguistics. Inside NLP, there are various techniques that are used by researchers. In this project, four of them is operated:

1- Normalization: Normalization means removing any additional things from the text (e.g. punctuations, white spaces)

2-Lemmatization or Stemming: These two approaches are applied to find the root of the words inside the text.

Lemmatization and Stemming have two different algorithms inside even though the aim is the same. Lemmatization is slower but has higher accuracy. Stemming is faster but does not have 100% accuracy.

In this project, because the database is rather small, lemmatization is selected and applied.

3-Stop Word Removal: This step removes words in the text that are used to connect but does not have a function for the process (e.g. a, the, list). As a result, the NLP process will be faster.

4-Named Entity Recognition (NER): NER is a process that is used to recognise named entities. Named entity is the actual subject/entity of a sentence (TechTarget, 2024). It can be a name, event, product, language and a subject similar to these.
In the process, there are two main steps:

1-Identification: Identifying if there is a named entity and finding its location.
2-Classification: Attaching a category/type label to these named entities.

In this project, identification step is used directly and not the classification. Named entities that are found are taken as individuals. The logic behind it is that if a string is not a named entity, it is certainly not an individual. For the classification part, the individuals of the ontology is the searching place.

Bag of words (BoW) is used to extract features from the text. It involves finding a representation of the word used in the text inside a collection .

Stanza (Stanford NER) is selected as the library to use for the NER. In addition, Bag of Words method in the code involves a list of classes, object properties and data properties as well as their substitutes that can be used in the NL questions ('Bag-of-words model', 2024).

6.3 Overall Code:

To give an overall look at the code, steps are listed below.

1- Input text is taken from the user.

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

Figure 9.3: Code that is taking user input

2- NLP techniques are implemented.

a- Bag of keywords are implemented as lists.

```
object_property_list = [['assign_work', 'assign work', 'assigned work', 'work assigned by', 'give work', 'given work',
                         'work given by'], ['belong_to', 'belong to'], ['buy', 'Buy', 'bought', 'Bought'],
                        ['come_to', 'come to', 'come', 'came', 'came to'], ['coordinate_with', 'coordinate with', 'work with
                        ['deal_with', 'deal with', 'help', 'helps', 'helped', 'dealt with', 'deals with', 'deal', 'deals',
                         'dealt'], ['displayed_in', 'displayed in', 'placed', 'placed in', 'found in', 'found', 'find'],
                        ['do_shopping_at', 'do shopping at', 'shopping', 'shopping at', 'do shopping'], ['give', 'gives',
                         'gave'], ['handle', 'handles', 'handled'], ['has_aisle', 'has aisle', 'have aisle', 'have aisles'],
                        ['has_location', 'has location', 'located in', 'located'], ['has_price', 'has price', 'have price',
                         'have prices', 'has prices'], ['has_product', 'has product', 'have product', 'have products',
                         'has products'], ['has_shift', 'has shift', 'in shift'], ['in_stock', 'in stock'], ['include',
                         'includes', 'included'], ['make_payment', 'make payment', 'made payment', 'made payments',
                         'make payments'], ['manage', 'manage'], ['provide_for', 'provide for', 'provided for', 'provides fo
                        ['sell', 'sold'], ['stock', 'in stock', 'Stock'], ['supply', 'Supply'], ['work_at', 'work at',
                         'Work at', 'work', 'Work']]
```

Figure 9.4: Bag of words for Object Properties

```
data_property_list = [['Age', 'age', 'old'], ['Amount', 'amount', 'total amount', 'Total amount', 'Total Amount'],
                      ['Calorie', 'calorie', 'calories', 'Calories'], ['City', 'city'], ['Country', 'country'],
                      ['Date', 'date'], ['Date_of_birth', 'date of birth', 'Date of birth'],
                      ['Date_of_expiry', 'date of expiry', 'Date of expiry'], ['Order_date', 'order date', 'Order date'],
                      ['Gender', 'gender'], ['Member_price', 'member price', 'Member price', 'price of member',
                       'Price of member'], ['Name', 'name'], ['Non-member_price', 'non member price', 'Non member price',
                       'non-member price', 'Non-member price', 'price of non member', 'Price of non member',
                       'price of non-member', 'Price of non-member'], ['Number', 'number', 'no', 'No', 'NO'],
                      ['Opening_year', 'year', 'Year','opening year', 'Opening year'], ['Operating_hours', 'Operating hours'
                       'operating hours', 'Operating hour','operating hour'], ['Order_no', 'Order no', 'order no',
                       'Order number', 'order number'], ['Product_no', 'Product no', 'product no', 'Product number',
                       'product number'], ['Quantity', 'quantity'], ['Revenue', 'revenue', 'revenues', 'Revenues'],
                      ['Shift_number', 'shift number', 'Shift number', 'Shift no', 'shift no'], ['Voltage', 'voltage'],
                      ['Watts', 'watts', 'watt', 'Watt'],
                      ['Working_hours', 'working hours', 'Working hours', 'working hour', 'Working hour']]
```

Figure 9.5: Bag of words for Data Properties

```
class_list = [['Aisle', 'aisle', 'Aisles', 'aisles'], ['Brand', 'brand', 'brands', 'Brands'],
              ['Location', 'location', 'place', 'Locations', 'locations', 'places', 'Places'],
              ['Order', 'order', 'Orders', 'orders'],
              ['Payment', 'payment', 'Payments', 'payments'], ['People', 'people', 'Person', 'person'],
              ['Customer', 'customer', 'Customers', 'customers'],
              ['Staff', 'staff', 'Worker', 'worker'],
              ['Member', 'member', 'Member customer', 'member customer', 'Members', 'members', 'member customers',
               'Member customers'],
              ['Non_member', 'Non member', 'non member', 'Non-member', 'non-member', 'Non member customer',
               'non member customer', 'Non-member customer', 'non-member customer', 'Non member customers',
               'non member customers', 'non-member customers', 'Non-member customers'],
              ['Manager', 'manager', 'Managers', 'managers'], ['Team_leader', 'Team leader', 'Team Leader',
               'team leader', 'team_leader', 'Team leaders', 'Team Leaders', 'team leaders', 'team_leaders'],
              ['Team_member', 'Team member', 'Team Member', 'team member', 'team_member', 'Team members',
               'team members', 'Team Members'],
              ['Warehouse_worker', 'Warehouse worker', 'warehouse worker', 'Warehouse Worker', 'warehouse_worker'
               'Warehouse workers', 'Warehouse Workers', 'warehouse workers'],
              ['Price', 'price',  'total price', 'Total price', 'Total Price', 'Prices', 'prices'],
              ['Products', 'products', 'Product', 'product'], ['Cosmetics_and_care', 'cosmetics and care',
               'Cosmetics and Care', 'Cosmetics and care', 'Cosmetics', 'Cosmetic', 'cosmetics', 'cosmetic', 'Care', 'care']
              ['Drink', 'drink', 'Drinks', 'drinks', 'hot drink', 'cold drink', 'Hot drink', 'Cold drink', 'Hot drinks',
               'Cold drinks', 'hot drinks', 'cold drinks'], ['Electronics', 'electronics', 'Electronic', 'electronic',
                'Electonic product', 'Electronic products', 'electronic products', 'electronic product'],
              ['Food', 'Foods', 'food', 'foods', 'meal', 'Meal', 'Meals', 'meals', 'dish', 'Dish', 'Dishes', 'dishes'],
              ['Bakery', 'bakery', 'Bakery products','bakery products', 'bakery product', 'Bakery product', 'Bakery Product'
               'Bakery Products'],
              ['Packed_food', 'Packed food', 'packed food', 'packed_food', 'packed foods', 'Packed foods', 'frozen food',
               'Frozen food', 'frozen foods', 'Frozen foods', 'Canned food', 'canned food', 'Canned foods', 'canned foods'],
              ['Refrigerated_food', 'refrigerated food', 'Refrigerated food', 'cold food', 'Cold food', 'refrigerated foods'
               'Refrigerated foods', 'cold foods', 'Cold foods'],
             ['Vegetable_and_fruit', 'Vegetable and Fruit', 'vegetable and fruit', 'Vegetable', 'vegetable', 'Fruit', 'fruit
              'Vegetables and Fruits', 'vegetables and fruits', 'Vegetables', 'vegetables', 'Fruits', 'fruits'],
              ['Home_essentials', 'Home essentials', 'Home Essentials', 'home essentials', 'home essential', 'Home essential
              ['Tobacco', 'tobacco', 'Tobacco product', 'Tobacco products', 'tobacco product', 'tobacco products'],
              ['Shift', 'shift', 'shifts', 'Shifts'], ['Stock', 'stock', 'stocks', 'Stocks'], ['Supermarket', 'supermarket',
               'Market', 'market', 'Grocery', 'grocery', 'Supermarkets', 'supermarkets','Markets', 'markets', 'Groceries',
               'groceries'], ['Vendor', 'vendor', 'Vendors', 'vendors'], ['Warehouse', 'warehouse', 'Warehouses', 'warehouse
```

Figure 9.6: Bag of words for Classes

b- Functions that are doing the tokenization, normalization, removing stop words, getting lemmas, finding object, data properties and classes are written.

```
#Getting tokens from the text

def tokenize(text):
    tokens = word_tokenize(text)
    return tokens
```

Figure 9.7: Code for Tokenization

```python
#Normalization

def normalize(text):
    #Remove all punctuation
    processed_text =  re.sub(f"[{re.escape(punctuation)}]", "", text)

    #Create the text again with single space between each word (remove 1+ spaces)
    processed_text = " ".join(processed_text.split())

    return processed_text
```

Figure 9.8: Code for Normalization



Figure 9.9: Code for Removing Stop Words

```python
#Lemmatization

def get_lemmas_from_tokens(tokens):
    lemma = WordNetLemmatizer()
    lemmas = []

    for token in tokens:
        temp = lemma.lemmatize(token, 'n')
        lemmas.append(lemma.lemmatize(temp, 'v'))
    return lemmas
```

Figure 9.10: Code for Lemmatization

```python
#Search the string in classes

def get_class_name(word):
    found_class_list = []
```

[redacted]

```python
#Search the string in data and object properties

def get_object_prop_name(word):
    found_prop_list = []
```

[redacted]

```python
def get_data_prop_name(word):
    found_prop_list = []
```

[redacted]

Figure 9.11: Code for Finding Classes, Data and Object Properties

3- Techniques that are written are applied to the text taken by the user in the given order:

- Named entity recognition (NER) is used to find individuals in the text if there is any. These strings are removed from the text to make next steps faster.

```python
#1- Named Entity Recognition (NER)

#stanza.download('en')
#nlp = stanza.Pipeline('en')

nlp = stanza.Pipeline("en",  download_method=stanza.DownloadMethod.NONE)
doc = nlp(text)

#if there is an entity, remove it from the text.

new_text = text

if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

Figure 9.12: NER Process

- Normalization is done.
- Tokens are constituted from the normalized text.
- Stop words are removed from the tokens.
- Lemmatization is applied to the remaining tokens.
- Classes, data properties and object properties are found from lemmas.

```
#Process the text

new_text = normalize(new_text)
tokens = tokenize(new_text)
#print(tokens)

clean_tokens = remove_stop_words(tokens)
print("Clean Tokens: ", clean_tokens)

lemmatized_words = get_lemmas_from_tokens(clean_tokens)
print("Lemmatized words: ", lemmatized_words)
```



Figure 9.13: Process that is Applied to Text

4- Connection to the Fuseki server is performed. SPARQL queries for different types of questions are implemented inside one function with the help of if-else statements. Five different types of questions are written to compose a wide ranged method. "SPARQLWrapper2" inside the "SPARQLWrapper" wrapper is used to compose queries.

Questions used for this part of the project:

- List all revenues of supermarkets. (Class: Supermarket, Data property: Revenue)
- List supermarkets and their locations. (Classes: Supermarket, Location)
- Who is working at Aldi? (Object property: work_at, Individual: Aldi)
- What is the brand of Orange Juice? (Class: Brand, Individual: Orange Juice)
- How many calories is Blueberry Muffin? (Data property: Calorie, Individual:Blueberry Muffin)

(Note: In the screenshots below, sometimes one line below in the cell is used in the code to cover all parts of code written in the screenshot.)

Connecting to Fuseki endpoint with sending a request

```
#pip install sparqlwrapper
```

```
from SPARQLWrapper import SPARQLWrapper2

def send_query(classes_list, data_properties_list, object_properties_list, entities_list):
    sparql = SPARQLWrapper2("http://localhost:3030/supermarket_warehouse/query")

    #One class, one data property
    if (len(classes_list) == 1) and (len(data_properties_list) == 1) and (len(entities_list) == 0)
```

Figure 9.14: Function that is connecting to the Fuseki Server (Database)

```
from SPARQLWrapper import SPARQLWrapper2

def send_query(classes_list, data_properties_list, object_properties_list, entities_list):
    sparql = SPARQLWrapper2("http://localhost:3030/supermarket_warehouse/query")
```

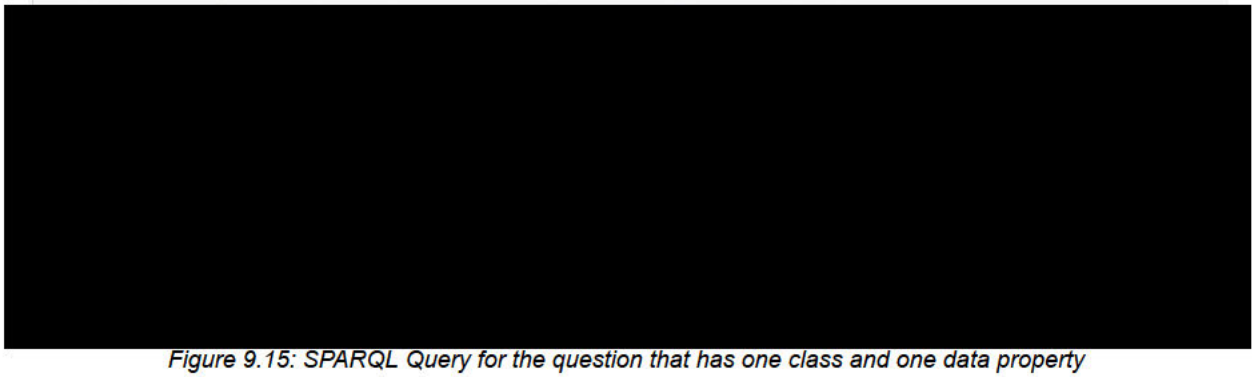


Figure 9.15: SPARQL Query for the question that has one class and one data property

```
    #One class, one entity
    elif (len(classes_list) == 1) and (len(data_properties_list) == 0) and (len(entities_list) == 1) and
```

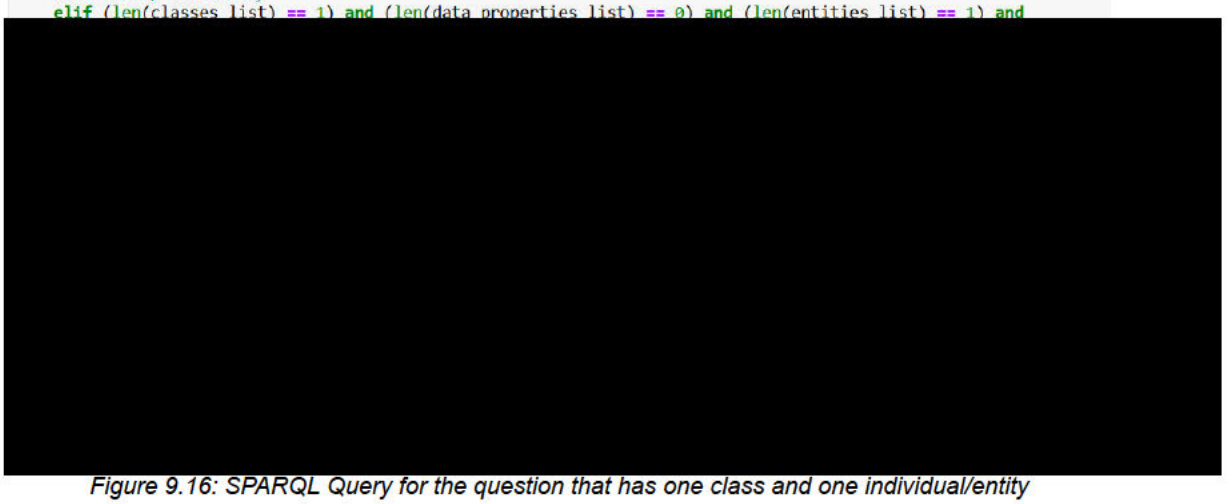


Figure 9.16: SPARQL Query for the question that has one class and one individual/entity

```
#One object property, one entity
elif (len(classes_list) == 0) and (len(data_properties_list) == 0) and (len(entities_list) == 1) and
```

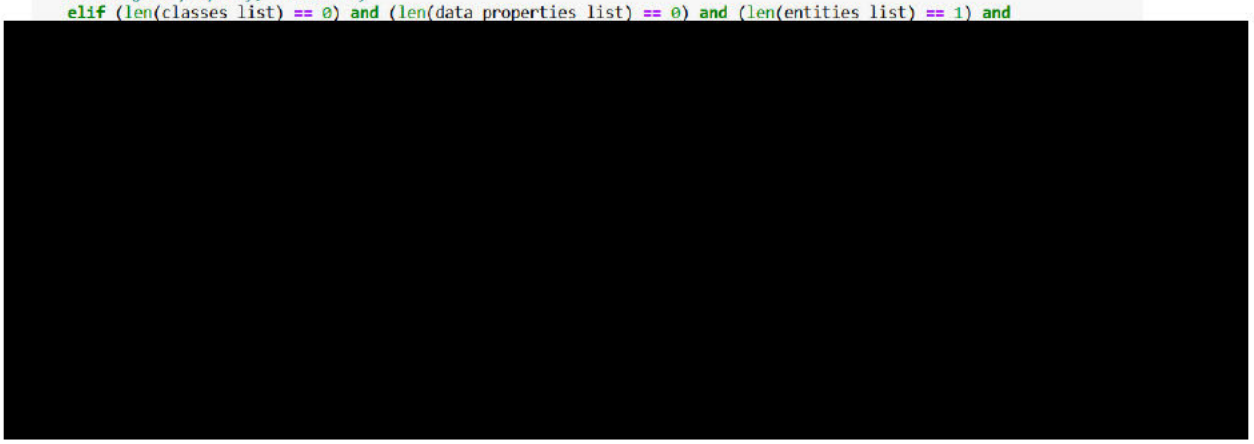


Figure 9.17: SPARQL Query for the question that has one object property and one individual/entity

```
#One data property, one entity
elif (len(classes_list) == 0) and (len(data_properties_list) == 1) and (len(entities_list) == 1) and
(len(object_properties_list) == 0):
```



Figure 9.18: SPARQL Query for the question that has one data property and one individual/entity

```
#Two classes
elif (len(classes_list) == 2) and (len(data_properties_list) == 0) and (len(entities_list) == 0) and
(len(object_properties_list) == 0):
```



Figure 9.19: SPARQL Query for the question that has two classes

```
    data = sparql.query().bindings
    return data
```

Figure 9.20: End of the function, query is created and result is returned from the function

5- Result is achieved using the “send_query” method and it is shown to the user.

```
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

Figure 9.21: End of the function, query is created and result is returned from the function

6.4 Code Validation:

Question 1- List all revenues of supermarkets.

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

```
What is your question?     List all revenues of supermarkets.
```

Figure 9.22: Taking user input for Q1

```
new_text = text

if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

```
2024-12-12 18:13:37 INFO: Loading these models for language: en (English):
============================================
| Processor    | Package                   |
--------------------------------------------
| tokenize     | combined                  |
| mwt          | combined                  |
| pos          | combined_charlm           |
| lemma        | combined_nocharlm         |
| constituency | ptb3-revised_charlm       |
| depparse     | combined_charlm           |
| sentiment    | sstplus_charlm            |
| ner          | ontonotes-ww-multi_charlm |
============================================

2024-12-12 18:13:37 INFO: Using device: cpu
2024-12-12 18:13:37 INFO: Loading: tokenize
2024-12-12 18:13:37 INFO: Loading: mwt
2024-12-12 18:13:37 INFO: Loading: pos
2024-12-12 18:13:38 INFO: Loading: lemma
2024-12-12 18:13:38 INFO: Loading: constituency
2024-12-12 18:13:38 INFO: Loading: depparse
2024-12-12 18:13:39 INFO: Loading: sentiment
2024-12-12 18:13:39 INFO: Loading: ner
2024-12-12 18:13:41 INFO: Done loading processors!
```

Figure 9.23: Function output of NER process for text

```
print("\nClasses :", class_names)
print("Data properties :", data_prop_names)
print("Object properties :", object_prop_names)


print("\nEntities: ")
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    print(doc.entities)
#   print(doc.entities[0].text)
#   print(doc.entities[0].type)
```

```
Clean Tokens:  ['revenues', 'supermarkets']
Lemmatized words:  ['revenue', 'supermarket']

Classes : ['Supermarket']
Data properties : ['Revenue']
Object properties : []

Entities:
```

Figure 9.24: Function output of NLP process for text

```
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

```
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_11 ,  3.7
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_15 ,  8.12
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_2 ,  33.8
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_13 ,  10.6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_10 ,  6.6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5 ,  17.8
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_6 ,  10.6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_7 ,  13.1
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_8 ,  8.1
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_9 ,  4.1
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_1 ,  65.7
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_12 ,  1.0
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_3 ,  23.3
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_4 ,  21.0
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_14 ,  13.11
```

Figure 9.25: Function output of SPARQL query for Q1

Question 2- List supermarkets and their locations.

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

```
What is your question?     List supermarkets and their locations.
```

Figure 9.26: Taking user input for Q2

```
new_text = text

if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

```
2024-12-12 18:21:36 INFO: Loading these models for language: en (English):
============================================
| Processor    | Package                   |
--------------------------------------------
| tokenize     | combined                  |
| mwt          | combined                  |
| pos          | combined_charlm           |
| lemma        | combined_nocharlm         |
| constituency | ptb3-revised_charlm       |
| depparse     | combined_charlm           |
| sentiment    | sstplus_charlm            |
| ner          | ontonotes-ww-multi_charlm |
============================================

2024-12-12 18:21:36 INFO: Using device: cpu
2024-12-12 18:21:36 INFO: Loading: tokenize
2024-12-12 18:21:36 INFO: Loading: mwt
2024-12-12 18:21:36 INFO: Loading: pos
2024-12-12 18:21:37 INFO: Loading: lemma
2024-12-12 18:21:37 INFO: Loading: constituency
2024-12-12 18:21:37 INFO: Loading: depparse
2024-12-12 18:21:38 INFO: Loading: sentiment
2024-12-12 18:21:38 INFO: Loading: ner
2024-12-12 18:21:39 INFO: Done loading processors!
```

Figure 9.27: Function output of NER process for text

```
print("\nClasses :", class_names)
print("Data properties :", data_prop_names)
print("Object properties :", object_prop_names)


print("\nEntities: ")
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    print(doc.entities)
#   print(doc.entities[0].text)
#   print(doc.entities[0].type)
```

```
Clean Tokens:  ['supermarkets', 'locations']
Lemmatized words:  ['supermarket', 'location']

Classes : ['Supermarket', 'Location']
Data properties : []
Object properties : []

Entities:
```

Figure 9.28: Function output of NLP process for text

```
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

```
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_11 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_3
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_15 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_3
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_2 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_2
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_13 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_14
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_10 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_14
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_6 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_7 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_6
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_8 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_8
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_9 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_9
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_1 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_1
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_12 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_3
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_3 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_3
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_4 ,  http://www.semanticweb.org/
tulegenoff/ontologies/2024/10/untitled-ontology-10#location_14
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_14 ,  http://www.semanticweb.or
g/tulegenoff/ontologies/2024/10/untitled-ontology-10#location_3
```

Figure 9.29: Function output of SPARQL query for Q2

Question 3- Who is working at Aldi?

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

```
What is your question?     Who is working at Aldi?
```

Figure 9.30: Taking user input for Q3

```python
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

```
2024-12-12 18:25:21 INFO: Loading these models for language: en (English):
=========================================
| Processor    | Package                |
-----------------------------------------
| tokenize     | combined               |
| mwt          | combined               |
| pos          | combined_charlm        |
| lemma        | combined_nocharlm      |
| constituency | ptb3-revised_charlm    |
| depparse     | combined_charlm        |
| sentiment    | sstplus_charlm         |
| ner          | ontonotes-ww-multi_charlm |
=========================================

2024-12-12 18:25:21 INFO: Using device: cpu
2024-12-12 18:25:21 INFO: Loading: tokenize
2024-12-12 18:25:21 INFO: Loading: mwt
2024-12-12 18:25:21 INFO: Loading: pos
2024-12-12 18:25:21 INFO: Loading: lemma
2024-12-12 18:25:21 INFO: Loading: constituency
2024-12-12 18:25:22 INFO: Loading: depparse
2024-12-12 18:25:23 INFO: Loading: sentiment
2024-12-12 18:25:23 INFO: Loading: ner
2024-12-12 18:25:24 INFO: Done loading processors!
```

Figure 9.31: Function output of NER process for text

```python
print("\nClasses :", class_names)
print("Data properties :", data_prop_names)
print("Object properties :", object_prop_names)


print("\nEntities: ")
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    print(doc.entities)
#   print(doc.entities[0].text)
#   print(doc.entities[0].type)
```

```
Clean Tokens:  ['Who', 'working']
Lemmatized words:  ['Who', 'work']

Classes : []
Data properties : []
Object properties : ['work_at']

Entities:
[{
  "text": "Aldi",
  "type": "ORG",
  "start_char": 18,
  "end_char": 22
}]
```

Figure 9.32: Function output of NLP process for text

```python
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

```
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#manager_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#manager_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#team_leader_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#team_leader_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#team_member_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#team_member_5 ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#supermarket_5
```

Figure 9.33: Function output of SPARQL query for Q3

Question 4- What is the brand of Orange Juice?

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

```
What is your question?     What is the brand of Orange Juice?
```

Figure 9.34: Taking user input for Q4

```
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

```
2024-12-12 18:30:21 INFO: Loading these models for language: en (English):
=============================================
| Processor    | Package                    |
---------------------------------------------
| tokenize     | combined                   |
| mwt          | combined                   |
| pos          | combined_charlm            |
| lemma        | combined_nocharlm          |
| constituency | ptb3-revised_charlm        |
| depparse     | combined_charlm            |
| sentiment    | sstplus_charlm             |
| ner          | ontonotes-ww-multi_charlm  |
=============================================

2024-12-12 18:30:21 INFO: Using device: cpu
2024-12-12 18:30:21 INFO: Loading: tokenize
2024-12-12 18:30:21 INFO: Loading: mwt
2024-12-12 18:30:21 INFO: Loading: pos
2024-12-12 18:30:22 INFO: Loading: lemma
2024-12-12 18:30:22 INFO: Loading: constituency
2024-12-12 18:30:23 INFO: Loading: depparse
2024-12-12 18:30:23 INFO: Loading: sentiment
2024-12-12 18:30:23 INFO: Loading: ner
2024-12-12 18:30:24 INFO: Done loading processors!
```

Figure 9.35: Function output of NER process for text

```
print("\nClasses :", class_names)
print("Data properties :", data_prop_names)
print("Object properties :", object_prop_names)


print("\nEntities: ")
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    print(doc.entities)
#   print(doc.entities[0].text)
#   print(doc.entities[0].type)
```

```
Clean Tokens:  ['brand']
Lemmatized words:  ['brand']

Classes : ['Brand']
Data properties : []
Object properties : []

Entities:
[{
  "text": "Orange Juice",
  "type": "PRODUCT",
  "start_char": 21,
  "end_char": 33
}]
```

Figure 9.36: Function output of NLP process for text

```
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

```
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#Innocent ,  http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#OrangeJuice
```

Figure 9.37: Function output of SPARQL query for Q4

Question 5- How many calories is Blueberry Muffin?

```
#1- Reading the input text from user

text = input("What is your question?     ")
```

```
What is your question?     How many calories is Blueberry Muffin?
```

Figure 9.38: Taking user input for Q5

```
new_text = text

if hasattr(doc, 'entities') and len(doc.entities) > 0:
    for i in range(len(doc.entities)):
        entity = doc.entities[i].text
        new_text = new_text.replace(entity, '')
```

```
2024-12-12 18:31:59 INFO: Loading these models for language: en (English):
============================================
| Processor    | Package                   |
--------------------------------------------
| tokenize     | combined                  |
| mwt          | combined                  |
| pos          | combined_charlm           |
| lemma        | combined_nocharlm         |
| constituency | ptb3-revised_charlm       |
| depparse     | combined_charlm           |
| sentiment    | sstplus_charlm            |
| ner          | ontonotes-ww-multi_charlm |
============================================

2024-12-12 18:31:59 INFO: Using device: cpu
2024-12-12 18:31:59 INFO: Loading: tokenize
2024-12-12 18:31:59 INFO: Loading: mwt
2024-12-12 18:31:59 INFO: Loading: pos
2024-12-12 18:31:59 INFO: Loading: lemma
2024-12-12 18:31:59 INFO: Loading: constituency
2024-12-12 18:32:01 INFO: Loading: depparse
2024-12-12 18:32:02 INFO: Loading: sentiment
2024-12-12 18:32:04 INFO: Loading: ner
2024-12-12 18:32:06 INFO: Done loading processors!
```

Figure 9.39: Function output of NER process for text

```
print("\nClasses :", class_names)
print("Data properties :", data_prop_names)
print("Object properties :", object_prop_names)


print("\nEntities: ")
if hasattr(doc, 'entities') and len(doc.entities) > 0:
    print(doc.entities)
#   print(doc.entities[0].text)
#   print(doc.entities[0].type)
```

```
Clean Tokens:  ['How', 'many', 'calories']
Lemmatized words:  ['How', 'many', 'calorie']

Classes : []
Data properties : ['Calorie']
Object properties : []

Entities:
[{
  "text": "Blueberry Muffin",
  "type": "PRODUCT",
  "start_char": 21,
  "end_char": 37
}]
```

Figure 9.40: Function output of NLP process for text

```
result = send_query(class_names, data_prop_names, object_prop_names, doc.entities)
for r in result:
    print(r['x'].value , ", ",r['y'].value)
```

```
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#BlueberryMuffin ,  320
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#BlueberryMuffin ,  320
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#BlueberryMuffin ,  320
http://www.semanticweb.org/tulegenoff/ontologies/2024/10/untitled-ontology-10#BlueberryMuffin ,  320
```

Figure 9.41: Function output of SPARQL query for Q5

7. Conclusion and Future Work

To give a conclusion, the Supermarket Warehouse project is covering all topics that are mentioned in this module. The ontology model developed for this assessment is just a small part of real life. To make a more inclusive ontology, several classes can be added, current classes extent can be developed and more relations can be defined by creating additional data properties and object properties as well as more individuals. Also, queries that are covered in the NLP part can be extended while more high level code can be written for NLP using other techniques.

8. References

'Ontology components' (2024) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Ontology_components (Accessed: 7 December 2024).

https://openai.com/index/chatgpt/ (2022) (Accessed: November 2024).

'is-a' (2024) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Is-a (Accessed: 9 December 2024).

'Natural language processing' (2024) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Natural_language_processing (Accessed: 12 December 2024).

TechTarget (2024) *What Is Named Entity Recognition (NER)?*. Available at: https://www.techtarget.com/whatis/definition/named-entity-recognition-NER (Accessed: 12 December 2024).

'Bag-of-words model' (2024) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Bag-of-words_model (Accessed: 12 December 2024).